

UD3.1 – Introducción a Android

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

1.- Introducción

La experiencia de uso en las aplicaciones móviles y en las aplicaciones de escritorio es muy diferente.

En una aplicación móvil la interacción del usuario **no empieza siempre en el mismo lugar**.

Por ejemplo:

- Si se abre la aplicación de correo electrónico lo más habitual es que se muestre la bandeja de entrada o la última ventana abierta en la aplicación.
- Si se está navegando por una página web y se pulsa el botón de contacto para enviar un mail, es probable que se abra la aplicación de correo pero directamente para escribir el correo electrónico.

1.- Introducción

En las aplicaciones de escritorio el punto de inicio de la aplicación es el método **main** que incluye el código que se ejecuta al iniciar la aplicación.

En las aplicaciones móviles no se puede realizar de la misma manera debido a que se puede iniciar una aplicación en diferentes puntos de la misma como se ha visto en el punto anterior.

Es por ello que el **ciclo de vida** de las aplicaciones móviles es distinto al de las aplicaciones de escritorio.

2.- Activity

La clase **Activity** es un componente crucial en una aplicación Android.

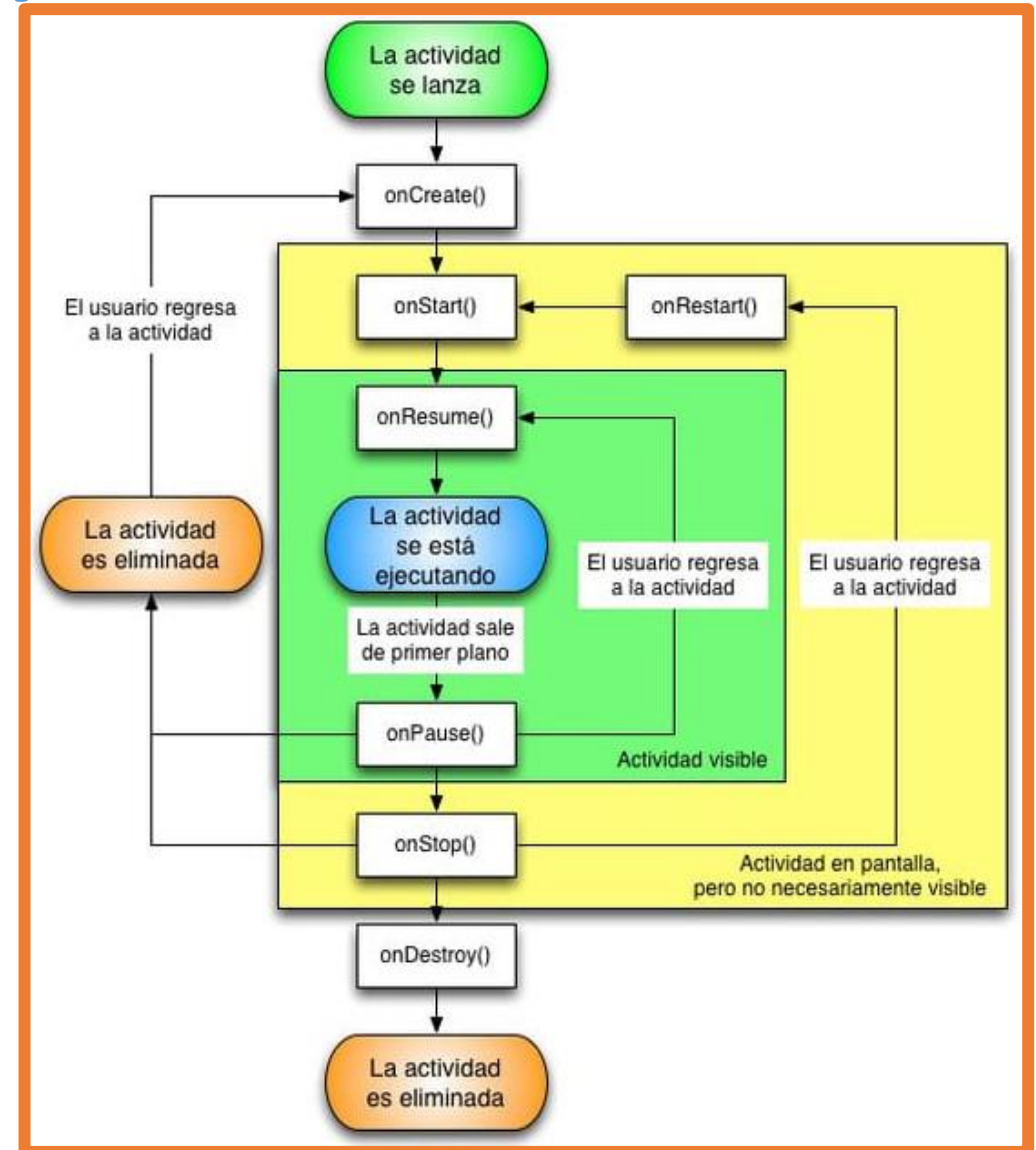
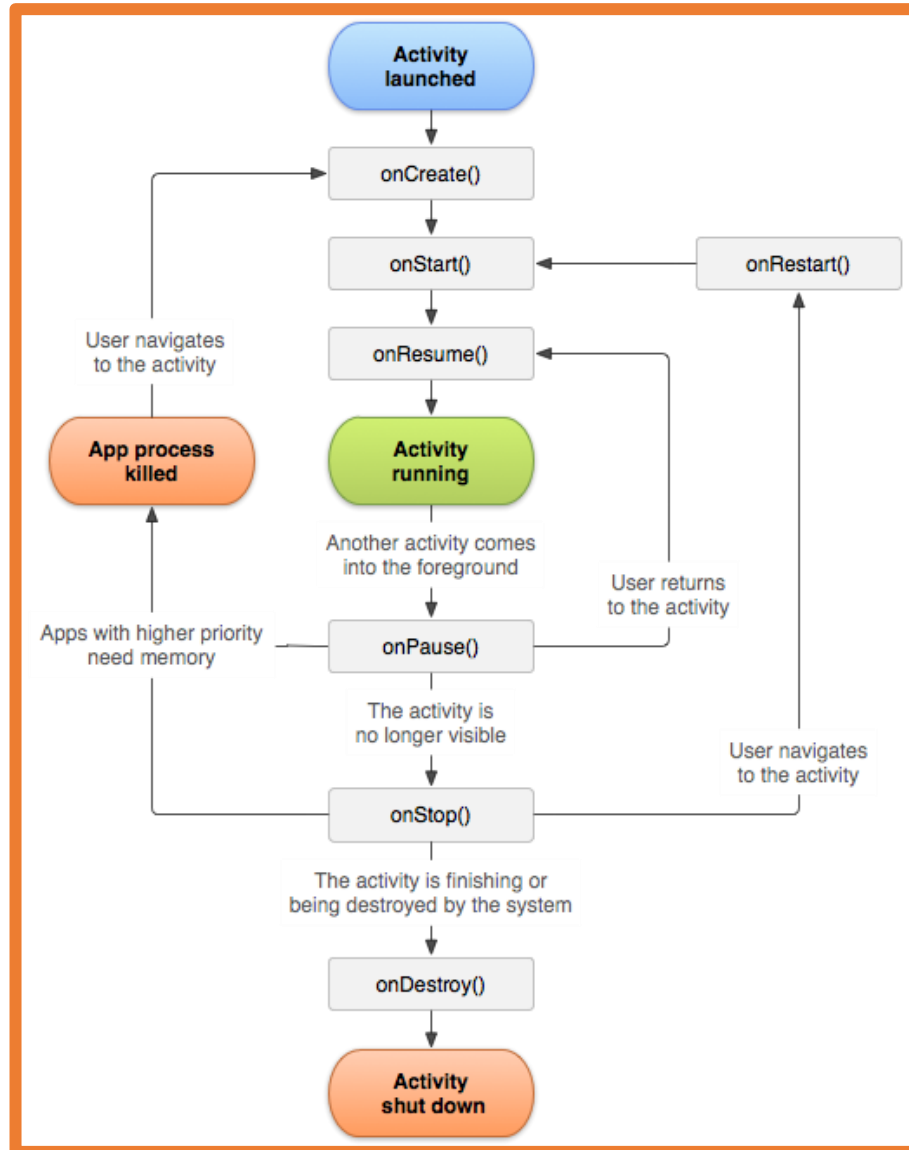
En Android **cada pantalla de la aplicación está definida en una Activity.**

Una **Activity** es el **punto de entrada** para la interacción del usuario con la aplicación.

En Android el código que inicia una **Activity** corresponde a una llamada a un **método** que corresponde a una **de las etapas específicas del ciclo de vida** de la **Activity**.

Conforme el usuario navega, sale y regresa a la aplicación, las diferentes **Activities** de la aplicación pasan por diferentes **estados** de su **ciclo de vida** (lifecycle).

3.- Ciclo de vida de una Activity



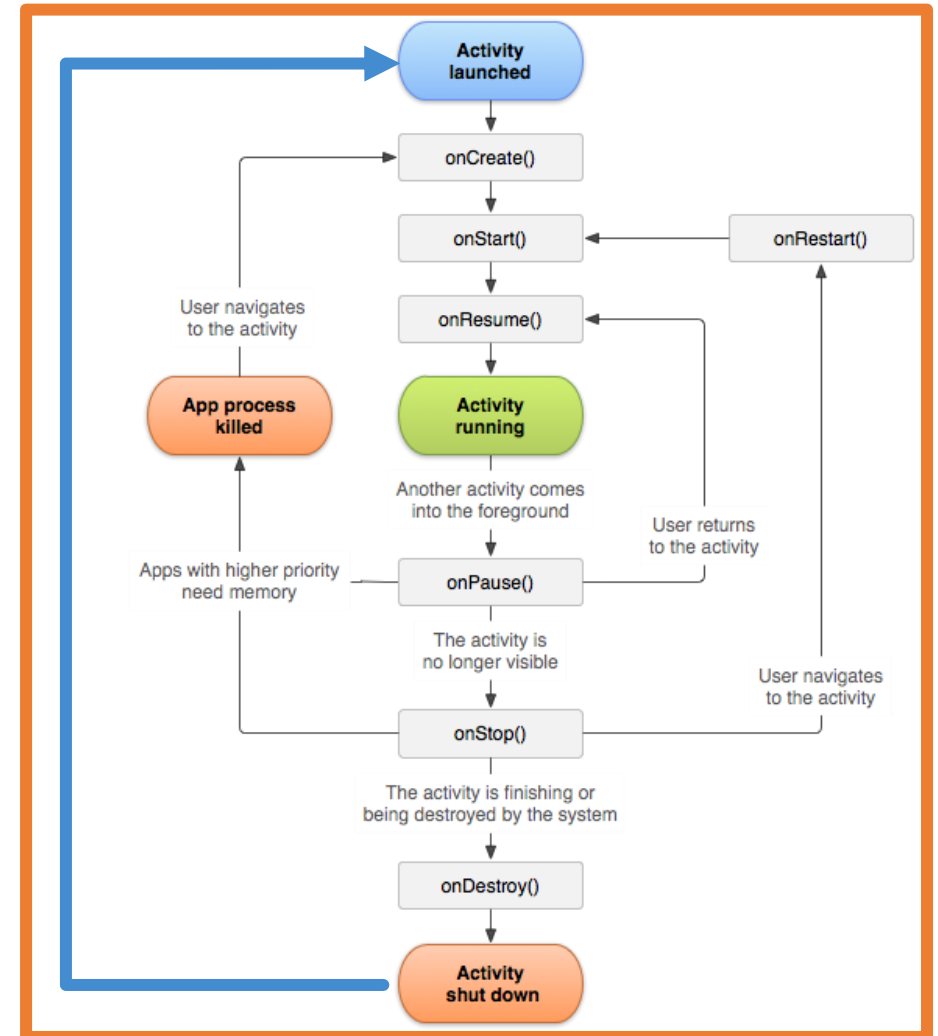
3.- Ciclo de vida de una Activity

Hay que destacar que Android tiene un comportamiento peculiar.

Una Activity activa se destruye y se vuelve a crear cuando:

- Se cambia la orientación del dispositivo.
- Se cambia entre los modos claro y oscuro.
- Se cambia la configuración del dispositivo, por ejemplo el idioma.

Tener en cuenta esto es muy importante a la hora de desarrollar las aplicaciones.



3.- Ciclo de vida de una Activity

La clase **Activity** proporciona una serie de **funciones de retorno** (callbacks) que permiten saber a la actividad que **ha cambiado de estado**:

- onCreate()
- onStart()
- onResume()
- onPause()
- onStop()
- onDestroy()

El sistema invoca a cada uno de estos callbacks cuando una actividad cambia de estado de su ciclo de vida (lifecycle).

3.- Ciclo de vida de una Activity

Con los **callbacks** del ciclo de vida se declara cómo se comporta la actividad cuando el usuario deja y vuelve a la actividad.

Por ejemplo:

Si se está creando un reproductor de video por streaming se puede indicar que se pause el vídeo y se finalice la conexión de red si el usuario cambia de aplicación.

Cuando el usuario vuelva a la actividad se puede reconectar y reanudar la reproducción.

4.- Programación imperativa vs programación declarativa

Antes de comenzar a desarrollar aplicaciones Android es necesario comprender dos paradigmas de programación:

- **Programación imperativa:**

Se indica qué se quiere y cómo se quiere hacer.

Es la programación "típica" usando estructuras de control (if, while, for...)

"Hemos visto que hay una mesa libre para dos personas justo al lado de la barra, nos gustaría sentarnos a cenar y primero se sentará mi amigo y luego me sentaré yo."

- **Programación declarativa:**

Se indica qué se quiere.

SQL es un lenguaje declarativo.

"Mesa para dos, por favor"

4.- Programación imperativa vs programación declarativa

El lenguaje de programación **Kotlin** dispone de características que lo acercan a la **programación declarativa**.

Ejemplo: Recorrer un array y transformarlo en otro

- Java: se realiza un for para pasar por todos los elementos, se indica cómo se transforma cada elemento y por último, se almacena en el array final.
- Kotlin: se utiliza la función map en la que se dice que una lista la mapee a otra sin indicar si tiene que crear una lista nueva, ni si tiene que añadir los elementos, ni el orden.

5.- Programación de aplicaciones nativas Android

Hoy en día para desarrollar aplicaciones nativas en Android hay dos opciones:

- Tradicional con **Views** (Vistas): programación imperativa.

La interfaz gráfica se define en archivos XML en los que se indican los elementos gráficos (Views) y en el código del programa se indica cómo se realizan todas las acciones.

- **Jetpack Compose**: programación imperativa-declarativa.

En el código del programa se indican los elementos gráficos (UI declarativas) y qué funcionalidad tienen (programación imperativa).

5.- Programación de aplicaciones nativas Android

Uno de los puntos fuertes de las **interfaces de usuario declarativas** es que los elementos de la interfaz se conectan al **estado de la Activity**.

De esta manera **si un elemento de la interfaz cambia, el estado cambia y la interfaz se repinta** para representar ese nuevo estado.

Esto se realiza **de manera automática** sin tener que indicar nada como programadores.

Este sistema está muy optimizado y **solo las partes afectadas por ese cambio de estado son las que se repintan**.

5.- Programación de aplicaciones nativas Android

Ventajas del uso de interfaces de usuario declarativas:

- Menos código.
- Código más sencillo y fácil de entender/leer.
- Evita clases intermedias que pueden proveer de errores.
- Intuitivas.
- Al engancharse al estado de la aplicación la vista se encarga de todo.
- Muy rápidas.
- Vistas previas de cualquier componente.
- Muy potentes.

Frameworks como **React Native**, **Flutter** o **Swift UI** utilizan el paradigma de interfaces de usuario declarativas.

5.- Programación de aplicaciones nativas Android

En este curso se verá el uso de Jetpack Compose por ser la tendencia actual del mercado.

En el desarrollo de aplicaciones nativas Android se pueden mezclar la manera tradicional con XML y Jetpack Compose.

En Aules están disponibles los apuntes del curso pasado donde se explica el desarrollo de aplicaciones nativas Android de la manera tradicional con Views y archivos XML.

6.- Jetpack Compose

Jetpack Compose es un **kit de herramientas** (toolkit) para crear y compilar **interfaces de usuario declarativas** para Android.

Se basa **100% en Kotlin**.

Se incorpora a partir de la versión **Artic Fox** (2021) de Android Studio.

Las aplicaciones desarrolladas con Jetpack Compose se pueden ejecutar en las versiones de **Android 5.0 (API 21) y superiores**.



6.- Jetpack Compose

Jetpack Compose solo está disponible para Android.

Se compone de:

- **Compilador:** plugin gradle que genera el código necesario.
- **Runtime:** entorno de ejecución que genera y mantiene el árbol de nodos para saber los elementos que se encuentran en la interfaz.
- **Librería de UI:** decide cómo se interpreta y se pinta el árbol de nodos.

6.- Jetpack Compose

Tanto el **compilador** como el **runtime** son "fijos" y trabajan de forma genérica.

La **librería de UI** es un componente que puede cambiar y ahora mismo la única versión estable es Jetpack Compose que es para Android.

Jetbrains está creando las librerías:

- Compose for Desktop (Windows, Mac y Linux)
- Compose for web (experimental)
- Compose for iOS (alpha)

Todas ellas se agrupan junto a Jetpack Compose en el proyecto [Compose Multiplatform](#) que permite desarrollar una aplicación con Compose y generar el ejecutable para Android, iOS, escritorio y web como ya ocurre con Flutter.

7.- Desarrollo Android con Jetpack Compose

El uso de Jetpack Compose para el desarrollo de aplicaciones Android consiste en definir la interfaz gráfica de la aplicación de manera declarativa.

Para definir la interfaz gráfica se usan componentes de Jetpack Compose que pueden ser los ofrecidos por el sistema o bien los propios definidos por el programador.

Un componente Jetpack Compose puede contener a otro componente Jetpack Compose. Es habitual usar este comportamiento para crear componentes propios que extiendan la funcionalidad de otros componentes ya existentes.

Si se necesita también se puede utilizar la programación imperativa: variables, clases, estructuras de control, funciones...

8.- Crear un proyecto Android

La mejor manera de **entender cómo funciona Android Studio** es **realizar una aplicación sencilla** donde se utilicen algunos de los componentes y funcionalidades.

Así, esta unidad va a consistir en explicar los conceptos a la vez que se crea una aplicación.

Algunos conceptos que se verán se explicarán más detenidamente en las siguientes unidades.

La aplicación que realizaremos consistirá en un **contador de clics** como la que se muestra en la imagen.



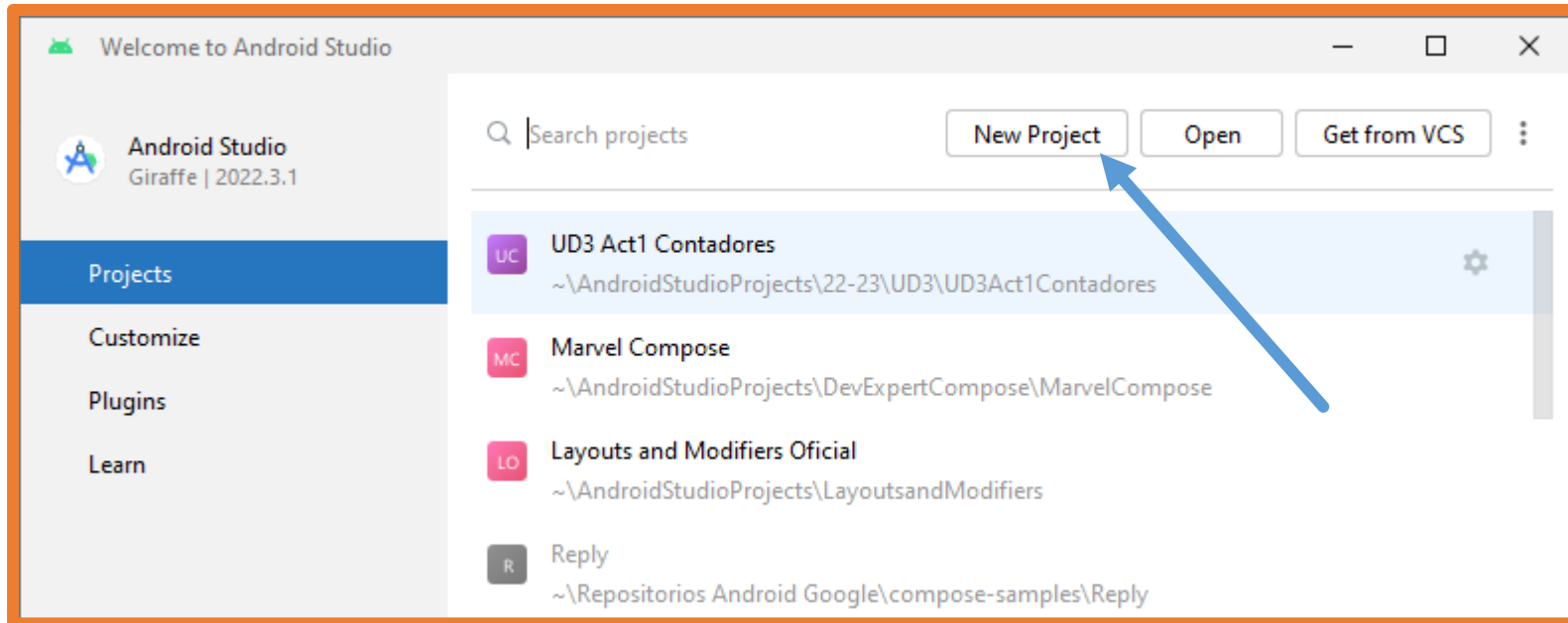
8.- Crear un proyecto Android

Una vez abierto Android Studio hay varias opciones para crear un proyecto:

- Si **no hay abierto** ningún proyecto Android:
Hacer clic en **New Project**.
- Si **hay un proyecto abierto** hay dos opciones:
 - Cerrar proyecto y a continuación hacer clic en **New Project**.
 - Crear un nuevo proyecto directamente.
- Crear proyecto **desde un control de versiones (VCS)**:

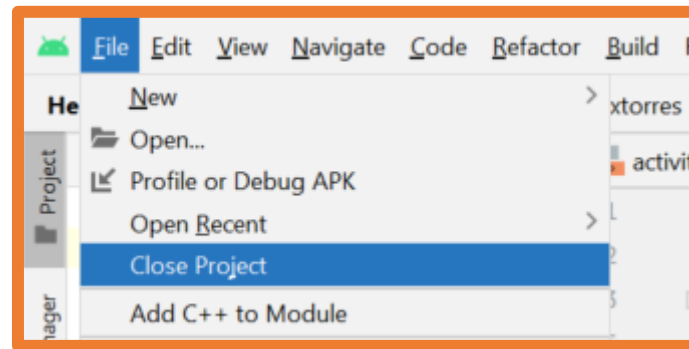
8.- Crear un proyecto Android

- Si **no hay abierto** ningún proyecto Android:
Hacer clic en **New Project**.

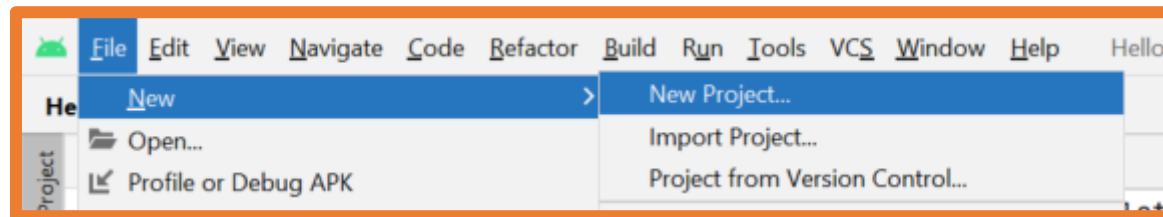


8.- Crear un proyecto Android

- Si **hay un proyecto abierto** hay dos opciones:
 - Cerrar proyecto y a continuación hacer clic en **New Project**.

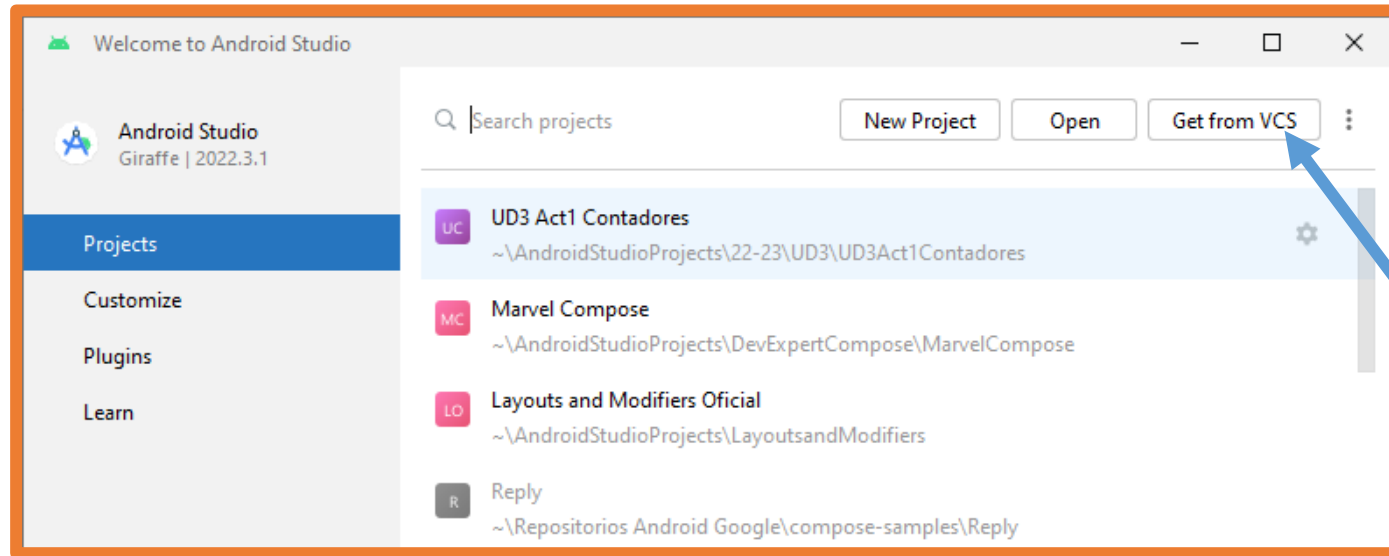


- Crear un nuevo proyecto directamente.



8.- Crear un proyecto Android

- Crear proyecto **desde un control de versiones (VCS)**:
Hacer clic en **Get from VCS**.
Se debe disponer de la URL del repositorio.

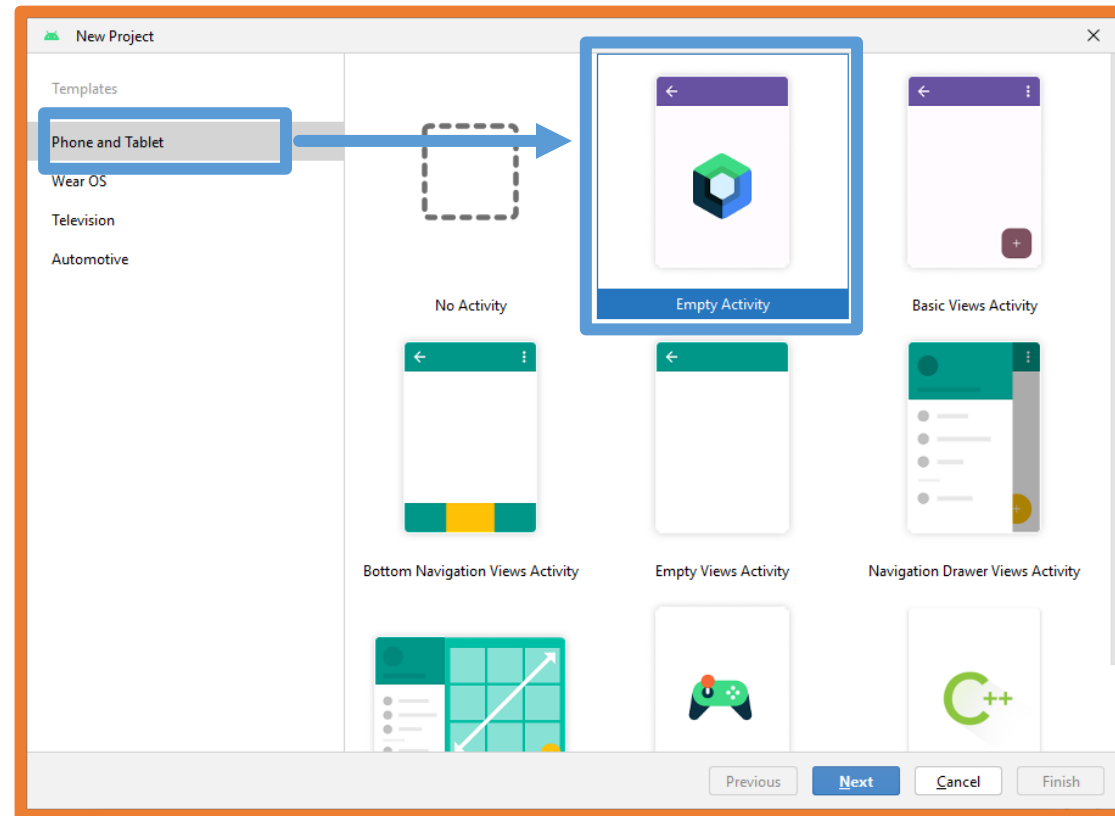


Esta opción descarga un proyecto ya creado y configurado por lo que las siguientes páginas no aplican a esta opción.

8.- Crear un proyecto Android

Al crear el proyecto, Android Studio muestra una ventana con todas las plantillas disponibles.

Se debe elegir **Phone and Tablet** y a continuación **Empty Activity** (utiliza Jetpack Compose).



Las plantillas con el texto Views en el nombre utilizan la programación tradicional con XML.

8.- Crear un proyecto Android

A continuación, se debe rellenar las opciones del proyecto:

- **Nombre:** debe ser significativo.
- **Paquete:** debe ser único, para estar seguros de esto se seguirá la siguiente estructura
com.tunombretuapellido.nombreproyecto
- **Save location:** directorio que se quiera
- **Minimum SDK:** API 24 ("Nougat"; Android 7.0)

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name: Contador de clics

Package name: com.alextorres.contadordeclics

Save location: C:\Users\Alex\AndroidStudioProjects\23-24\UD3\Contadordeclics

Minimum SDK: API 24 ("Nougat"; Android 7.0)

ⓘ Your app will run on approximately 95,4% of devices.
[Help me choose](#)

Build configuration language ⓘ Kotlin DSL (build.gradle.kts) [Recommended]

Previous Next Cancel Finish

8.- Crear un proyecto Android

SDK mínimo

La elección del SDK mínimo es un paso crucial en el inicio de un proyecto.

- Versión más baja posible → soporte a la mayor cantidad de dispositivos.
- Versión más alta posible → tener todas las características y funcionalidades.

En **todas las actividades del curso** se va a elegir la versión **API 24** (alcance de un 95,4%).

Aunque se podría elegir sin problema las versiones **API 27** (90,2%) o **API 28** (84,1%) ya que hoy en día pocos dispositivos están por debajo de esas versiones.

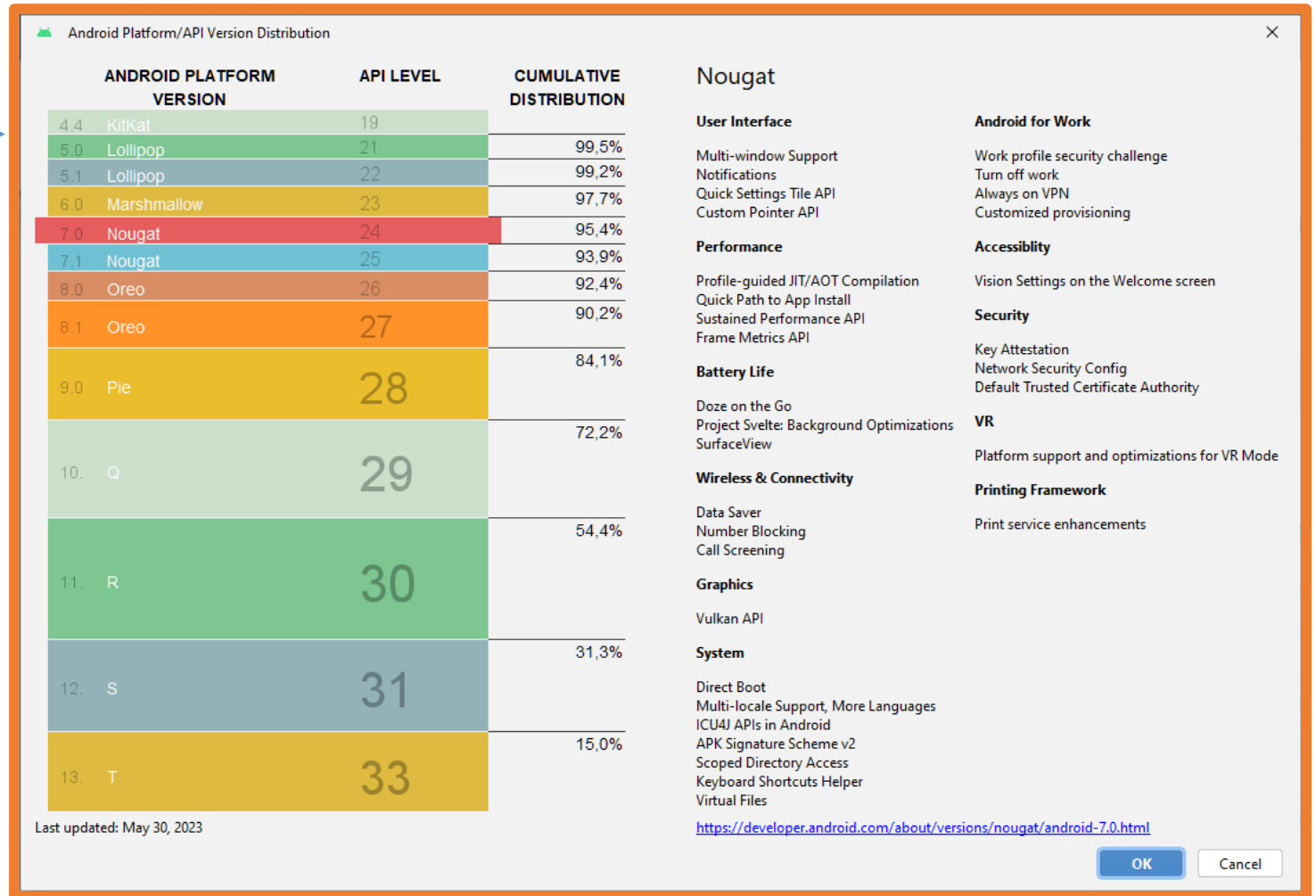
8.- Crear un proyecto Android

SDK mínimo

Minimum SDK

API 24 ("Nougat"; Android 7.0)

 Your app will run on approximately 95,4% of devices.
[Help me choose](#)



8.- Crear un proyecto Android

Una vez seleccionadas todas las opciones se debe hacer clic en **Finish**.

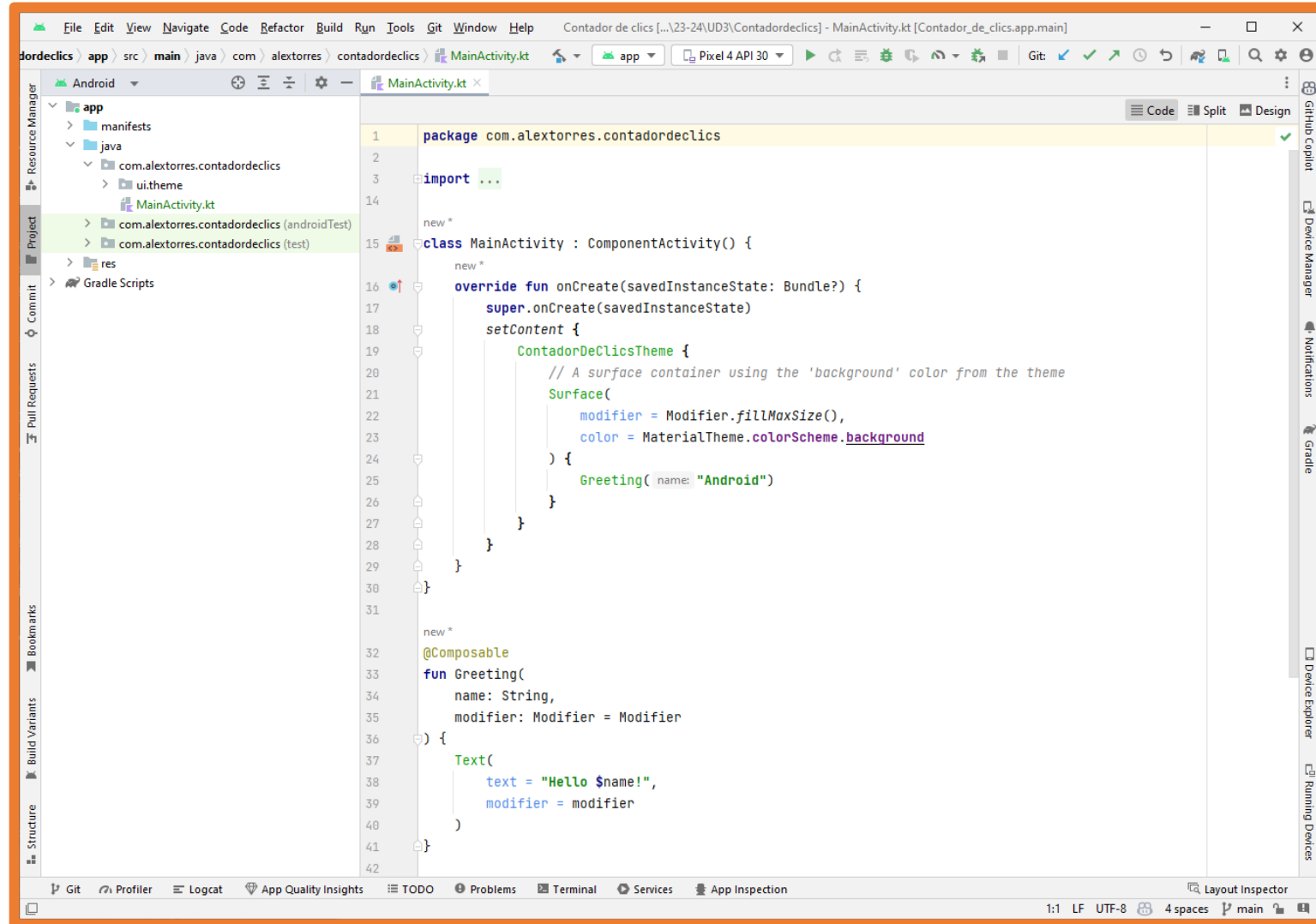
En ese punto **Android Studio comenzará a crear el proyecto**.

Durante la creación del proyecto Android Studio realizará la descarga de todos los componentes necesarios.

Se recomienda no interactuar con el programa hasta que no finalice por completo la creación del proyecto.

En la parte inferior derecha se puede consultar la barra de progreso con las descargas y creación del proyecto.

8.- Crear un proyecto Android

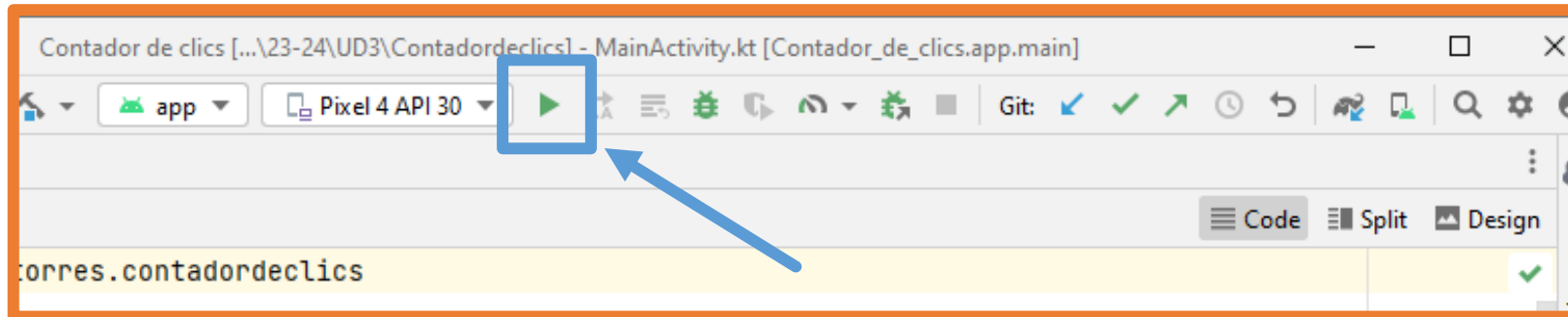


Práctica

Actividad

Crear proyecto llamado "Test".

Ejecútalo en el emulador de Android Studio.

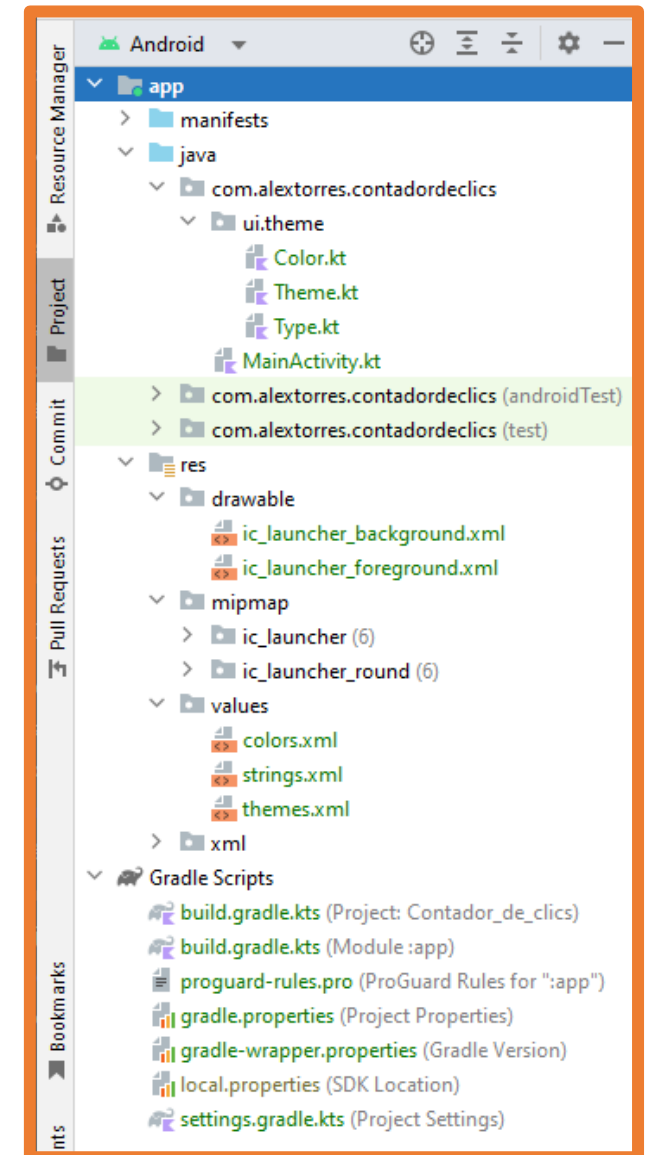


9.- Estructura de un proyecto Android

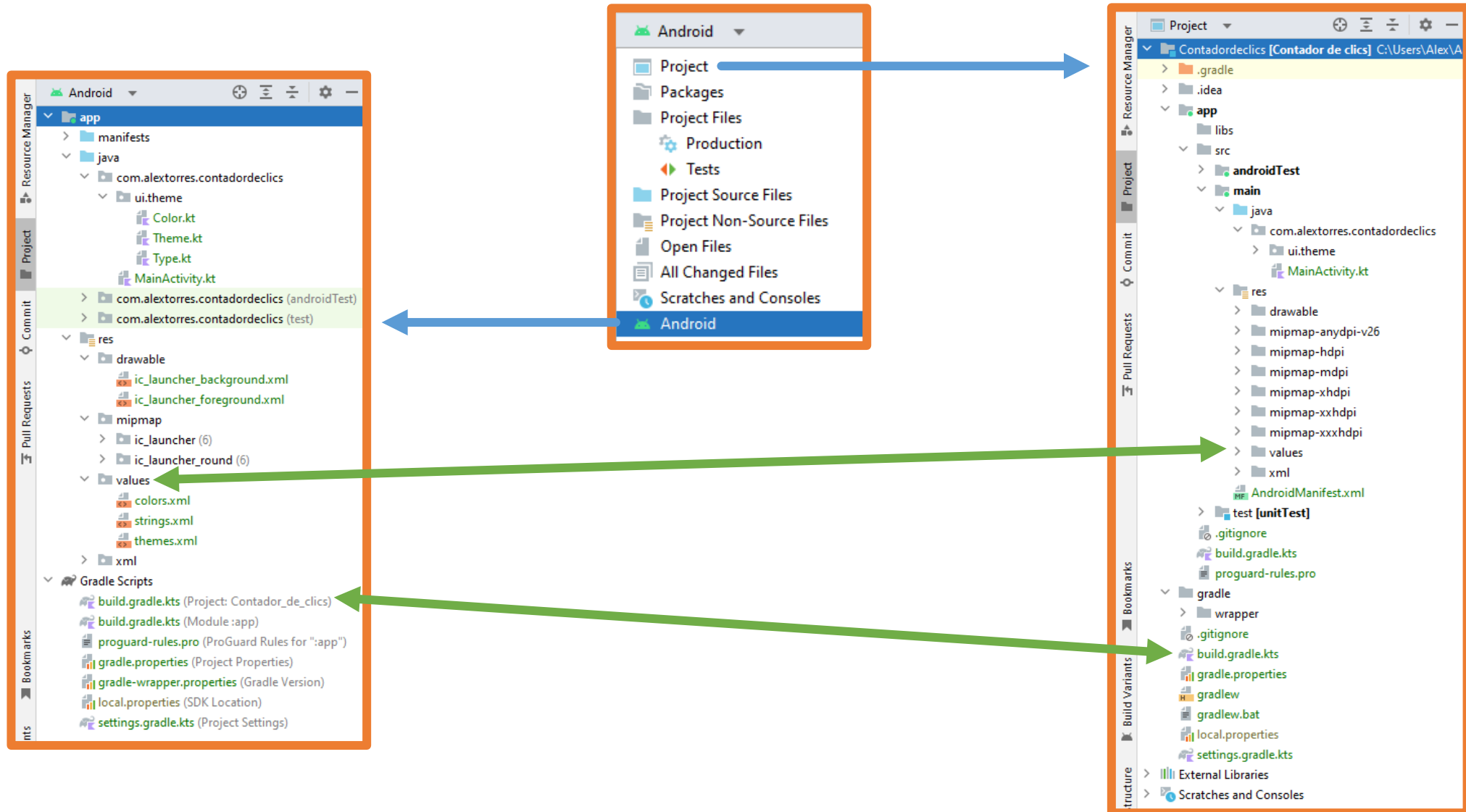
Una vez creado el proyecto se pueden ver todos los **archivos del mismo**.

Depende de la visualización que se seleccione (Project, Android...) los archivos se podrán encontrar en un lugar o en otro.

A continuación, se explicará los más importantes.



9.- Estructura de un proyecto Android



9.- Estructura de un proyecto Android

Archivo: AndroidManifest.xml

Project → app/src/main/AndroidManifest.xml

Android → manifests/AndroidManifest.xml

Describe las características fundamentales de la aplicación y sus componentes.

En él se especifican las **Activities** (pantallas) que tiene la aplicación y qué permisos requieren dichas Activities: cámara, contactos, internet...

9.- Estructura de un proyecto Android

Archivos: build.gradle

Android Studio utiliza **Gradle** para compilar y construir la aplicación.

Hay un archivo **build.gradle** para todo el proyecto y otro archivo **build.gradle** por cada módulo del proyecto.

Por lo general, solo interesará el archivo **build.gradle** del módulo **app**.

En este archivo están las dependencias de compilación de la aplicación y también la configuración predeterminada.

Project → app/build.gradle

Android → Gradle Scripts/build.gradle.kts (Module: app)

9.- Estructura de un proyecto Android

Archivo: build.gradle.kts (Module: app)

- compiledSdk → Versión a la que se va a compilar.
Por defecto, es la última versión SDK instalada en el ordenador
- applicationId → Nombre completo del paquete de la aplicación.
- minSdk → Versión mínima de SDK especificada al crear el proyecto.
Será la versión más antigua que admita la aplicación.
- targetSdk → Versión más alta con la que se prueba la aplicación.
- dependencias → Sección donde añadir las dependencias que se quieren instalar para la aplicación.

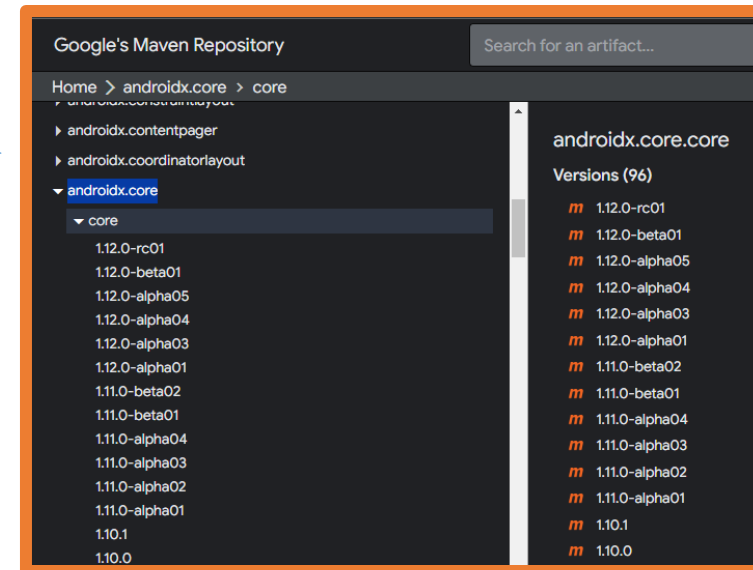
9.- Estructura de un proyecto Android

Archivo: build.gradle.kts (Module: app)

dependencias → Sección donde añadir las dependencias que se quieren instalar para la aplicación.

Si se quieren consultar las últimas versiones de las dependencias ofrecidas por Google para Android se debe ir a <https://maven.google.com/web/index.html>.

```
implementation("androidx.core:core-ktx:1.10.1")  
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.6.1")  
implementation("androidx.activity:activity-compose:1.7.2")
```



Para otras dependencias se debe consultar su documentación oficial.

9.- Estructura de un proyecto Android

Archivo: MainActivity.kt

Project → app/src/main/java/com.alextores.contadordeclics/MainActivity.kt

Android → java/com.alextores.contadordeclics/MainActivity.kt

En este archivo se **programará el comportamiento de esta ventana.**

Por defecto contiene:

- La definición de la Activity principal y su método onCreate. Por el ciclo de vida de las Activities el código de onCreate se ejecutará cuando la actividad alcance ese estado.
- Componente de Jetpack Compose con un texto (Text).
- Componente de Jetpack con una previsualización (@Preview).

9.- Estructura de un proyecto Android

Carpeta: ui/theme

Project → app/src/main/java/com.alextores.contadordeclics/ui.theme

Android → java/com.alextores.contadordeclics/ui.theme

En este directorio se encuentran los archivos que permiten configurar el tema que usa la aplicación.

Por defecto Jetpack Compose utiliza un tema basado en [Material Design](#) (guía de diseño de interfaces diseñada por Google).

Con los archivos incluidos en ui.theme se puede extender ese tema.

9.- Estructura de un proyecto Android

Carpeta: res

Project → app/src/main/res

Android → res

Este directorio contiene los recursos de la aplicación.

Carpeta: drawable

Project → app/src/main/res/drawable

Android → res/drawable

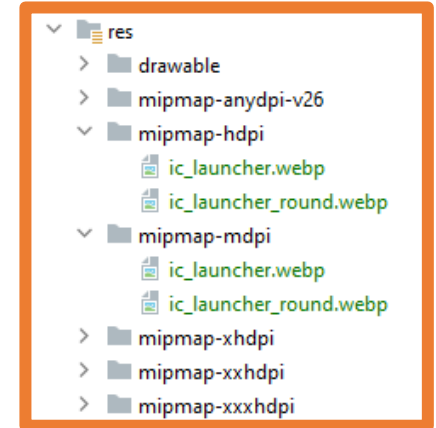
Directorio donde almacenar las imágenes de la aplicación

9.- Estructura de un proyecto Android

Carpeta: mipmap

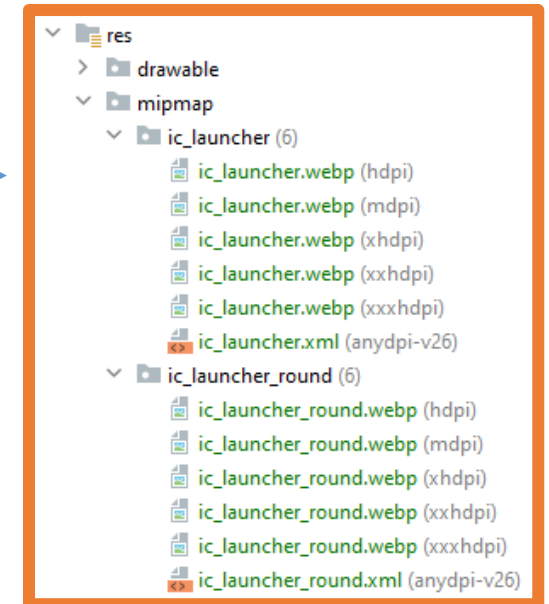
Project → app/src/main/res/mipmap-*RESOLUCIÓN*

Directorios que contienen el icono de la aplicación para las diferentes densidades de píxeles de pantalla.



Android → res/mipmap

En la vista Android se agrupan los archivos por su nombre. Junto al nombre se puede ver la *RESOLUCIÓN* en la que se utilizan.



9.- Estructura de un proyecto Android

Carpeta: mipmap

Nomenclatura de *RESOLUCIÓN* en Android:

- xxxhdpi → 640 dpi
- xxhdpi → 480 dpi
- xhdpi → 320 dpi
- hdpi → 240 dpi
- mdpi → 160 dpi

10.- Icono de la aplicación

El icono de la aplicación se ve en las siguientes situaciones:

- En el menú de aplicaciones.
- En la pantalla principal (si se añade a ella).
- Durante el inicio de la aplicación (Splash Screen).
- En la lista de aplicaciones abiertas.

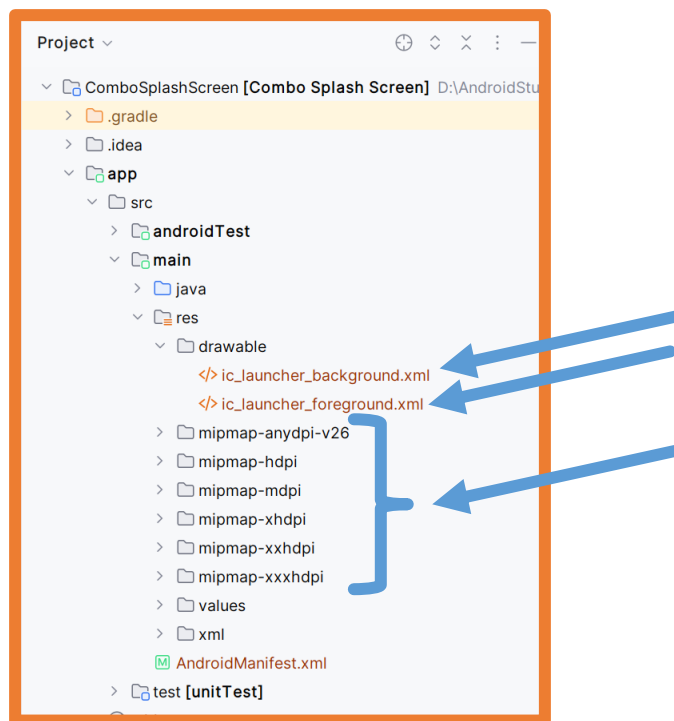
Si el icono de la aplicación es un logotipo la imagen debe ser vectorial (formato XML), pero si el icono es una foto entonces será un mapa de bits (formatos jpg, png...).

En caso de ser vectorial es interesante usar [el diseño de iconos adaptables](#) de Android que además ofrecen la posibilidad de efectos visuales.

10.- Icono de la aplicación

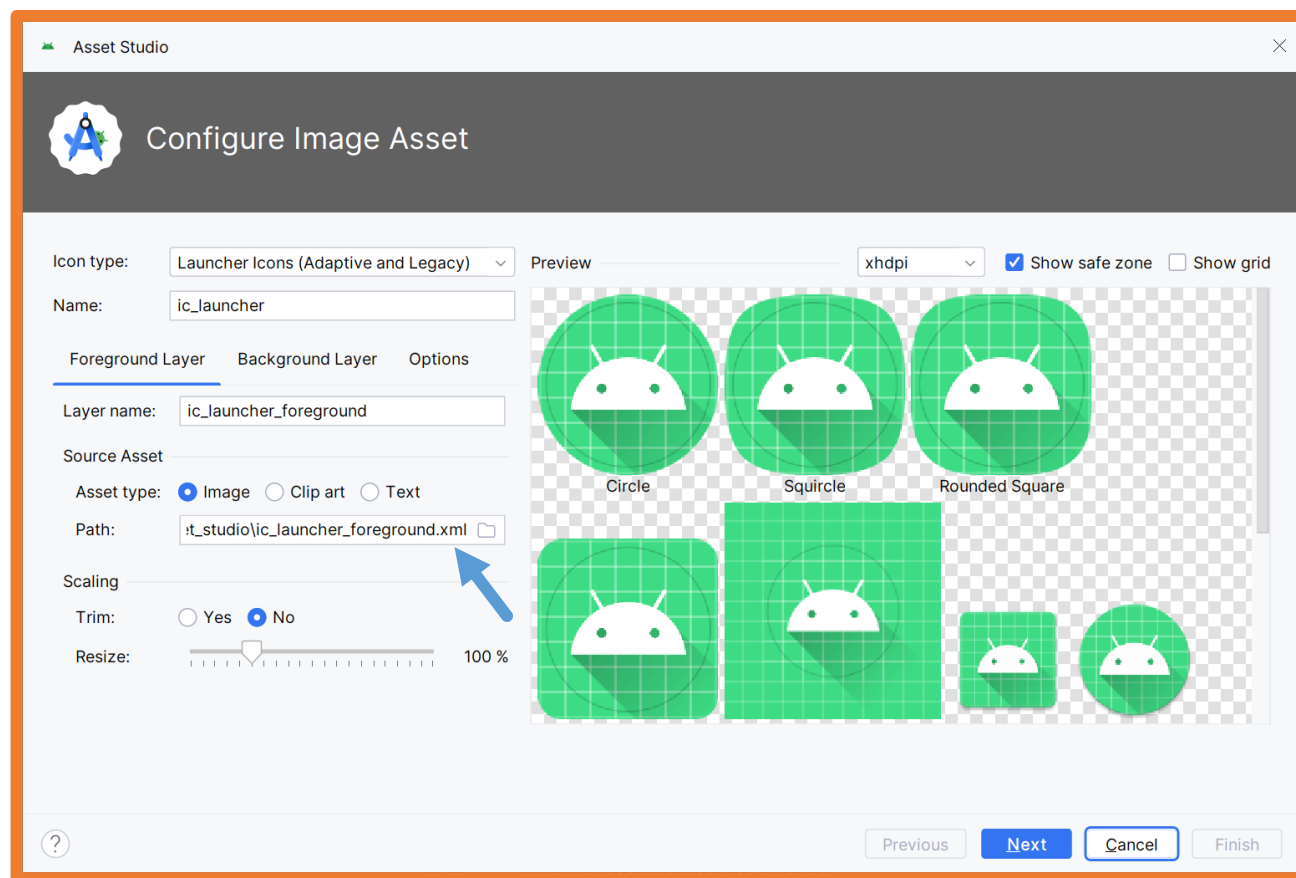
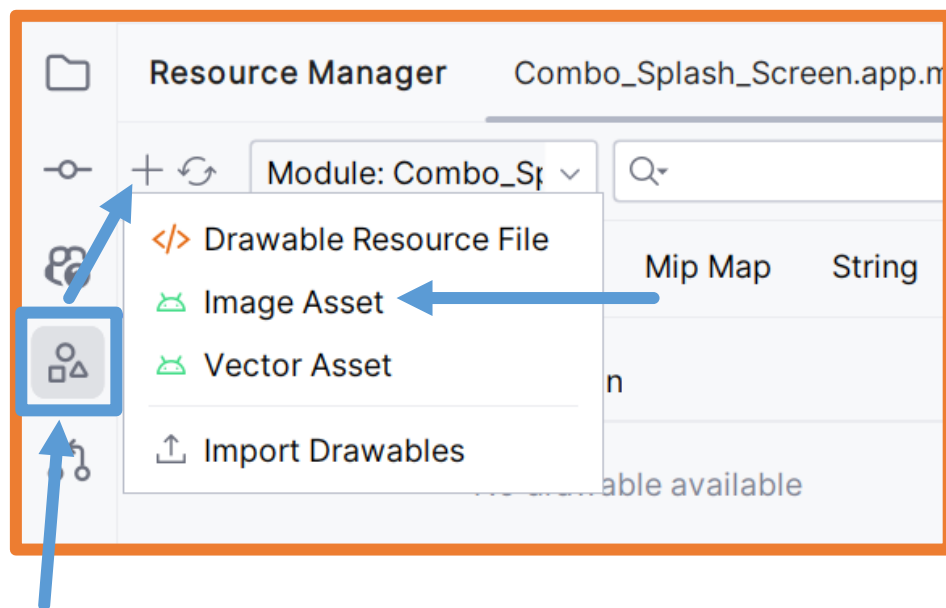
Ya sea una imagen vectorial o en mapa de bits los pasos para añadir una imagen como icono de la aplicación son los mismos.

- Desde la vista de **Project** de Android Studio se deben eliminar:
Los archivos **ic_launcher_background.xml**, **ic_launcher_foreground.xml**.
Todas las carpetas que contengan la palabra **mipmap**.



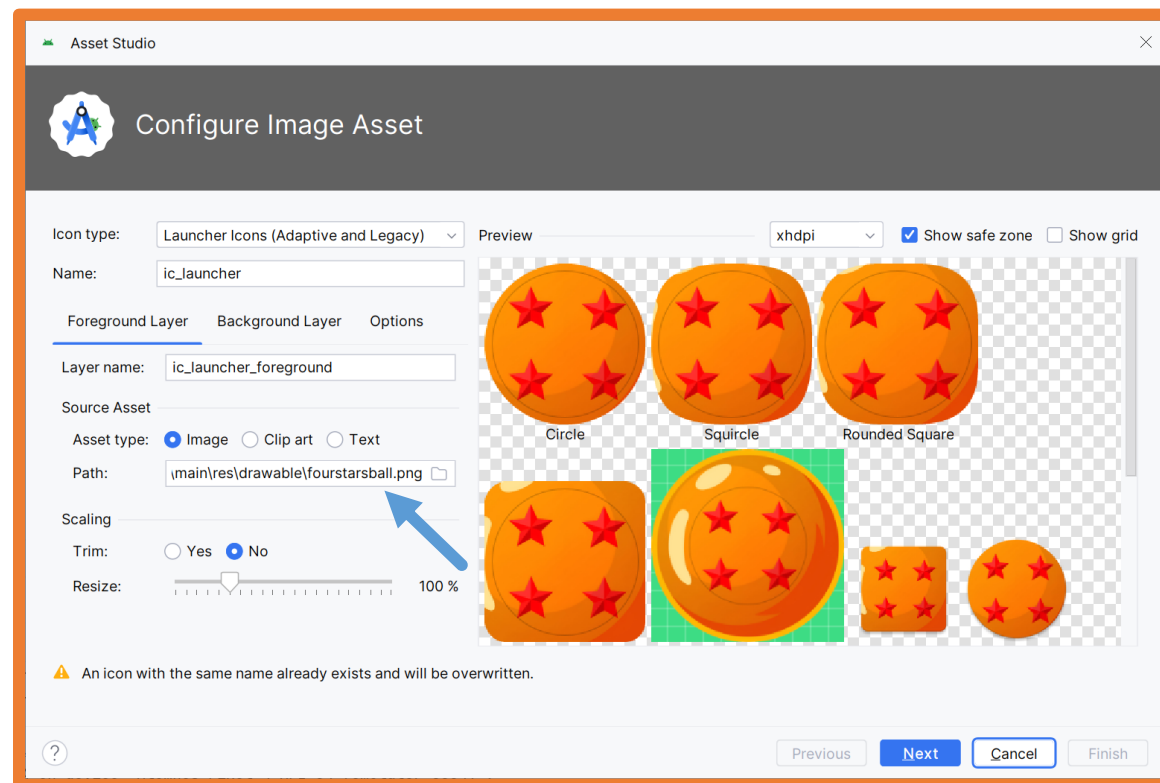
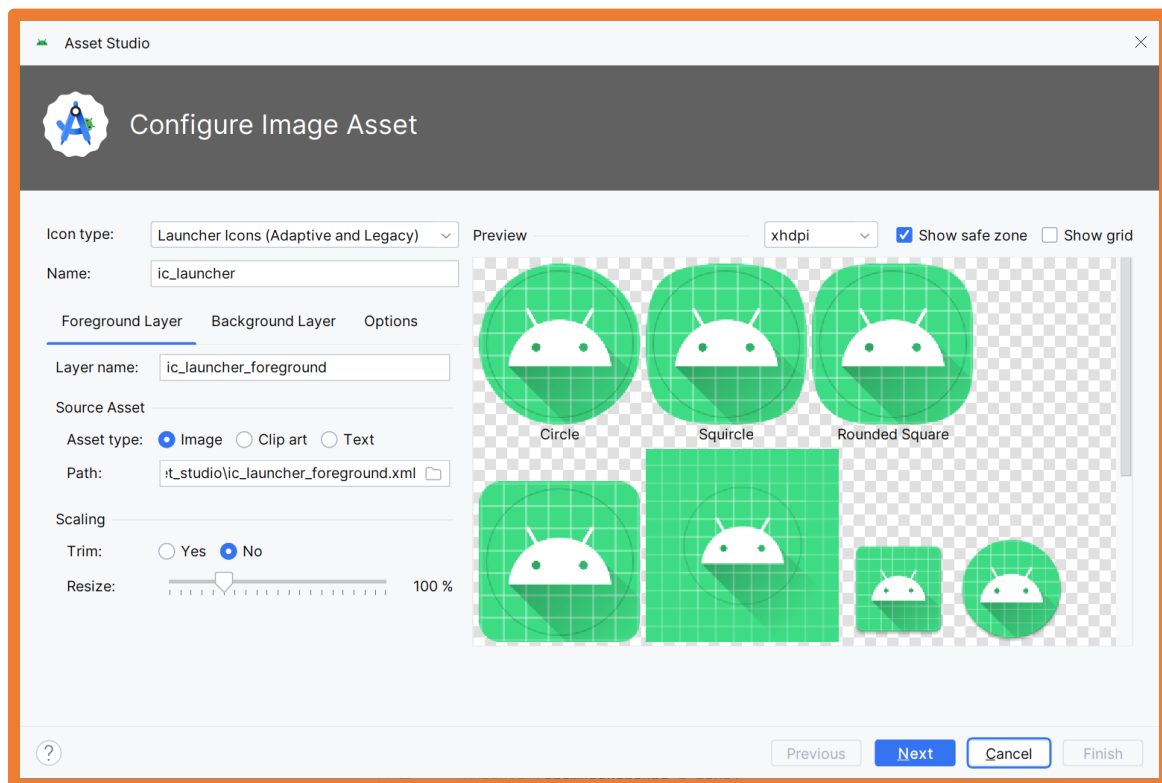
10.- Icono de la aplicación

- Se debe añadir la imagen primero al proyecto para que se encuentre en la carpeta **res/drawable**.
- A continuación, se tiene que añadir la imagen desde el **Resource Manager** como **Image Asset** dejando la configuración por defecto y seleccionando la imagen nueva en el apartado **Path**.



10.- Icono de la aplicación

- Se debe añadir la imagen desde el **Resource Manager** como **Image Asset** dejando la configuración por defecto y seleccionando la imagen nueva en el apartado **Path**:

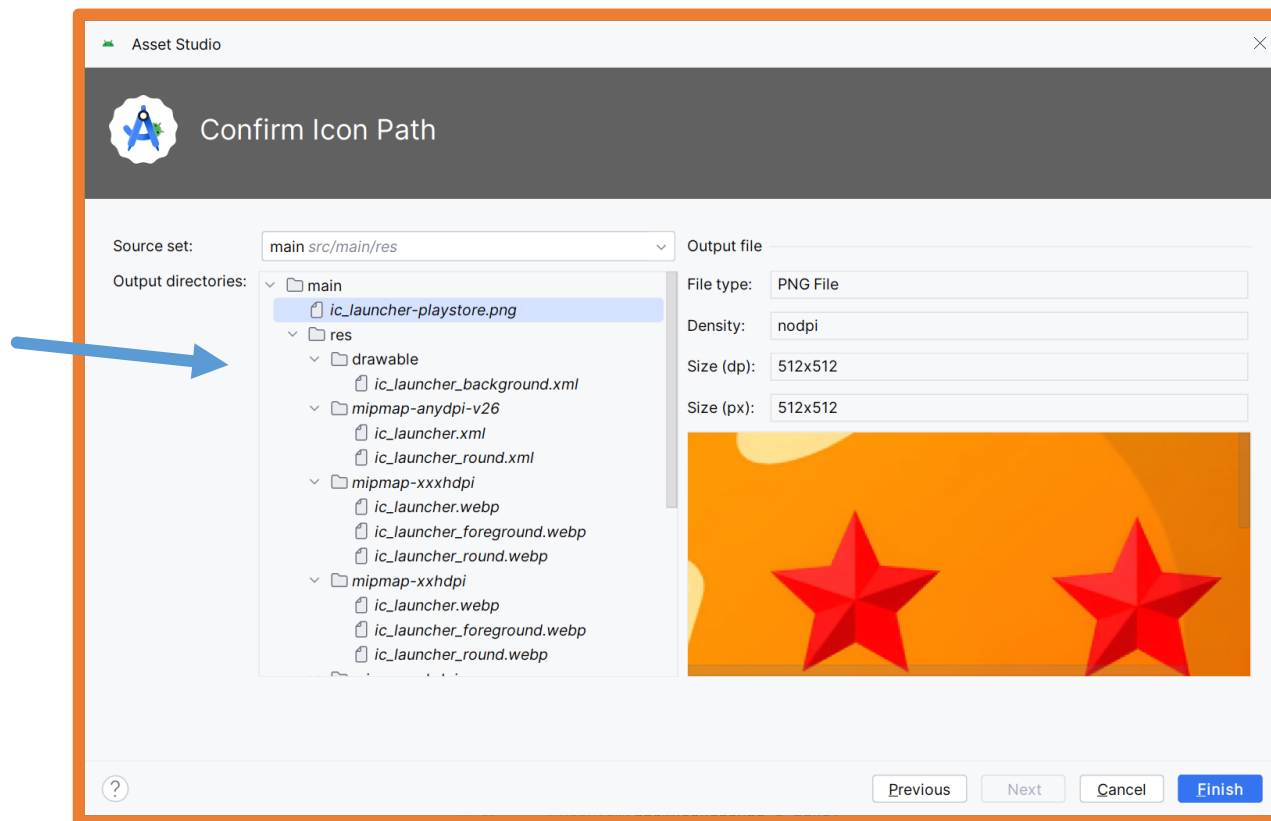


Se puede cambiar la configuración de escalado si fuera necesario.

10.- Icono de la aplicación

- Se debe añadir la imagen desde el **Resource Manager** como **Image Asset** dejando la configuración por defecto y seleccionando la imagen nueva en el apartado **Path**:

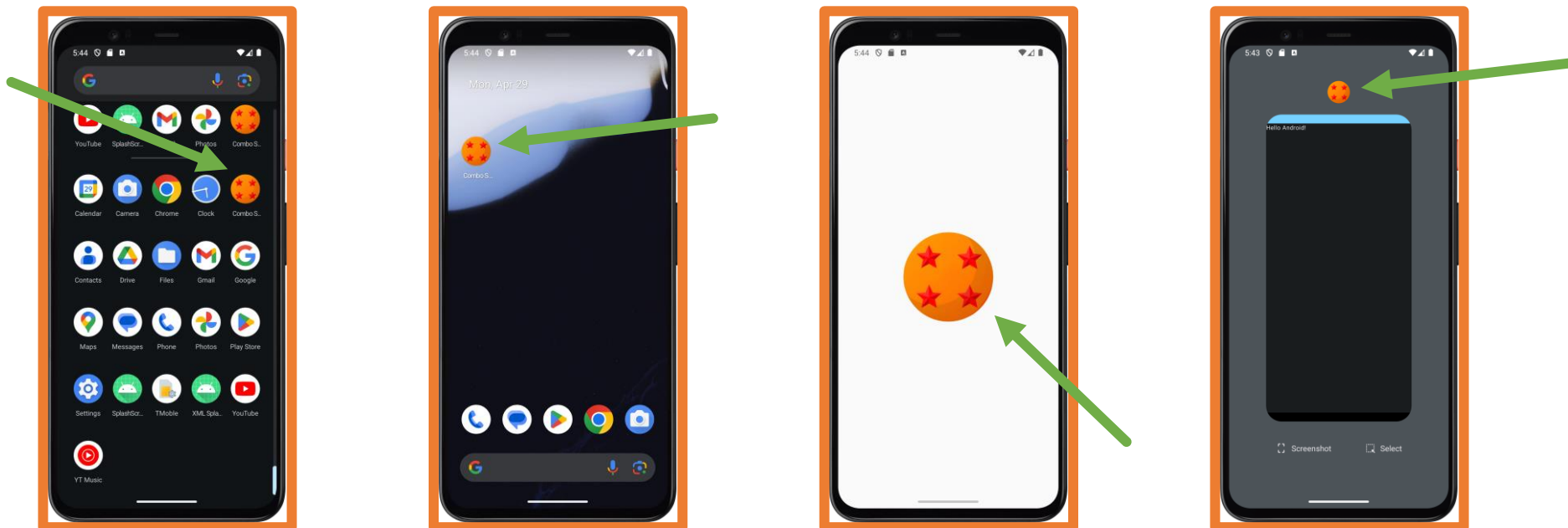
En el último paso se pueden observar todos los archivos que se van a crear.



10.- Icono de la aplicación

A partir de este momento la aplicación ya dispone de un nuevo icono.

- En el menú de aplicaciones.
- En la pantalla principal (si se añade a ella).
- Durante el inicio de la aplicación (Splash Screen).
- En la lista de aplicaciones abiertas.



11.- Configuración de Android Studio

Jetpack Compose está integrado totalmente dentro de Android Studio.

Aún así hay algunas configuraciones que se puede cambiar para mejorar la productividad y la experiencia del programador.

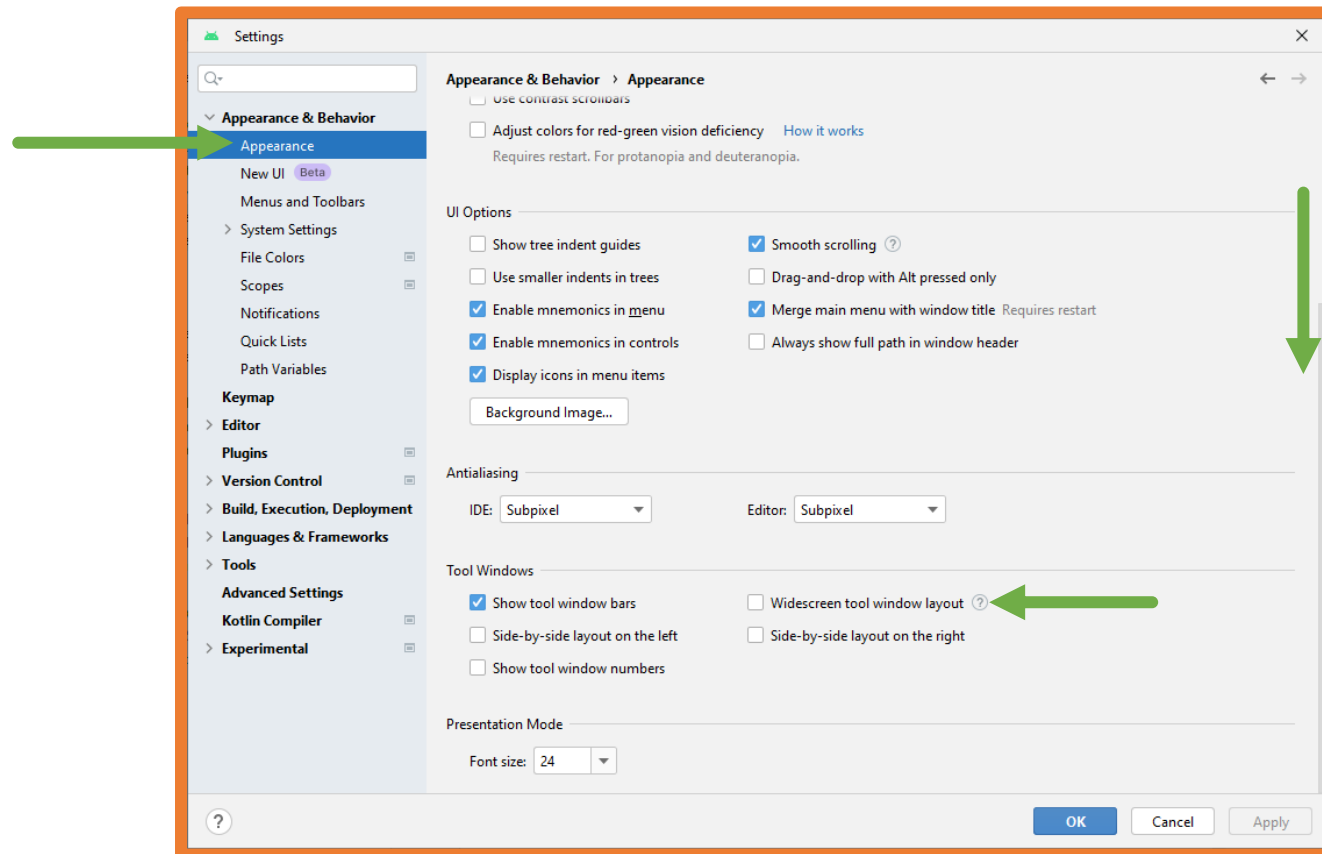
A continuación, se muestran algunas de estas configuraciones que pueden ser de ayuda durante el curso.

File → Settings **CTRL+ALT+S**

11.- Configuración de Android Studio

Barras laterales siempre visibles

Si se dispone de una pantalla ultra ancha es posible que siempre se quieran visibles las barras laterales que generalmente muestran: Project (árbol de archivos del proyecto) y Running Devices (Emulador).



11.- Configuración de Android Studio

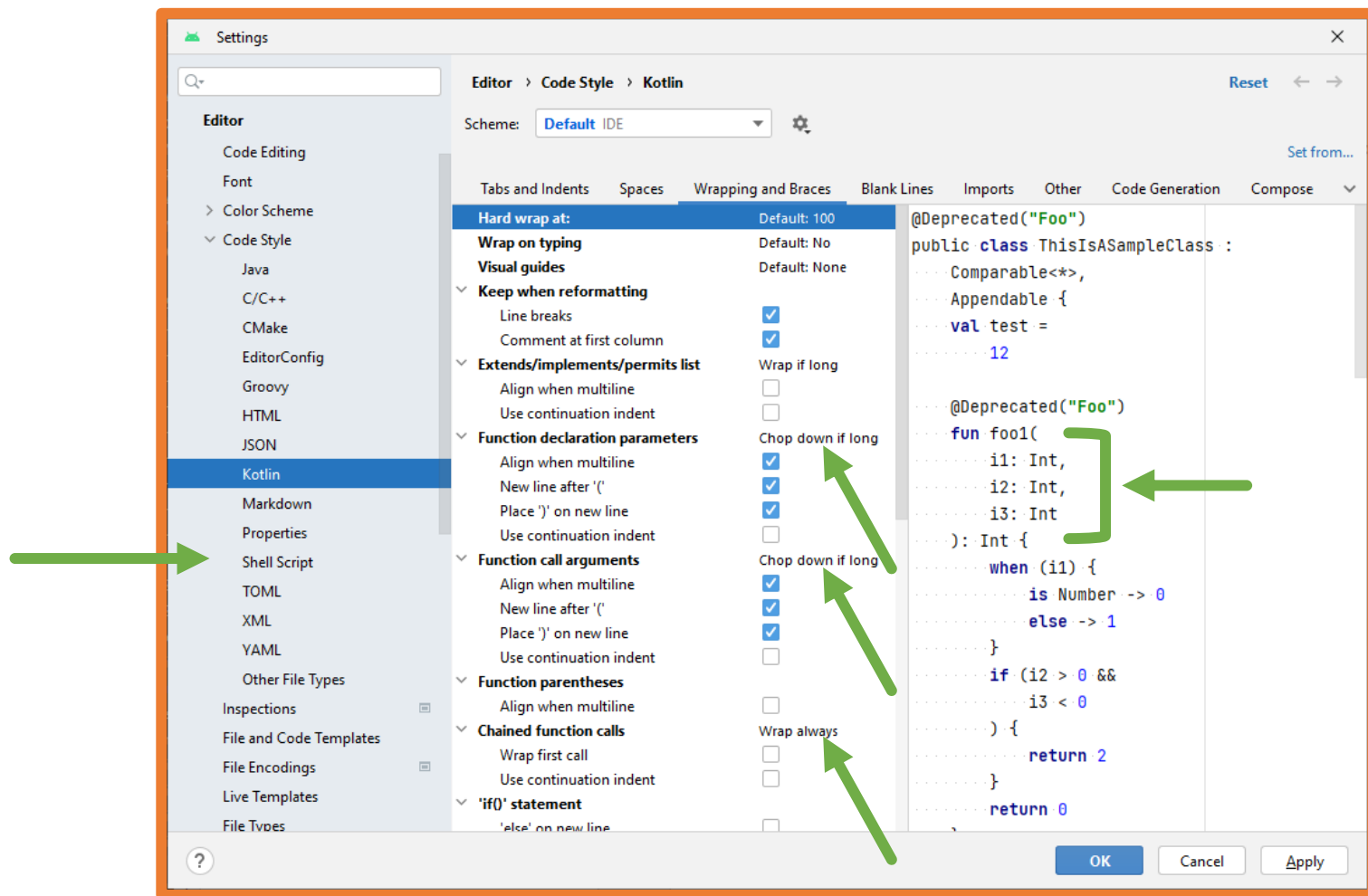
Formatear el código con las guías de estilo de Jetpack Compose

IntelliJ IDEA y **Android Studio** están desarrollados por **JetBrains** por lo que la combinación de teclas **CTRL+ALT+L** usada anteriormente formatea el código automáticamente con las guías de estilo de Kotlin.

Aún así, para visualizar mejor el código Kotlin para Jetpack Compose se pueden cambiar algunas opciones.

11.- Configuración de Android Studio

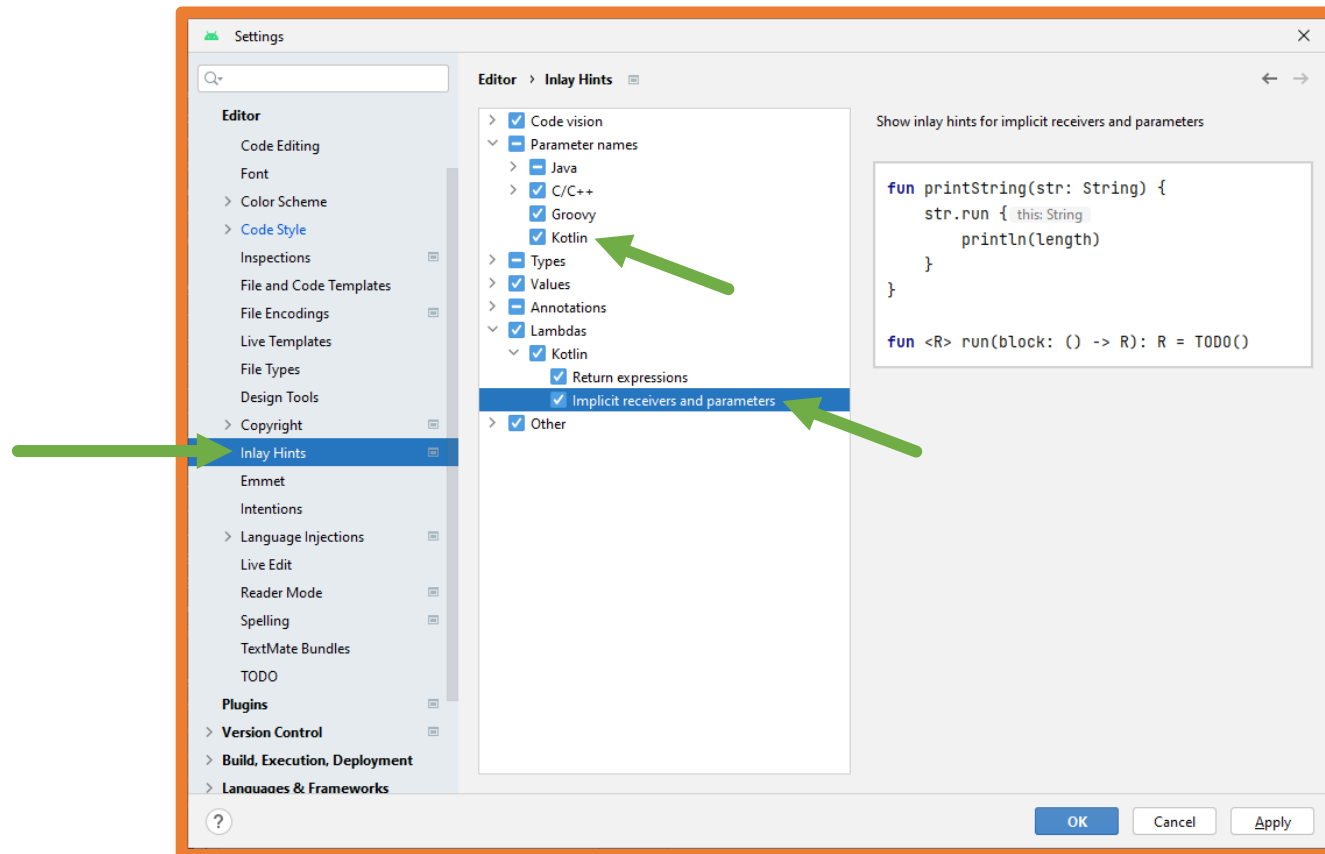
Formatear el código con las guías de estilo de Jetpack Compose



11.- Configuración de Android Studio

Pistas de código

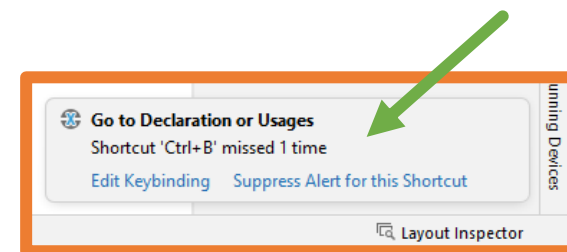
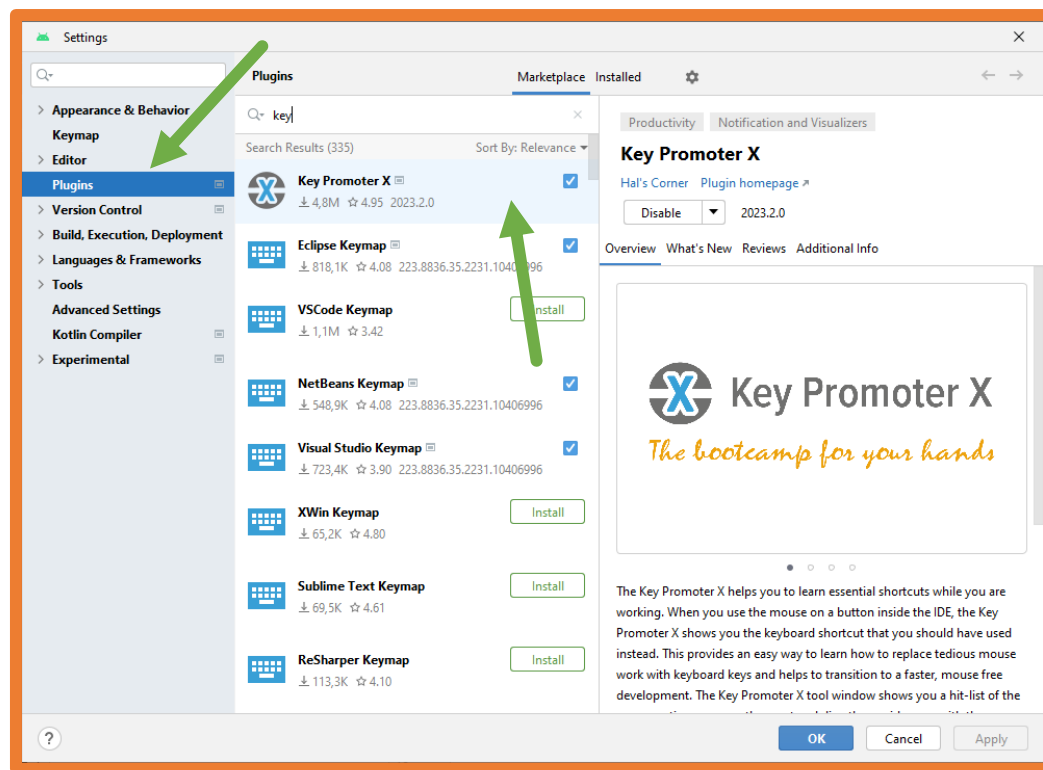
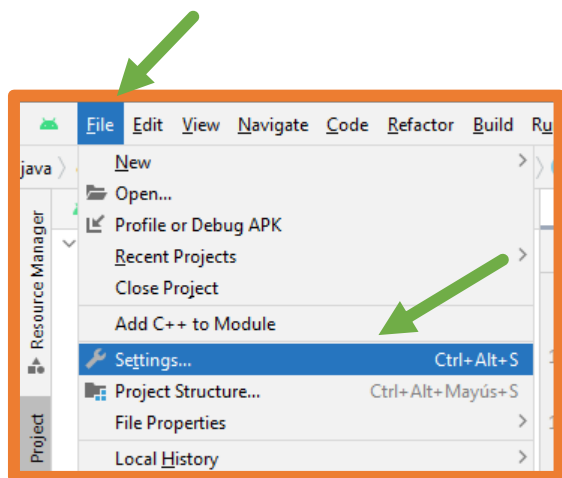
Android Studio puede mostrar en el editor diferentes pistas en el código que ayudan al programador a conocer el orden de los parámetros en las llamadas a las funciones, contexto en el que se está...



11.- Configuración de Android Studio

Plugins

Android Studio dispone de los mismos plugins que IntelliJ IDEA.



12.- Unidades de medida en Android

A la hora de desarrollar aplicaciones Android es muy importante conocer las unidades de medida que se utilizan.

Android puede utilizar las siguientes unidades de medida:

- dp → **d**ensity-independent **p**ixels
- sp → **s**cale-independent **p**ixels
- in → pulgadas
- mm → milímetros
- pt → puntos
- px → píxeles.

12.- Unidades de medida en Android

- **dp (densiti-independent pixels):**

1dp equivale a un píxel en una pantalla de 160dpi.

Es una **unidad flexible que cambiará según los dpi de la pantalla:**

$$dp = (\text{ancho en píxeles} * 160) / \text{densidad de la pantalla}$$

Es la solución más eficiente para mostrar elementos de manera uniforme en pantallas con diferentes densidades.

Se usa para todos los tamaños/medidas/distancias menos las del texto.

- **sp (scale-independent pixels):**

Unidad similar a dp pero que **se escala según el tamaño de fuente.**

Se ajusta a la densidad de pantalla y a las **preferencias del usuario en el sistema.**

Se usa para texto.

12.- Unidades de medida en Android

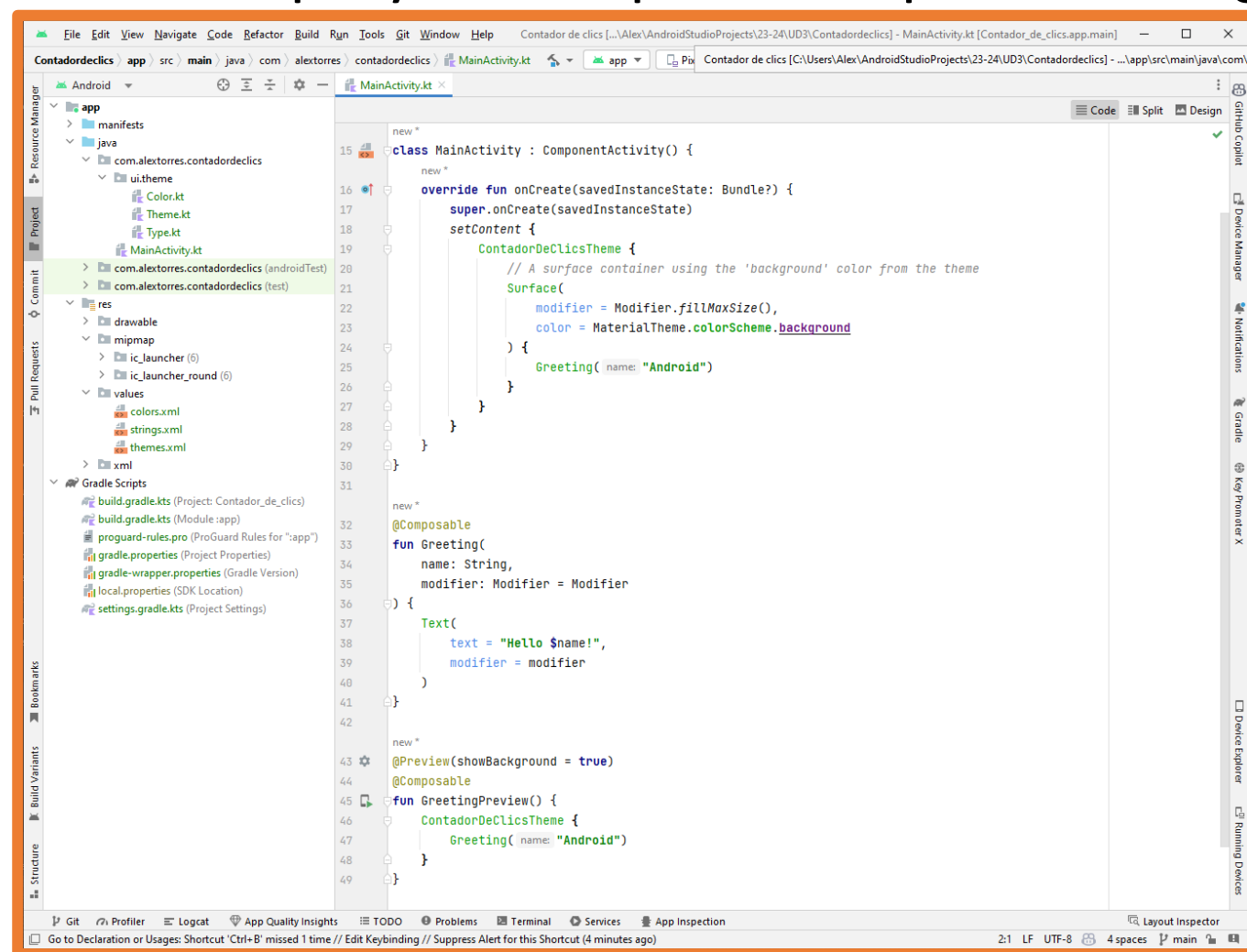
- **in** (pulgadas):
Pulgadas reales según el tamaño físico de la pantalla.
- **mm** (milímetros)
Milímetros reales según el tamaño físico de la pantalla.
- **pt** (puntos)
Un punto es $1/72$ de una pulgada según el tamaño físico de la pantalla.
- **px** (píxeles):
Se corresponde con un píxel **real** de la pantalla.
No se aconseja su uso debido a que los diferentes dispositivos tienen diferentes densidades de píxeles → **ppi** (pixels per inch).

UD3.2 – Introducción a Android

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

1.- Primera aplicación Android: Contador de clics

El contenido inicial de un proyecto Jetpack Compose es el siguiente:



1.- Primera aplicación Android: Contador de clics

Se pueden distinguir tres partes:

Clase MainActivity:

- Extiende a `ComponentActivity` la cual es una Activity que permite componentes de Jetpack Compose.
- Esta función contiene el método `onCreate` que será el que se ejecute al iniciar la aplicación.
- Dentro de `onCreate` se carga el tema del proyecto y dentro de él se llama a un componente `Surface` que a su vez llama a la función `Greeting`.

Función Greeting:

- Recibe un `String` y un `Modifier` y genera un componente `Text` de Jetpack Compose.
- Esta función es un componente de Jetpack Compose ya que está etiquetada con **@Composable**.

Función GreetingPreview:

- Carga el tema del proyecto y dentro de él llama a la función `Greeting`.
- Esta función es un componente de Jetpack Compose ya que está etiquetada con **@Composable**.
- Esta función permite que se pueda previsualizar su contenido al estar etiquetada con **@Preview**.

```
class MainActivity : ComponentActivity() {
    new *
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ContadorDeClicsTheme {
                // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    Greeting(name: "Android")
                }
            }
        }
    }
}

new *
@Composable
fun Greeting(
    name: String,
    modifier: Modifier = Modifier
) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

new *
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    ContadorDeClicsTheme {
        Greeting(name: "Android")
    }
}
```

1.- Primera aplicación Android: Contador de clics

En el código se pueden ver los siguientes componentes Jetpack Compose:

Surface: componente del sistema que utiliza Material Design que permite definir una elevación, un fondo...

Text: componente del sistema para mostrar texto

ContadorDeClicksTheme: componente propio que se crea con el proyecto y extiende al tema por defecto para Material Design. Está definido en el archivo `ui.theme/Theme.kt`.

Greeting: componente propio que extiende la funcionalidad del componente `Text`.

GreetingPreview: componente propio que sirve para previsualizar el componente `Greeting`.

1.- Primera aplicación Android: Contador de clics

@Composable

Todos los componentes Jetpack Compose, ya sean del sistema o propios, son funciones que deben estar etiquetadas con **@Composable**.

Propios

```
@Composable  
fun Greeting(  
    name: String,  
    modifier: Modifier = Modifier  
) {
```

```
@Composable  
fun ContadorDeClicsTheme(  
    darkTheme: Boolean = isSystemInDarkTheme(),  
    // Dynamic color is available on Android 12+  
    dynamicColor: Boolean = true,  
    content: @Composable () -> Unit  
) {
```

Del sistema

```
@Composable  
@NonRestartableComposable  
fun Surface(  
    modifier: Modifier = Modifier,  
    shape: Shape = RectangleShape,  
    color: Color = MaterialTheme.co
```

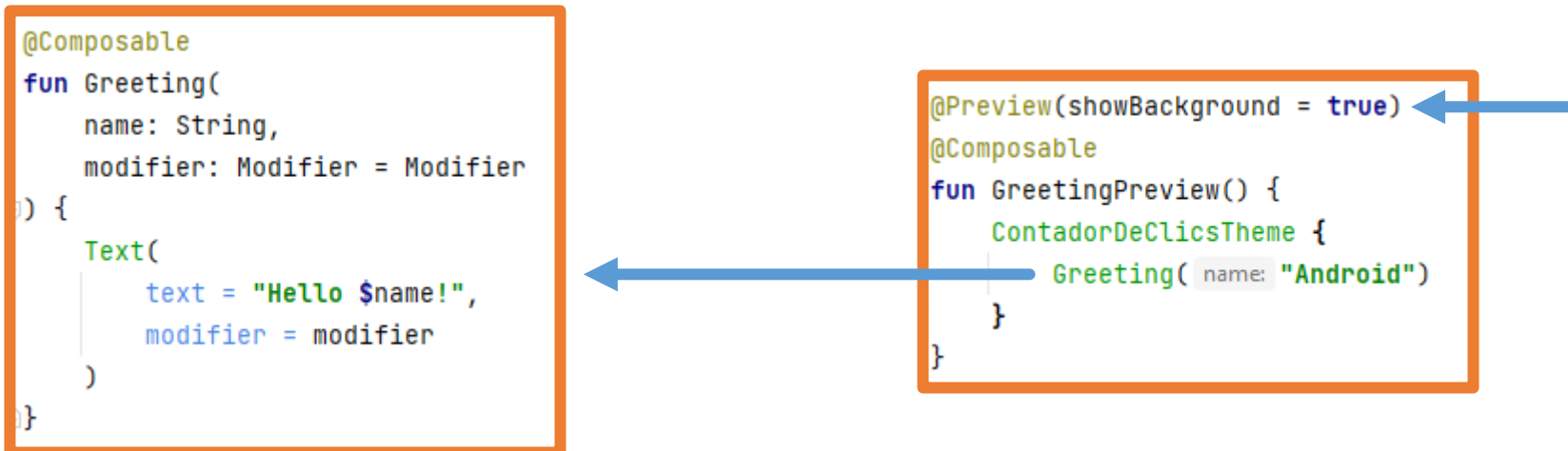
```
@Composable  
fun Text(  
    text: String,  
    modifier: Modifier = Modifier,  
    color: Color = Color.Unspecified,  
    fontSize: TextUnit = TextUnit.Unspecified,
```

1.- Primera aplicación Android: Contador de clics

@Preview

Android Studio permite ver **una previsualización en tiempo real** de los componentes que se definan, para ello se debe etiquetar un componente con **@Preview** como ocurre con la función GreetingPreview.

No se pueden previsualizar componentes que reciben funciones, para solucionar esto se crean componentes que envuelvan a esos que reciben funciones.



1.- Primera aplicación Android: Contador de clics

@Preview

Es muy importante que la previsualización muestre lo mismo que se mostrará en la ejecución de la aplicación por eso se puede realizar el siguiente cambio:

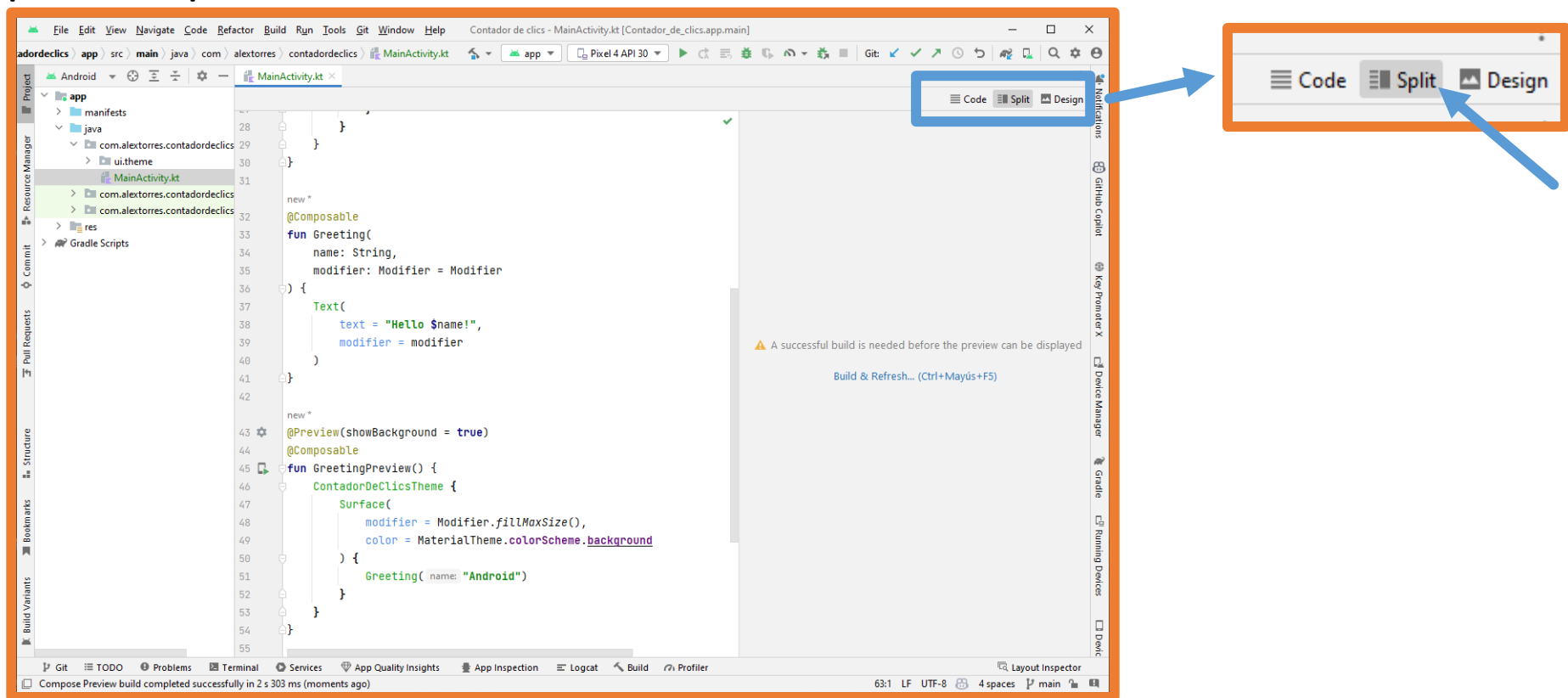
```
class MainActivity : ComponentActivity() {  
    new *  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            ContadorDeClicsTheme {  
                // A surface container using the 'background' color from the theme  
                Surface(  
                    modifier = Modifier.fillMaxSize(),  
                    color = MaterialTheme.colorScheme.background  
                ) {  
                    Greeting( name: "Android")  
                }  
            }  
        }  
    }  
}
```

```
@Preview(showBackground = true)  
@Composable  
fun GreetingPreview() {  
    ContadorDeClicsTheme {  
        Surface(  
            modifier = Modifier.fillMaxSize(),  
            color = MaterialTheme.colorScheme.background  
        ) {  
            Greeting( name: "Android")  
        }  
    }  
}
```

1.- Primera aplicación Android: Contador de clics

@Preview

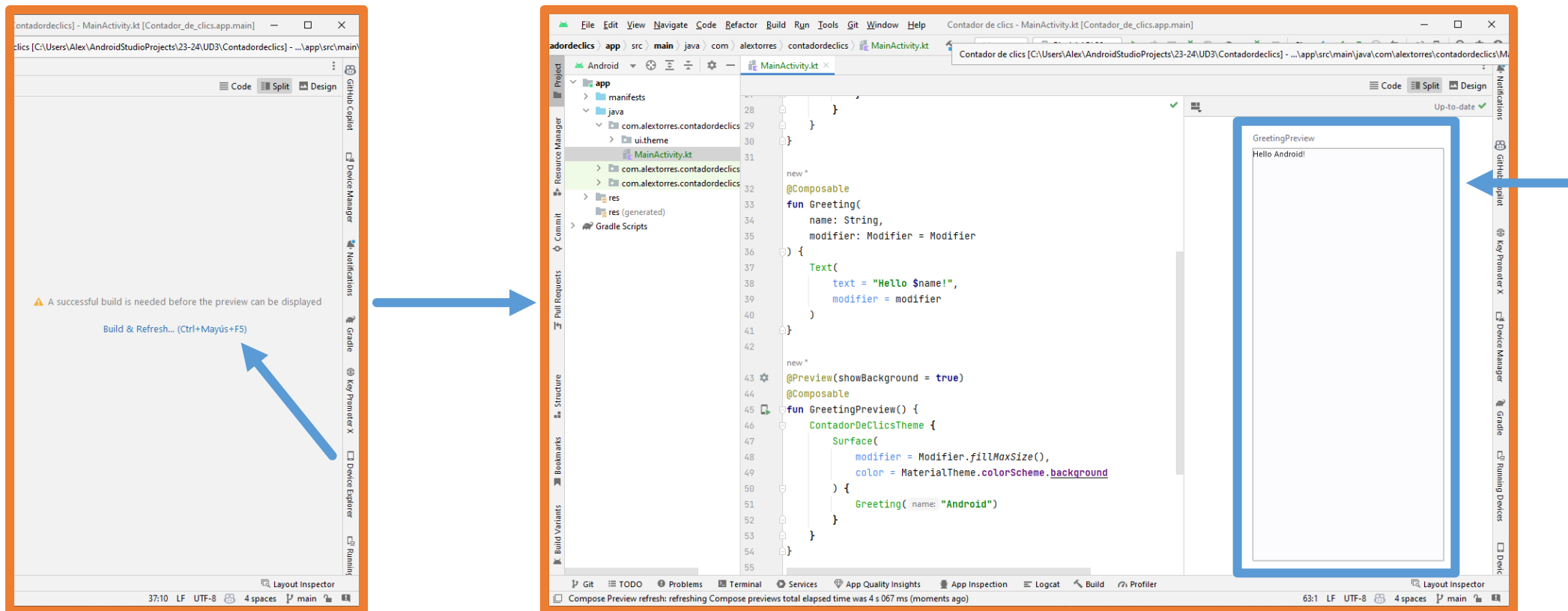
Para poder ver las previsualizaciones en Android Studio se debe seleccionar la opción **Split** en la parte superior derecha.



1.- Primera aplicación Android: Contador de clics

@Preview

La primera vez que se quiere previsualizar un componente y cuando hay cambios grandes o errores en el build se deberá pulsar en **Build & Refresh...**



1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

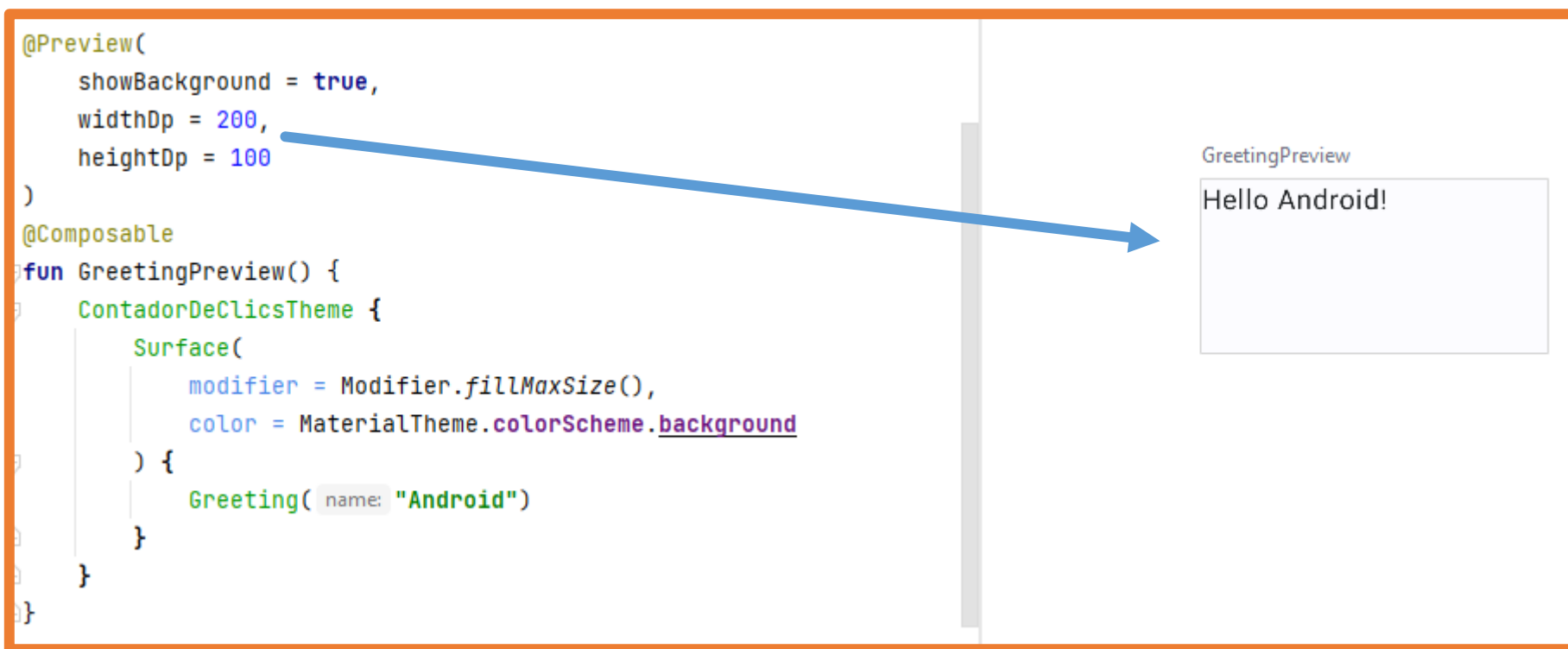
Pulsando la tecla CONTROL y haciendo clic sobre @Preview se pueden ver todas las opciones disponibles:

```
@annotation class Preview(  
    val name: String = "",  
    val group: String = "",  
    @IntRange(from = 1) val apiLevel: Int = -1,  
    // TODO(mount): Make this Dp when they are inline classes  
    val widthDp: Int = -1,  
    // TODO(mount): Make this Dp when they are inline classes  
    val heightDp: Int = -1,  
    val locale: String = "",  
    @FloatRange(from = 0.01) val fontScale: Float = 1f,  
    val showSystemUi: Boolean = false,  
    val showBackground: Boolean = false,  
    val backgroundColor: Long = 0,  
    @UiMode val uiMode: Int = 0,  
    @Device val device: String = Devices.DEFAULT,  
    @Wallpaper val wallpaper: Int = Wallpapers.NONE,  
)
```

1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

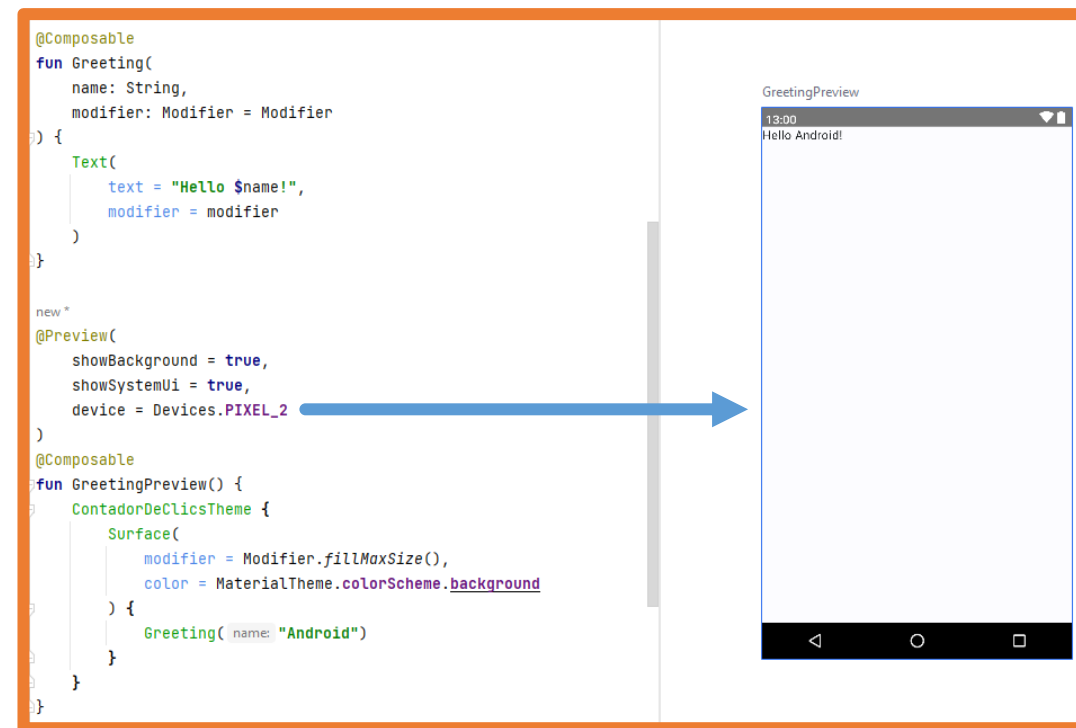
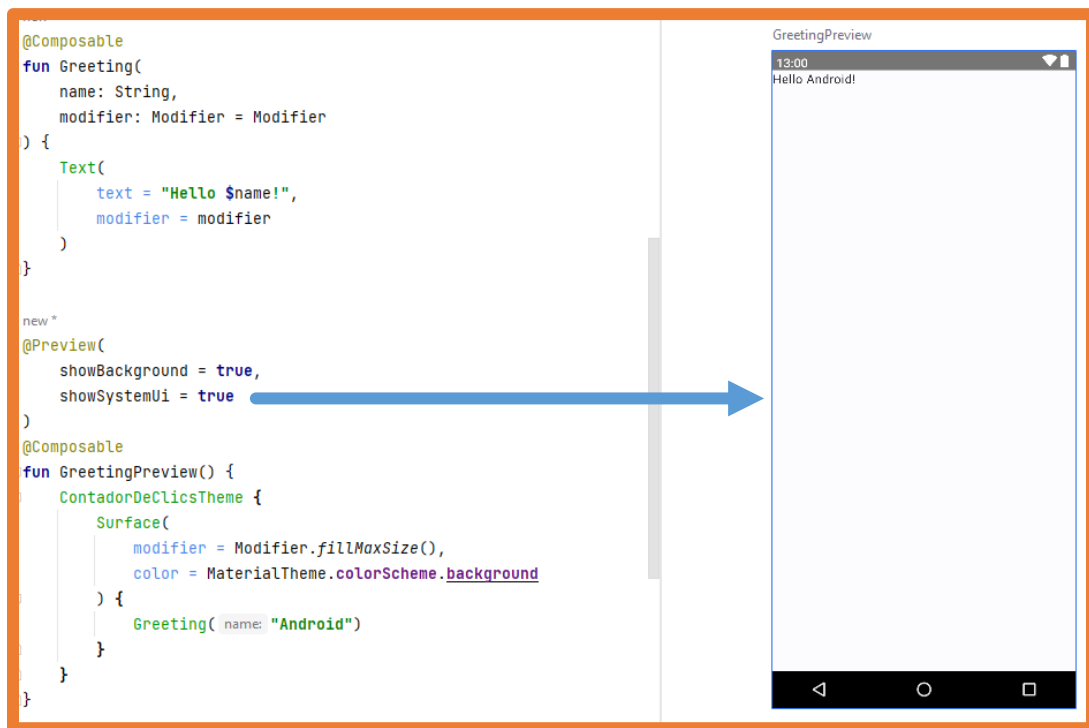
Por ejemplo, se puede indicar un tamaño a la previsualización:



1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

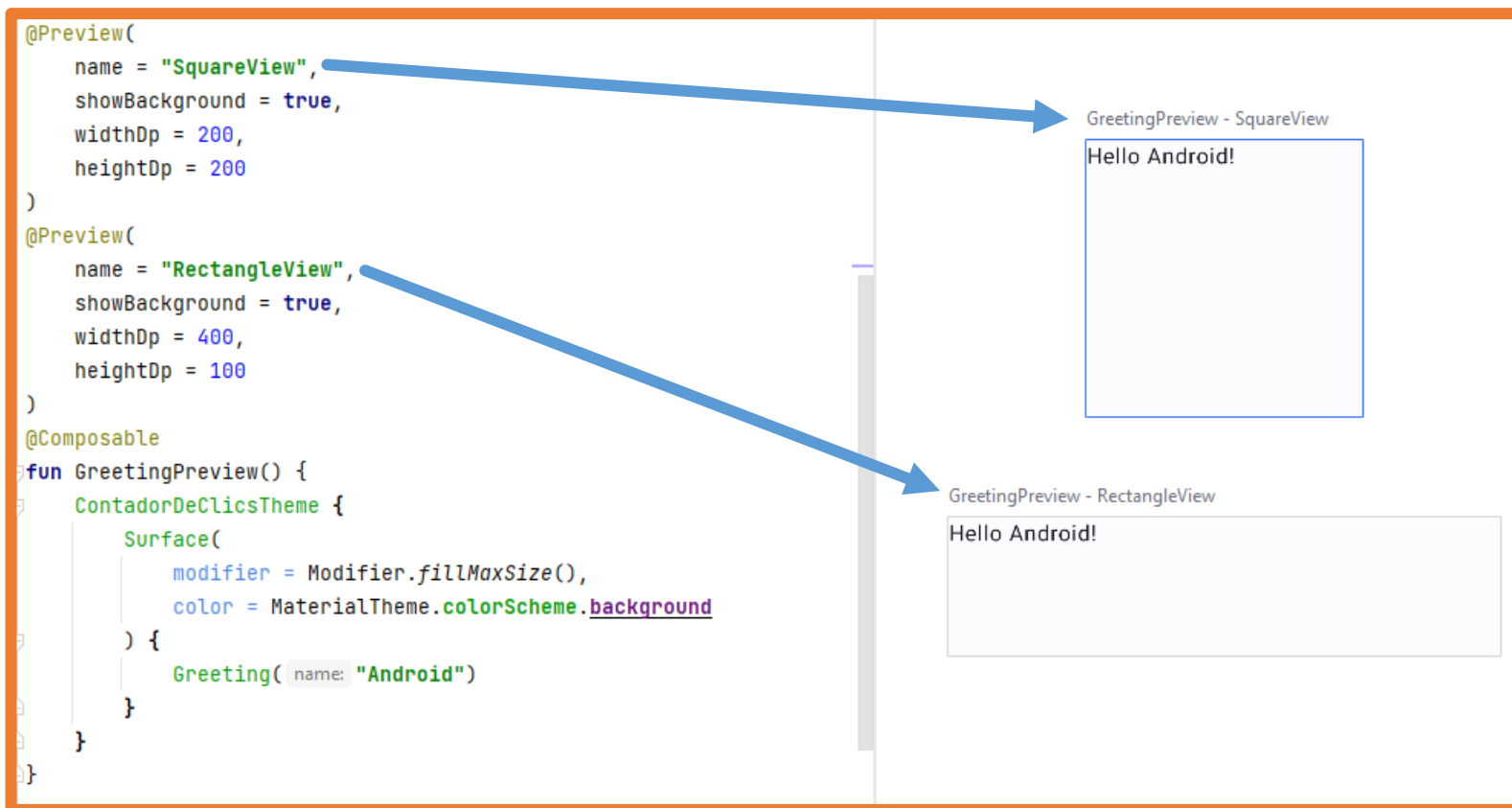
Se puede ver el componente dentro de la interfaz del sistema o incluso indicar un dispositivo concreto:



1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

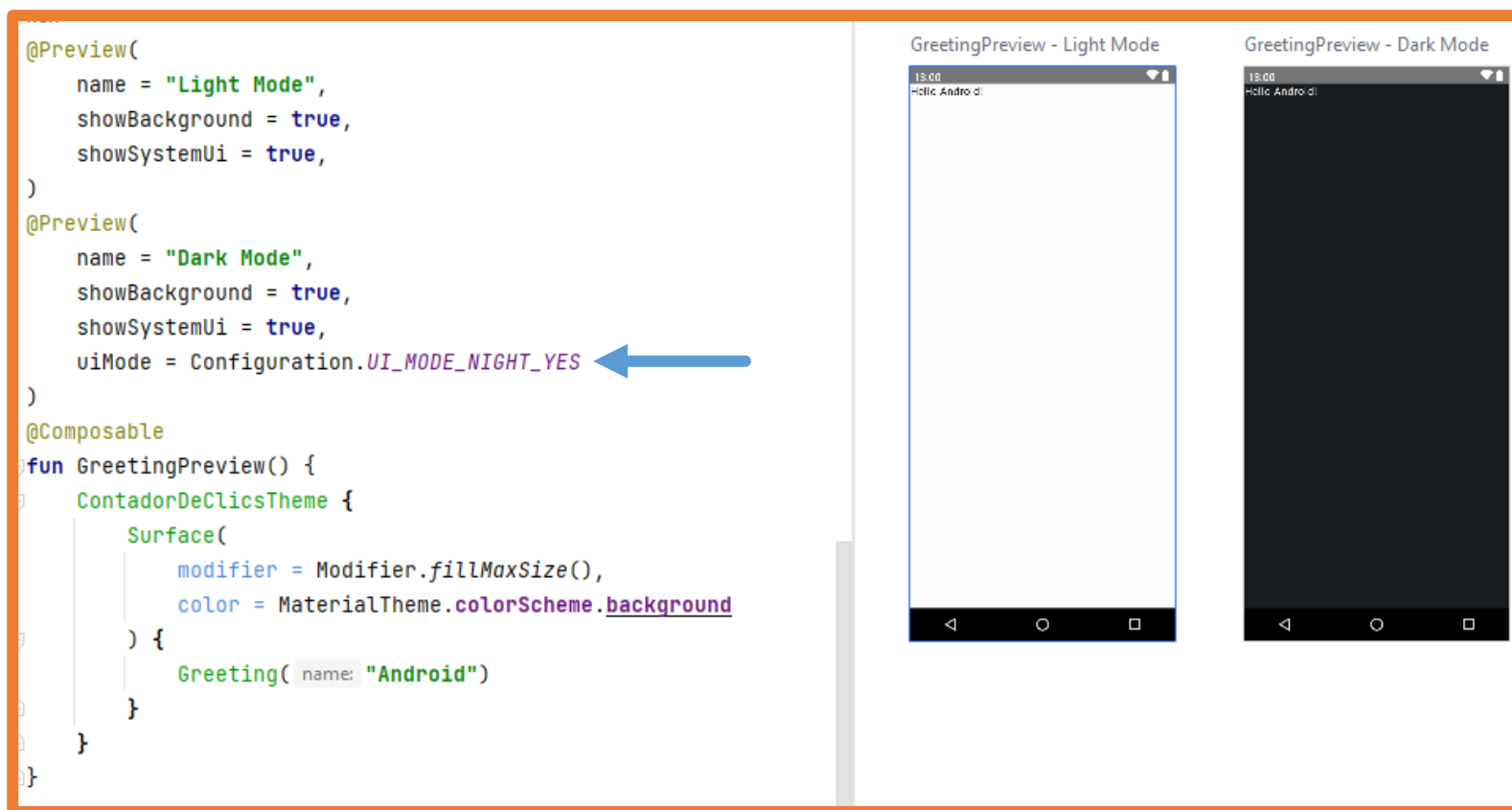
Se pueden crear varias previsualizaciones para un componente:



1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

Esto tiene especial utilidad para mostrar los modos claro y oscuro.



1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

También pueden haber varios componentes con previsualización:

```
@Preview(
    showBackground = true,
    widthDp = 200,
    heightDp = 100
)
@Composable
fun WelcomePreview() {
    Text(text: "Hola Rick!")
}

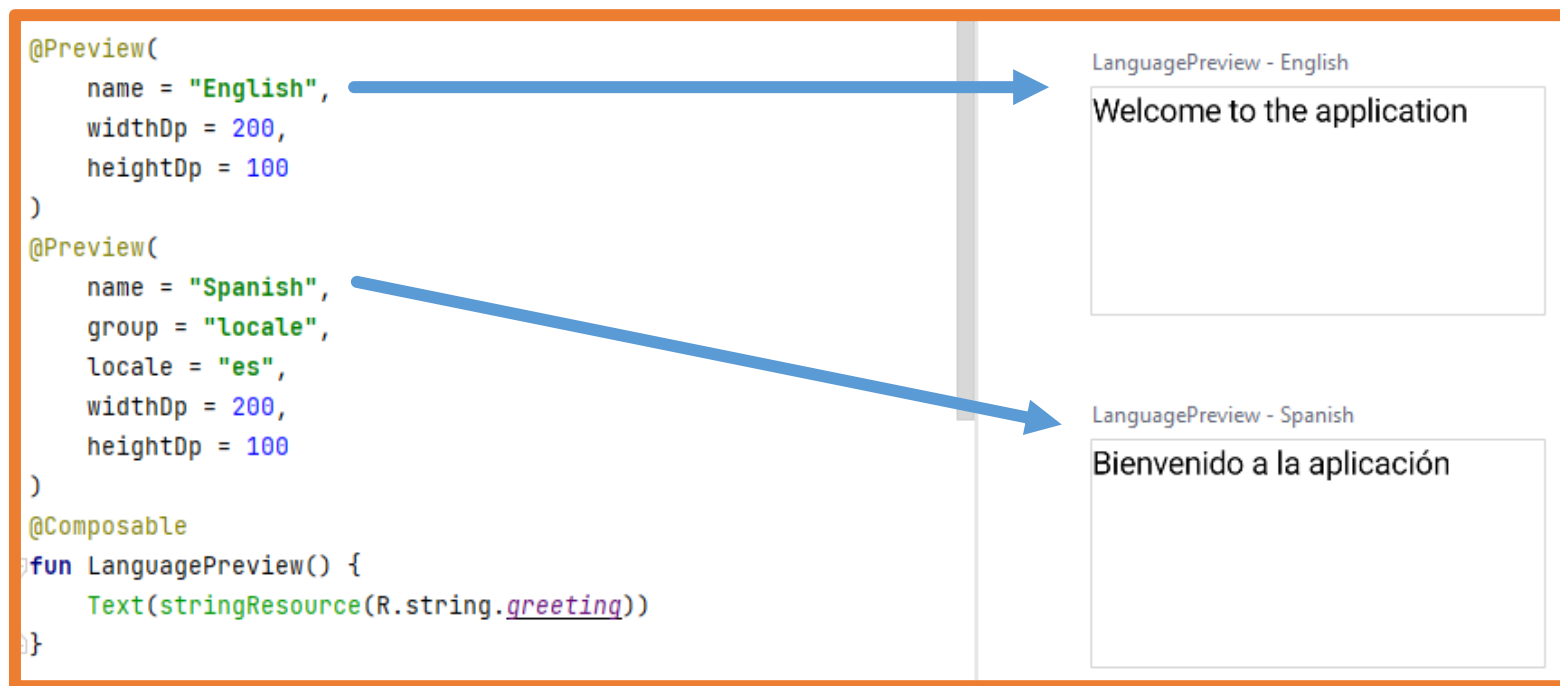
new *
@Preview(showBackground = true)
@Composable
fun GoodbyePreview() {
    Text(text: "Adiós Rick!")
}
```

The diagram illustrates the mapping between Kotlin code and Android Studio previews. On the left, two code snippets are shown. The first snippet defines a function `WelcomePreview()` with a `Text` component displaying "Hola Rick!". A blue arrow points from this function name to a preview window titled "WelcomePreview" which shows the text "Hola Rick!". The second snippet defines a function `GoodbyePreview()` with a `Text` component displaying "Adiós Rick!". A blue arrow points from this function name to a preview window titled "GoodbyePreview" which shows the text "Adiós Rick!".

1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

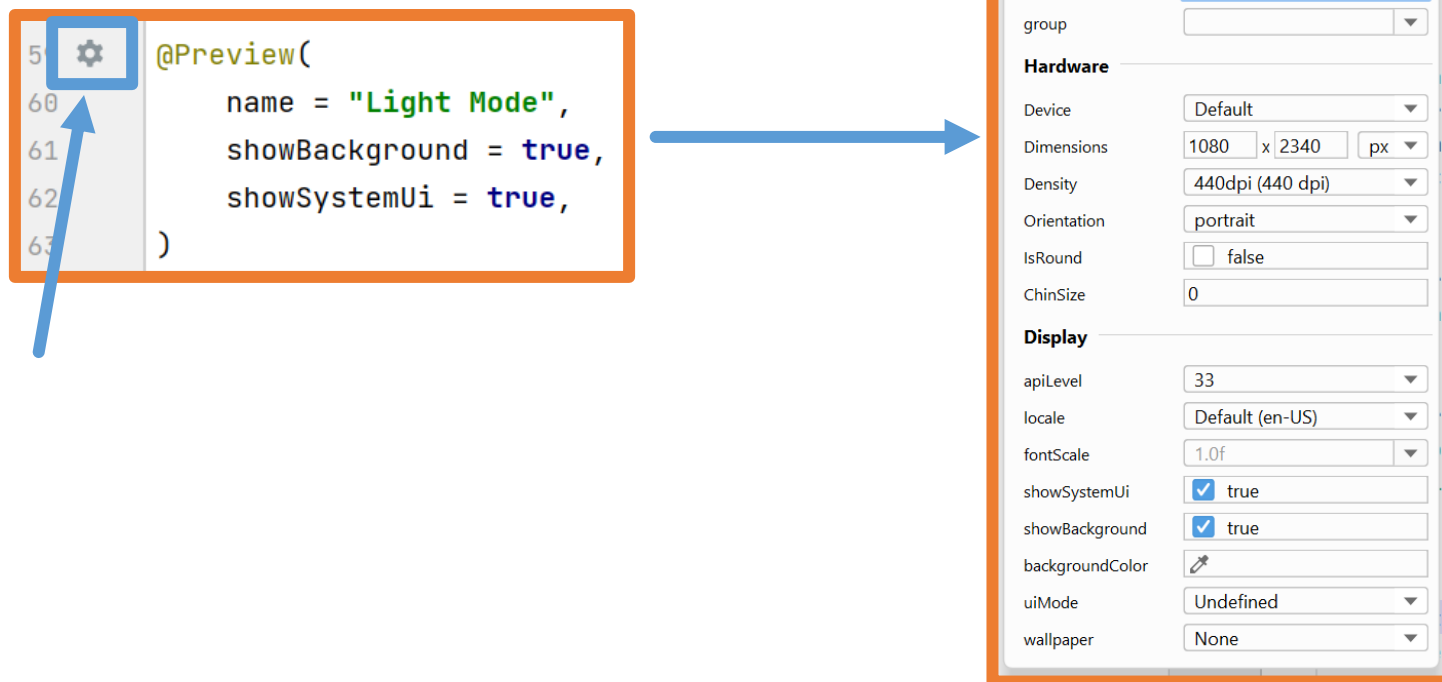
Las previsualizaciones también permiten ver cómo quedan los componentes cuando se está desarrollando una aplicación multi idioma:



1.- Primera aplicación Android: Contador de clics

@Preview – Opciones

Cuando se crea una previsualización Android Studio marca esa línea para poder acceder a todas las opciones disponibles desde un menú contextual:



1.- Primera aplicación Android: Contador de clics

@Preview – Limitaciones

Las previsualizaciones tienen una serie de limitaciones:

- No pueden recibir parámetros.
- No tienen acceso a los archivos.
- No tienen acceso a la red (no cargarán datos de internet).
- Algunas API no funcionan completamente bien.

En la documentación oficial está toda la información sobre [@Preview](#).

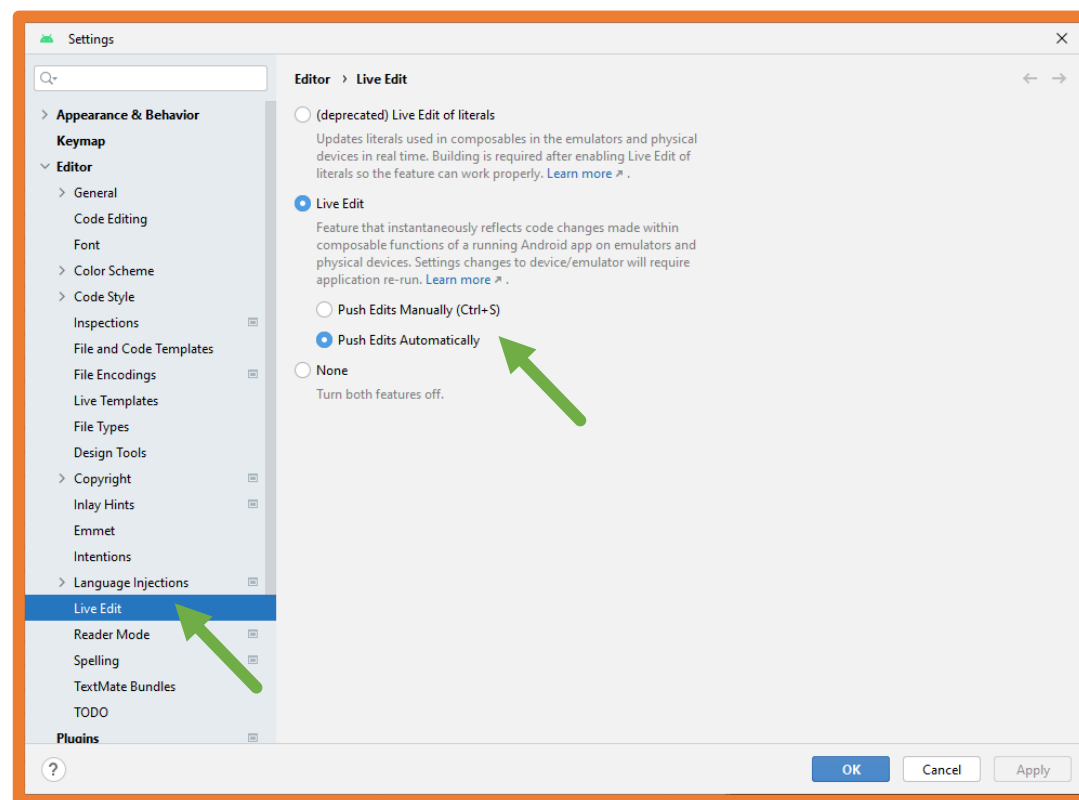
Las últimas versiones de Android Studio han mejorado la ejecución de Jetpack Compose y permiten visualizar los cambios en tiempo real → **Live Edit**.

1.- Primera aplicación Android: Contador de clics

Live Edit

Si se quiere que los cambios en el código actualicen automáticamente la aplicación en el emulador, se debe configurar la opción **Live Edit** de Android Studio.

File → Settings (CONTROL+ALT+S)



1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

La aplicación **Contador de clics** necesita un texto y un botón así que se va a modificar el código para que lo incluya.

Lo primero será **eliminar la función Greeting y todas sus llamadas**.

También se cambiará el nombre de la preview GreetingPreview por el nombre **ContentPreview**.

A continuación, se creará un componente Jetpack Compose llamado **Content**.

```
class MainActivity : ComponentActivity() {
    new *
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ContadorDeClicsTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                }
            }
        }
    }
}

new *
@Composable
fun Content() {
}

new *
@Preview(showBackground = true)
@Composable
fun ContentPreview() {
    ContadorDeClicsTheme {
        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
        }
    }
}
```

1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

En el componente **Content** se añaden tanto el texto como el botón.

El texto se añade con un componente **Text** y el botón se añade con un componente **Button**.

Se puede observar que el componente **Button** recibe una función lambda como parámetro **onClick** y otra función lambda como contenido del propio botón.

El contenido del botón en este caso es un texto.

```
@Composable
fun Content() {
    Text(text: "Has hecho clic 0 veces")
    Button(onClick = {

    }) { this: RowScope
        Text(text: "¡PÚLSAME!")
    }
}
```

1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

Como se estudió en la UD2, en Kotlin si el último parámetro es una función lambda, se puede extraer ese parámetro fuera de los paréntesis.

Aunque las dos maneras funcionan igual, en Jetpack Compose si el último parámetro es una función lambda se extrae fuera de los paréntesis.

```
@Composable
fun Content() {
    Text(text: "Has hecho clic 0 veces")
    Button(
        onClick = {
        },
        content = { this: RowScope
            Text(text: "¡PÚLSAME!")
        }
    )
}
```



```
@Composable
fun Content() {
    Text(text: "Has hecho clic 0 veces")
    Button(onClick = {
    }) { this: RowScope
        Text(text: "¡PÚLSAME!")
    }
}
```



1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

Para poder previsualizar la interfaz gráfica y ver la interfaz gráfica en el emulador se deben añadir llamadas a la función **Content** tanto en **onCreate** como en **ContentPreview**.

```
class MainActivity : ComponentActivity() {  
    new *  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            ContadorDeClicsTheme {  
                Surface(  
                    modifier = Modifier.fillMaxSize(),  
                    color = MaterialTheme.colorScheme.background  
                ) {  
                    Content() ←  
                }  
            }  
        }  
    }  
}
```

```
@Preview(showBackground = true)  
@Composable  
fun ContentPreview() {  
    ContadorDeClicsTheme {  
        Surface(  
            modifier = Modifier.fillMaxSize(),  
            color = MaterialTheme.colorScheme.background  
        ) {  
            Content() ←  
        }  
    }  
}
```

1.- Primera aplicación Android: Contador de clics

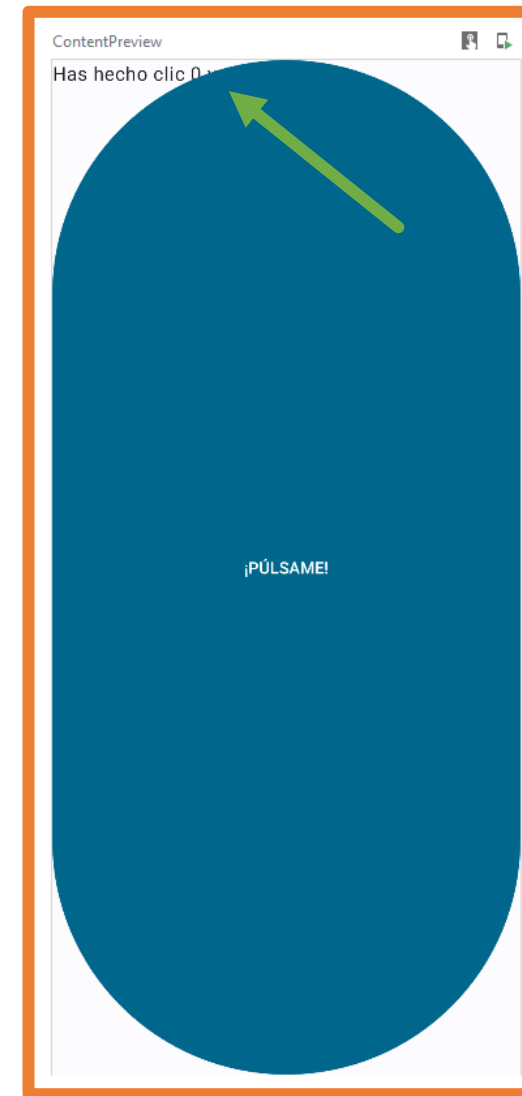
Contenido de la aplicación

En la previsualización se puede observar que los dos componentes ocupan toda la pantalla (se nota más con el botón).

También se puede observar que los dos componentes se superponen, esto es debido a que no hay ningún componente de tipo **layout** en la interfaz.

Los componentes de tipo **layout** permiten organizar los componentes de la interfaz gráfica.

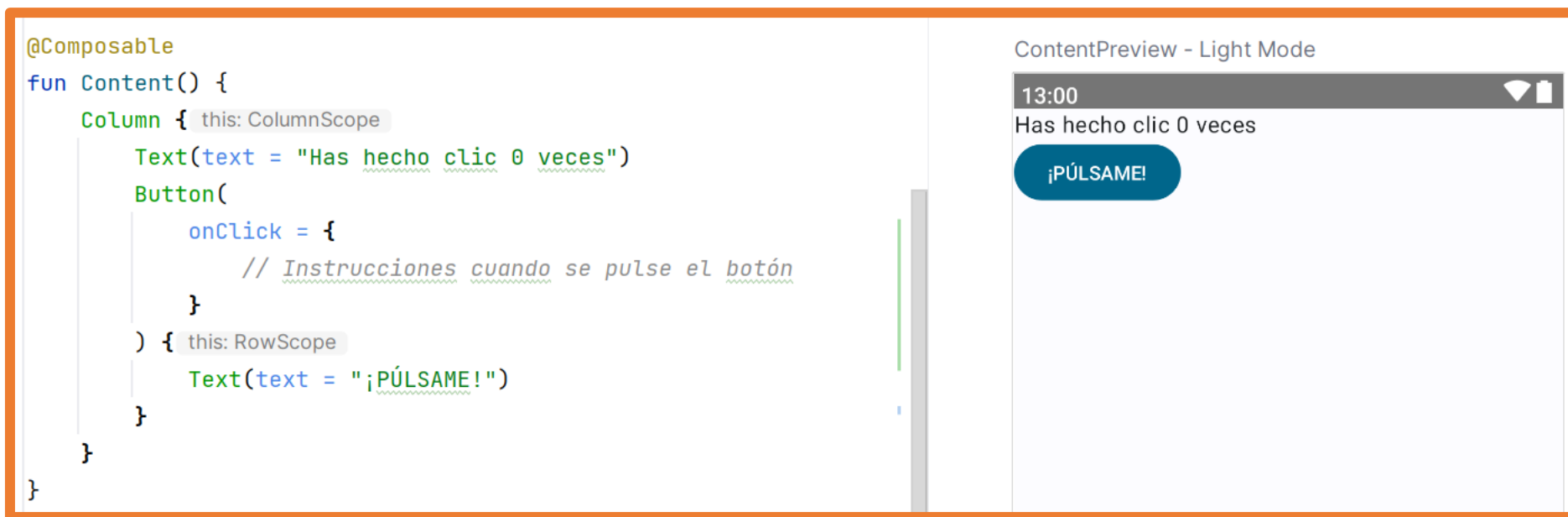
Para solucionar esto se va a utilizar el componente **Column** que permite organizar la interfaz en forma de columna.



1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

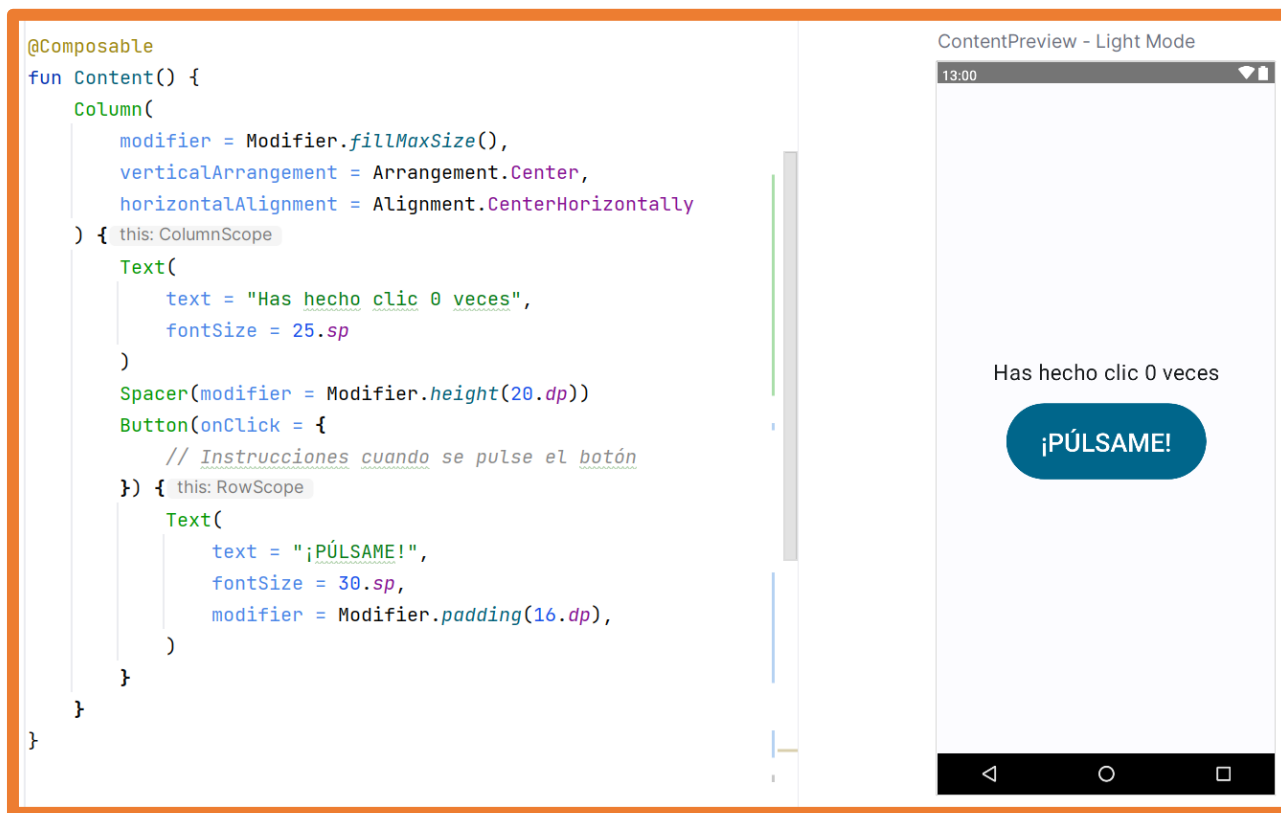
Se añade el componente **Column** y se introducen en él tanto el texto como el botón.



1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

A continuación, se añaden algunas modificaciones para mejorar la interfaz.



1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

Este código es **una muestra de las buenas prácticas** programando en Kotlin con Jetpack Compose:

- Se utilizan los parámetros con nombre en las llamadas.
- Si en la llamada hay varios parámetros, estos se escriben cada uno en una línea.
- Si el último parámetro es una función lambda se extrae de los paréntesis.

```
@Composable
fun Content() {
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) { this: ColumnScope
        Text(
            text = "Has hecho clic 0 veces",
            fontSize = 25.sp
        )
        Spacer(modifier = Modifier.height(20.dp))
        Button(onClick = {
            // Instrucciones cuando se pulse el botón
        }) { this: RowScope
            Text(
                text = "¡PÚLSAME!",
                fontSize = 30.sp,
                modifier = Modifier.padding(16.dp),
            )
        }
    }
}
```

1.- Primera aplicación Android: Contador de clics

Funcionalidad de la aplicación

En este punto ya se ha terminado con la UI declarativa.

Ahora es el momento de implementar la funcionalidad de la aplicación.

Se necesita una variable de tipo entera para almacenar la cantidad de veces que se ha hecho clic.

Se crea esa variable inicializada a cero y se incluye en el primer texto.

En el código del botón se añade uno a esa variable.

```
@Composable
fun Content() {
    var times = 0
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) { this: ColumnScope
        Text(
            text = "Has hecho clic $times veces",
            fontSize = 25.sp
        )
        Spacer(modifier = Modifier.height(20.dp))
        Button(onClick = {
            times++
        }) { this: RowScope
            Text(
                text = "¡PÚLSAME!",
                fontSize = 30.sp,
                modifier = Modifier.padding(16.dp),
            )
        }
    }
}
```

1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

Si se ejecuta en el emulador la aplicación, aunque se puede comprobar que el botón funciona no se actualiza el número de veces.

Esto es debido a que como se indicó anteriormente los elementos de la interfaz se conectan al **estado de la Activity** y si el estado de la Activity no se actualiza la interfaz no se vuelve a pintar.

Para solucionar esto se debe cambiar la variable para que sea una **variable de estado** de esta manera cuando esta variable cambie el estado de la Activity también lo hará y se volverá a pintar la interfaz.

1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación

```
@Composable
fun Content() {
    var times by remember { mutableStateOf( value: 0) }
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
```

Con este cambio se puede ejecutar la aplicación y comprobar que la aplicación funciona.

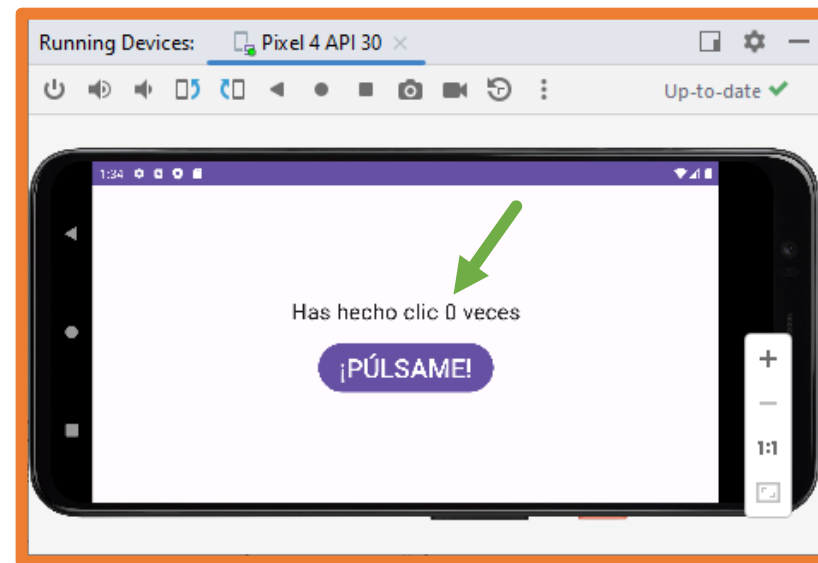
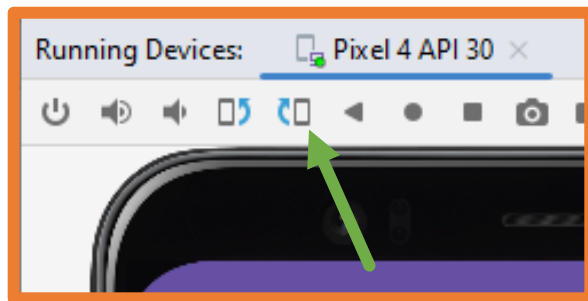
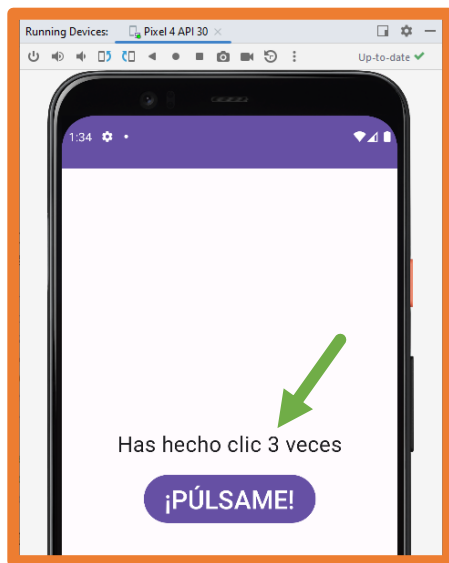
1.- Primera aplicación Android: Contador de clics

Contenido de la aplicación – Estados de Activity

Al principio de la unidad se explicó que hay en situaciones en las que la Activity se destruye y se vuelve a crear, por ejemplo, al cambiar la orientación del dispositivo.

Cuando esto ocurre se ejecuta la Activity desde el principio por lo que las variables se vuelven a crear.

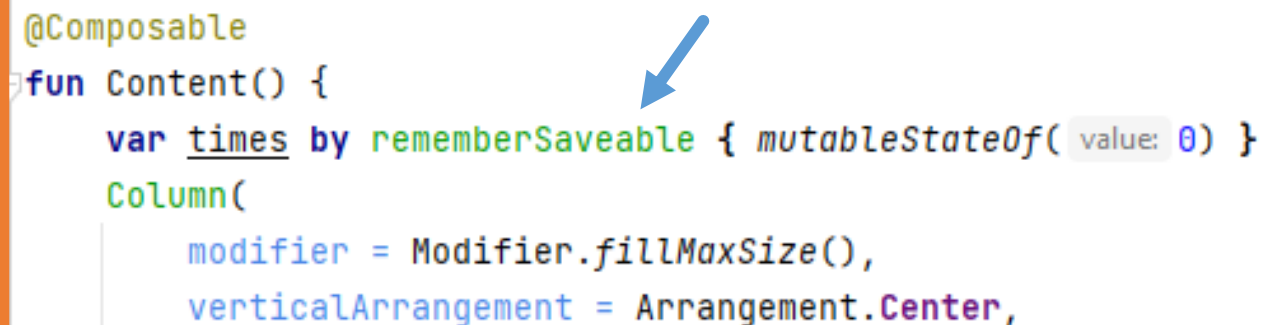
Si se cambia la orientación en la aplicación se observará que el número de clics se pierde.



1.- Primera aplicación Android: Contador de clics

Estados de Activity

Para solucionar este comportamiento se debe cambiar la declaración de la variable para que se guarde aunque se destruya la Activity.



```
@Composable
fun Content() {
    var times by rememberSaveable { mutableStateOf(0) }
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
```

A blue arrow points to the `rememberSaveable` function in the code snippet, highlighting the change from `remember` to `rememberSaveable` to ensure state persistence across Activity restarts.

Con este cambio se puede ejecutar la aplicación y comprobar que la aplicación funciona correctamente aunque se cambie de orientación, de modo claro/oscuro o incluso la configuración del dispositivo.

Práctica

Actividad 0

Contador de clics

Actividad 1

Estadísticas

Actividad 2

Conversor

UD3.3 – Introducción a Android

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

1.- Centralizar valores en el proyecto

En la aplicación se han utilizado literales para los Strings y para los tamaños.

```
Text(  
    text = ";PÚLSAME!",  
    fontSize = 30.sp,  
    modifier = Modifier.padding(16.dp)  
)
```

A esta práctica se le denomina como **Hardcode values**.

Esto **no es una buena práctica**:

- ¿Qué pasa si se decide que todos los botones tengan el mismo tamaño de texto y en un futuro se cambia este tamaño?
- ¿Qué se debe hacer si se quiere crear una aplicación multi idioma?

1.- Centralizar valores en el proyecto

En Android Studio se pueden **centralizar valores** (similar a crear variables) para poder utilizar esos valores en cualquier punto del proyecto.

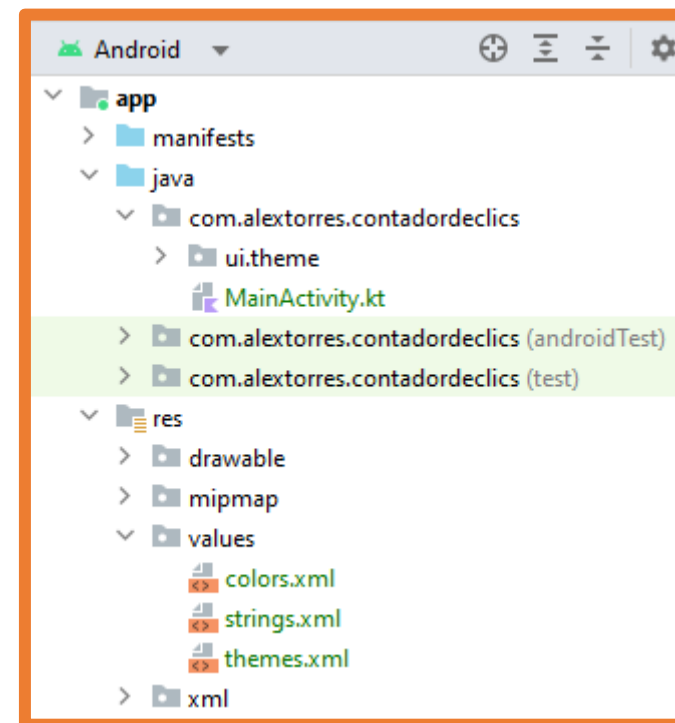
De esta manera los cambios serán más sencillos de realizar.

Como buena práctica se deberán centralizar al menos todas las cadenas de texto.

1.- Centralizar valores en el proyecto

Por defecto Android Studio ya crea dos archivos para estas tareas:

- colors.xml → centralizar **colores**
- string.xml → centralizar **cadenas**



A continuación, se verá cómo usarlo y como crear un archivo similar para las **medidas y tamaños**.

1.- Centralizar valores en el proyecto

Se pueden **crear colores y cadenas** añadiendo el código directamente a los **archivos correspondientes**, siguiendo el patrón ya establecido en ellos.

`<color name="NOMBRE">#CODIGO_COLOR</color>`

El código de color se puede representar mediante dos Dígitos hexadecimales para cada componente:

Sin transparencia: RedGreenBlue

Con transparencia: AlphaRedGreenBlue

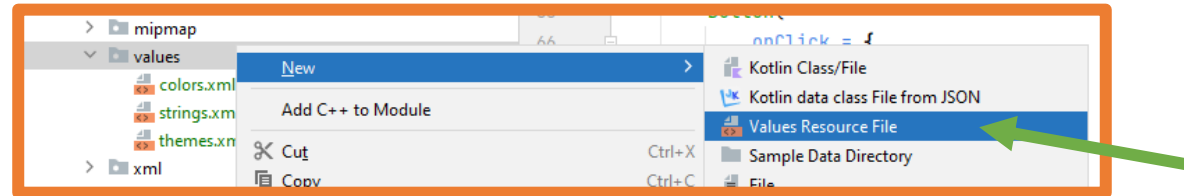
```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
     <color name="white">#FFFFFFFF</color>
</resources>
```

`<string name="NOMBRE">CADENA</string>`

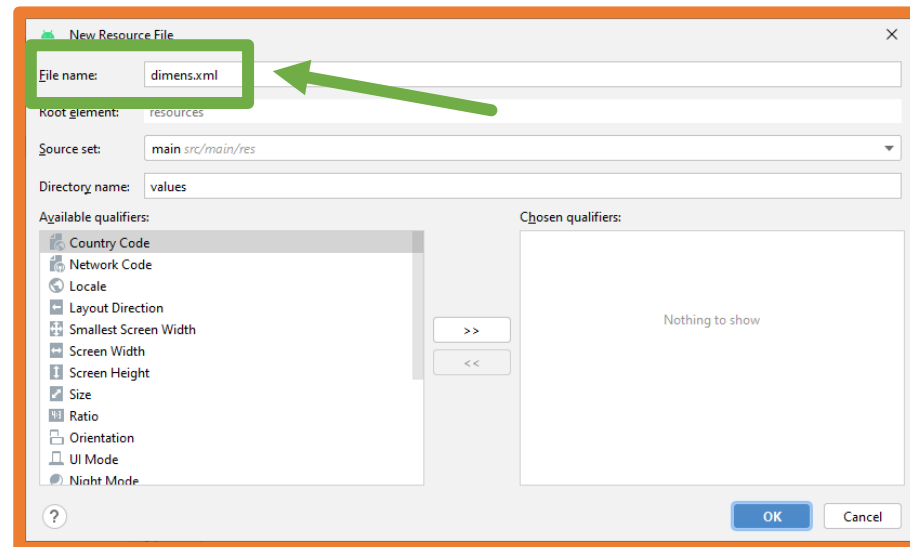
```
<resources>
     <string name="app_name">Contador de clics</string>
</resources>
```

1.- Centralizar valores en el proyecto

Para crear el archivo de dimensiones se deben seguir los siguientes pasos:
Clic derecho sobre **values** → **New** → **Values Resource File**.

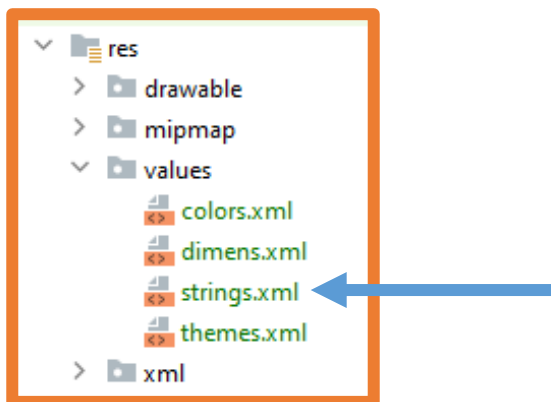


En la ventana que se abre se debe indicar el nombre del archivo: **dimens.xml** y el resto de opciones dejarlas por defecto.



1.- Centralizar valores en el proyecto

De esta manera se tendrá también el archivo **dimens.xml** disponible.



La estructura del archivo es similar al de los anteriores como se muestra en la siguiente imagen.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="button_text">25sp</dimen>
</resources>
```

Se pueden añadir todas las dimensiones que se necesiten.

2.- Uso de valores centralizados

Kotlin crea por defecto la clase **R** y la rellena con todos los recursos del proyecto (datos obtenidos de la carpeta **res**).

Así, una vez centralizados los valores se puede usando la clase **R** a todo lo que se ha definido en los archivos colors.xml, dims.xml, strings.xml.

```
R.string.counter_text
```

```
R.color.orange
```

```
R.dimen.button_text
```

2.- Uso de valores centralizados

Teniendo el acceso a los valores centralizados con la clase **R** se dispone de tres funciones para acceder a los valores centralizados según el archivo donde se encuentren:

- **stringResource**
- **dimensionResource** (si son sp se debe añadir detrás `.value.sp`)
- **colorResource**

```
Text(  
    text = "¡PÚLSAME!",  
    fontSize = 30.sp,  
    modifier = Modifier.padding(16.dp),  
)
```

```
Text(  
    text = stringResource(id = R.string.clickme),  
    fontSize = dimensionResource(id = R.dimen.button_text).value.sp,  
    modifier = Modifier.padding(dimensionResource(id = R.dimen.button_padding)),  
    color = colorResource(id = R.color.teal_200)  
)
```

3.- Añadir valores centralizados

Primera manera para añadir valores centralizados:

- Añadir directamente el valor al archivo XML correspondiente.

```
<resources>
  <string name="app_name">Contador de clics</string>
  <string name="greeting">Welcome to the application</string>
  <string name="clickme">¡PÚLSAME!</string>
</resources>
```

```
<resources>
  <dimen name="button_text">30sp</dimen>
  <dimen name="button_padding">16dp</dimen>
</resources>
```

```
<resources>
  <color name="purple_200">#FFBB86FC</color>
  <color name="purple_500">#FF6200EE</color>
  <color name="purple_700">#FF3700B3</color>
  <color name="teal_200">#FF03DAC5</color>
  <color name="teal_700">#FF018786</color>
  <color name="black">#FF000000</color>
  <color name="white">#FFFFFFFF</color>
  <color name="lightred">#FFFF2929</color>
  <color name="darkred">#FFAF0000</color>
</resources>
```

3.- Añadir valores centralizados

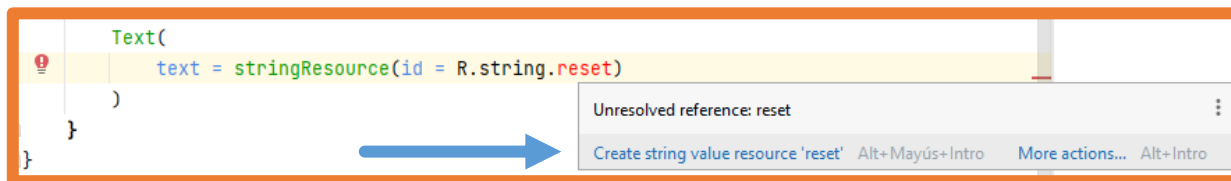
Segunda manera para añadir valores centralizados:

- En el código en Kotlin se puede indicar con una de las funciones anteriores un nuevo valor centralizado y desde ahí indicarle a Android Studio que añada el valor al XML.

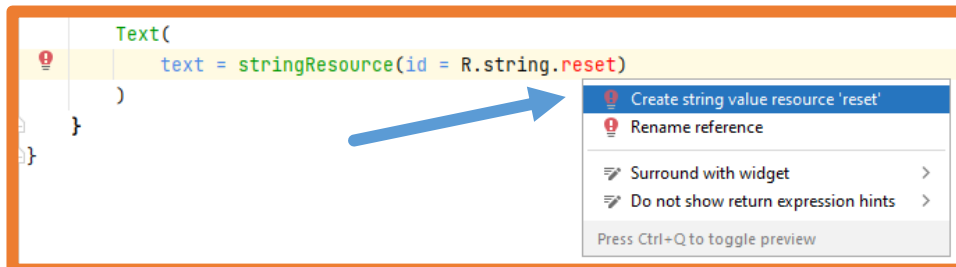
Al añadir el nuevo valor Android Studio lo marca como error:

```
Text(  
    text = stringResource(id = R.string.reset)  
)
```

Al situar el cursor encima del error aparece la ayuda contextual:



La ayuda contextual también aparece con el cursor encima del error al pulsar ALT+INTRO:

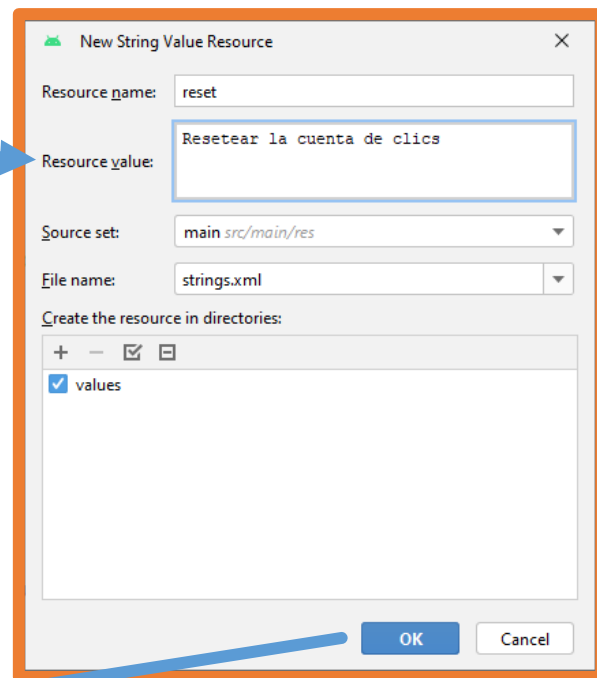


3.- Añadir valores centralizados

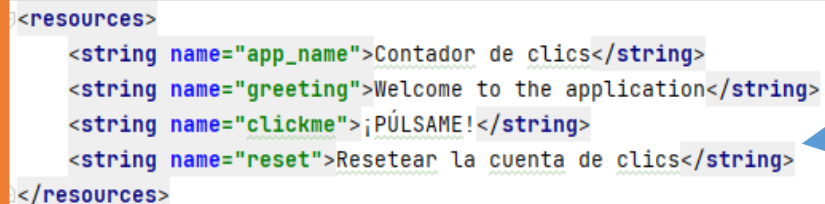
Segunda manera para añadir valores centralizados:

- En el código en Kotlin se puede indicar con una de las funciones anteriores un nuevo valor centralizado y desde ahí indicarle a Android Studio que añada el valor al XML.

Al pulsar **Create string value resource 'reset'** aparece la siguiente ventana en la que se debe rellenar el valor a centralizar.



Una vez añadido el valor y pulsado el botón **OK**, el valor se abrirá centralizado en su archivo.



```
<resources>
  <string name="app_name">Contador de clics</string>
  <string name="greeting">Welcome to the application</string>
  <string name="clickme">¡PÚLSAME!</string>
  <string name="reset">Resetea la cuenta de clics</string>
</resources>
```

4.- Aplicación multi idioma

Crear **aplicaciones multi idioma** con Android Studio es **muy sencillo**, simplemente se debe crear un archivo **strings.xml** para cada idioma soportado.

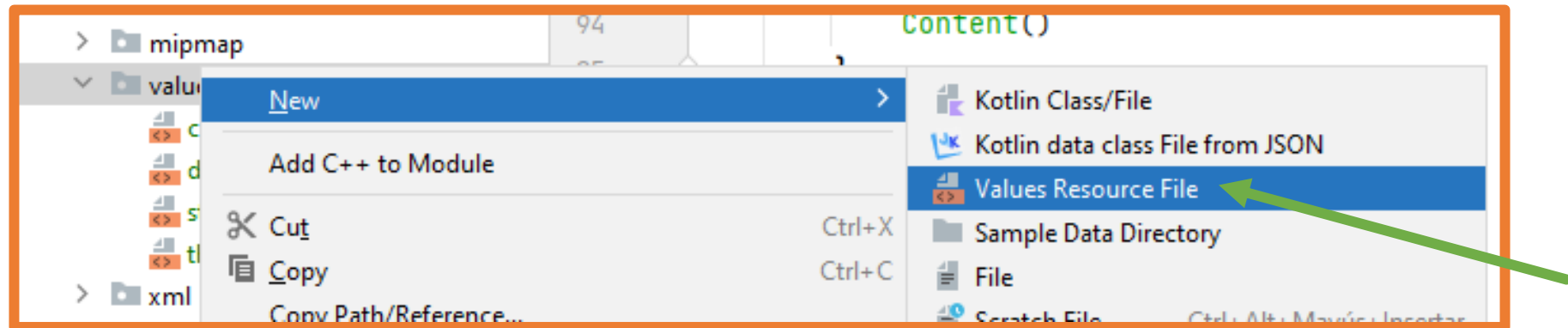
- Si solo se tiene un archivo **strings.xml** como ocurre nada más crear el proyecto, ese archivo se cargará sea cual sea el idioma configurado en el dispositivo.
- Si el dispositivo tiene un idioma configurado y no existe archivo **strings.xml** para ese idioma se cargará el archivo strings.xml por defecto (el que se crea junto al proyecto).
- Si el dispositivo tiene uno o más idiomas configurados y hay archivos **strings.xml** para esos idiomas, se cargará el primero de la lista de la configuración del móvil.
- Si se tienen varios archivos **strings.xml** y en uno no aparece una de las cadenas traducidas, se cargará la cadena en el idioma por defecto

Lo más habitual es que el archivo strings.xml creado con el proyecto sea para el idioma inglés.

4.- Aplicación multi idioma

Para crear un archivo strings.xml para un idioma se deben seguir los siguientes pasos:

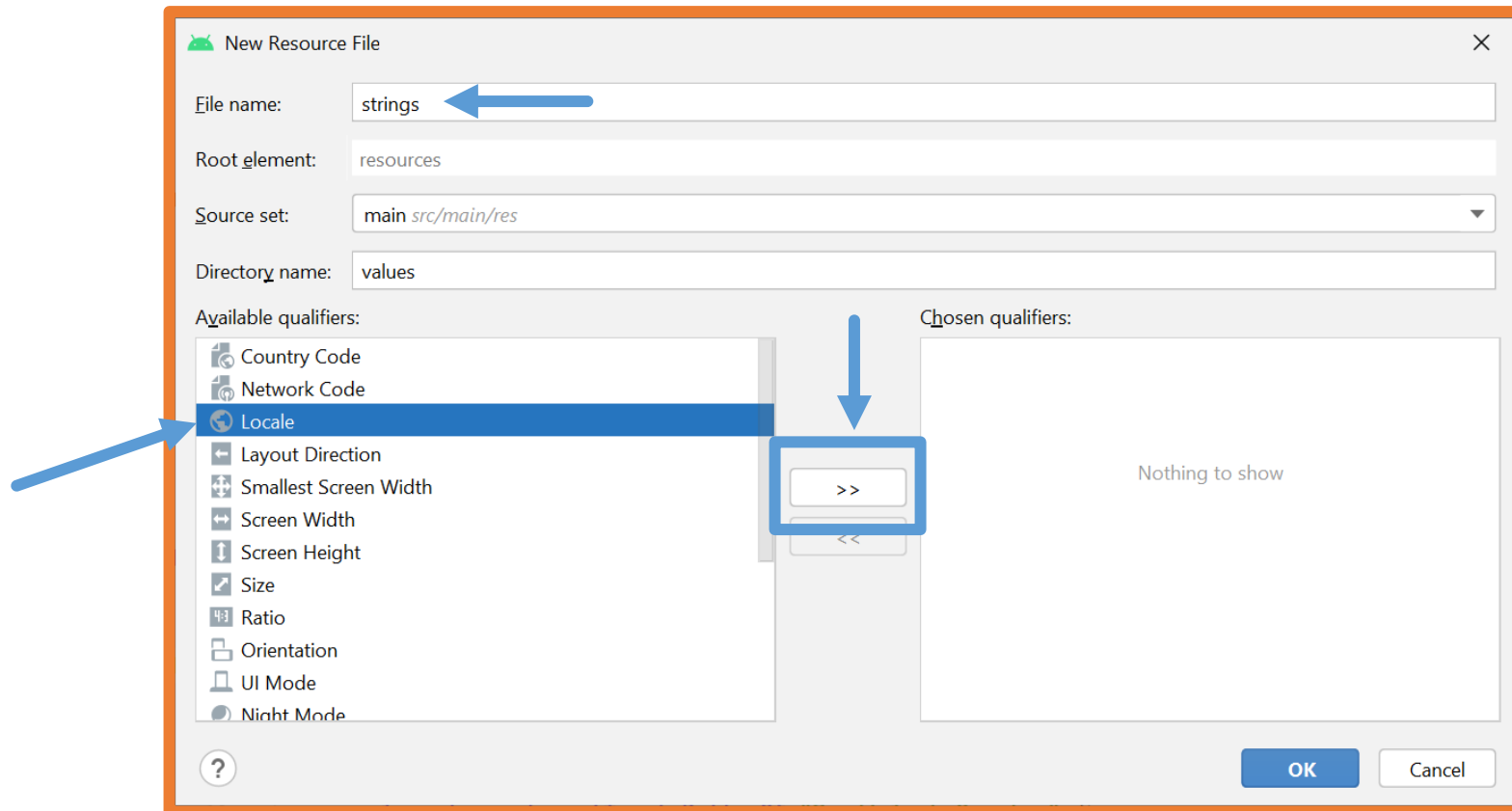
Clic derecho sobre la carpeta **values** → **New** → **Values Resource File**



4.- Aplicación multi idioma

Nombre de archivo: **strings**

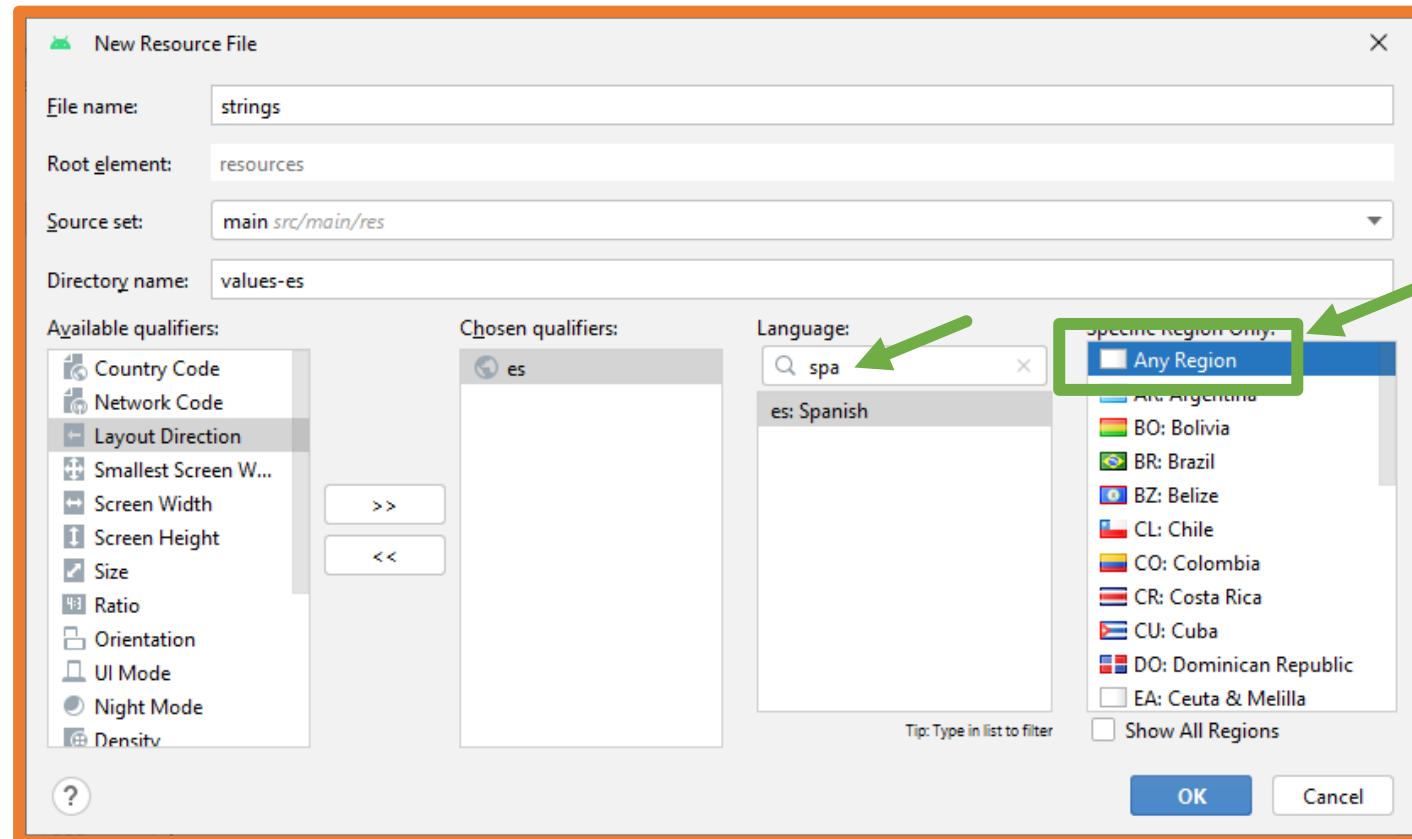
Seleccionar **Locale** y pulsar el botón >>



4.- Aplicación multi idioma

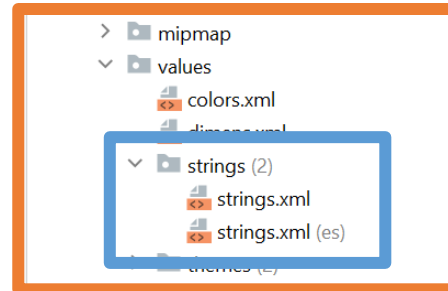
Seleccionar el idioma y región deseada.

Pulsar el botón **OK**

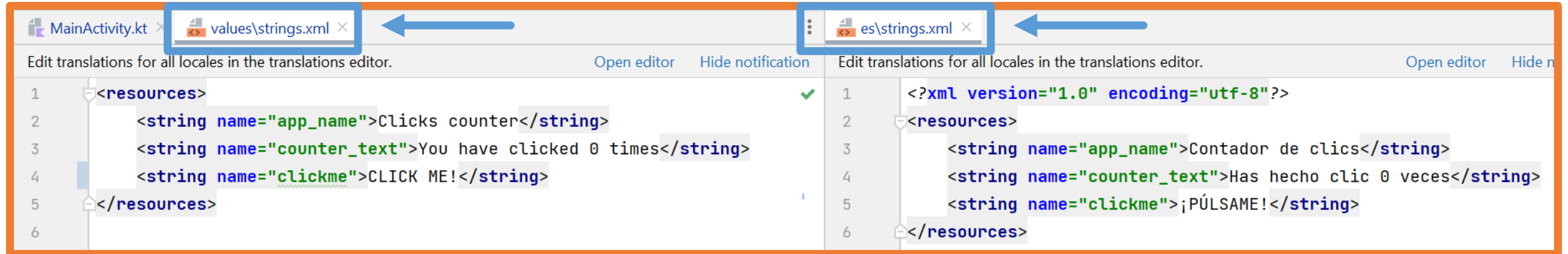


4.- Aplicación multi idioma

En el inspector del proyecto se puede ver cómo se ha creado un segundo archivo para centralizar cadenas:

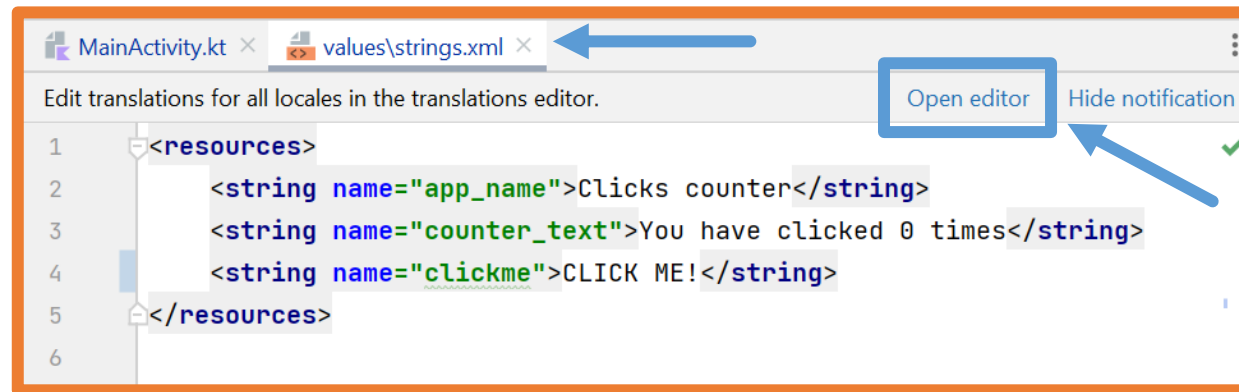


El siguiente paso es replicar el contenido del primer archivo strings.xml al segundo y traducir el valor de las cadenas:

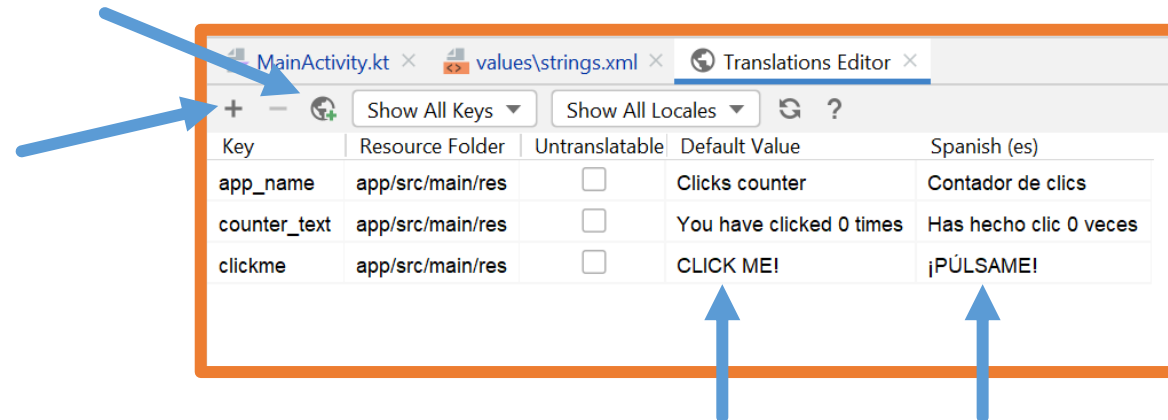


4.- Aplicación multi idioma

Mediante el editor de idiomas también se pueden añadir cadenas para los diferentes idiomas e incluso añadir nuevos archivos **strings.xml**:

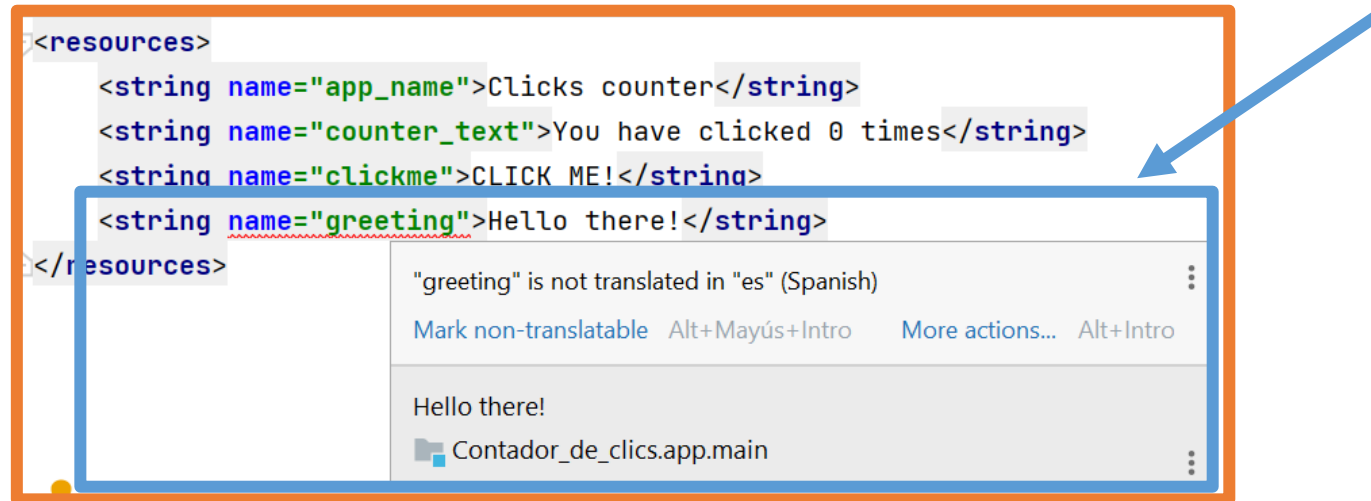


En el editor aparecen tantas columnas como archivos strings.xml haya:



4.- Aplicación multi idioma

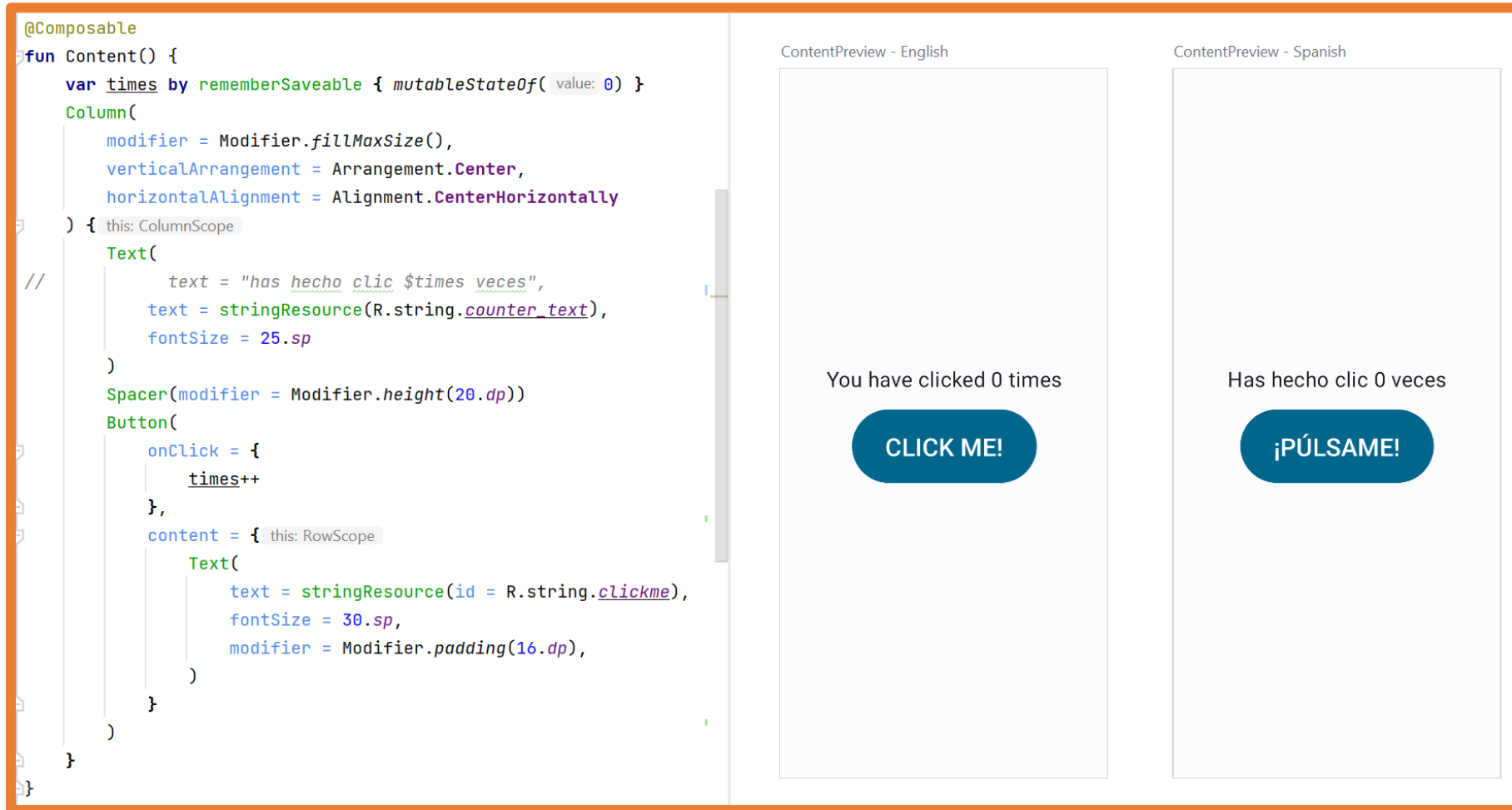
En el momento en el que se tienen diferentes archivos strings.xml para diferentes idiomas, al añadir una cadena a uno de los archivos Android Studio avisará que se debe traducir a los otros idiomas.



Si una cadena no se traduce a algún idioma Android Studio mostrará la cadena definida en el archivo strings.xml por defecto.

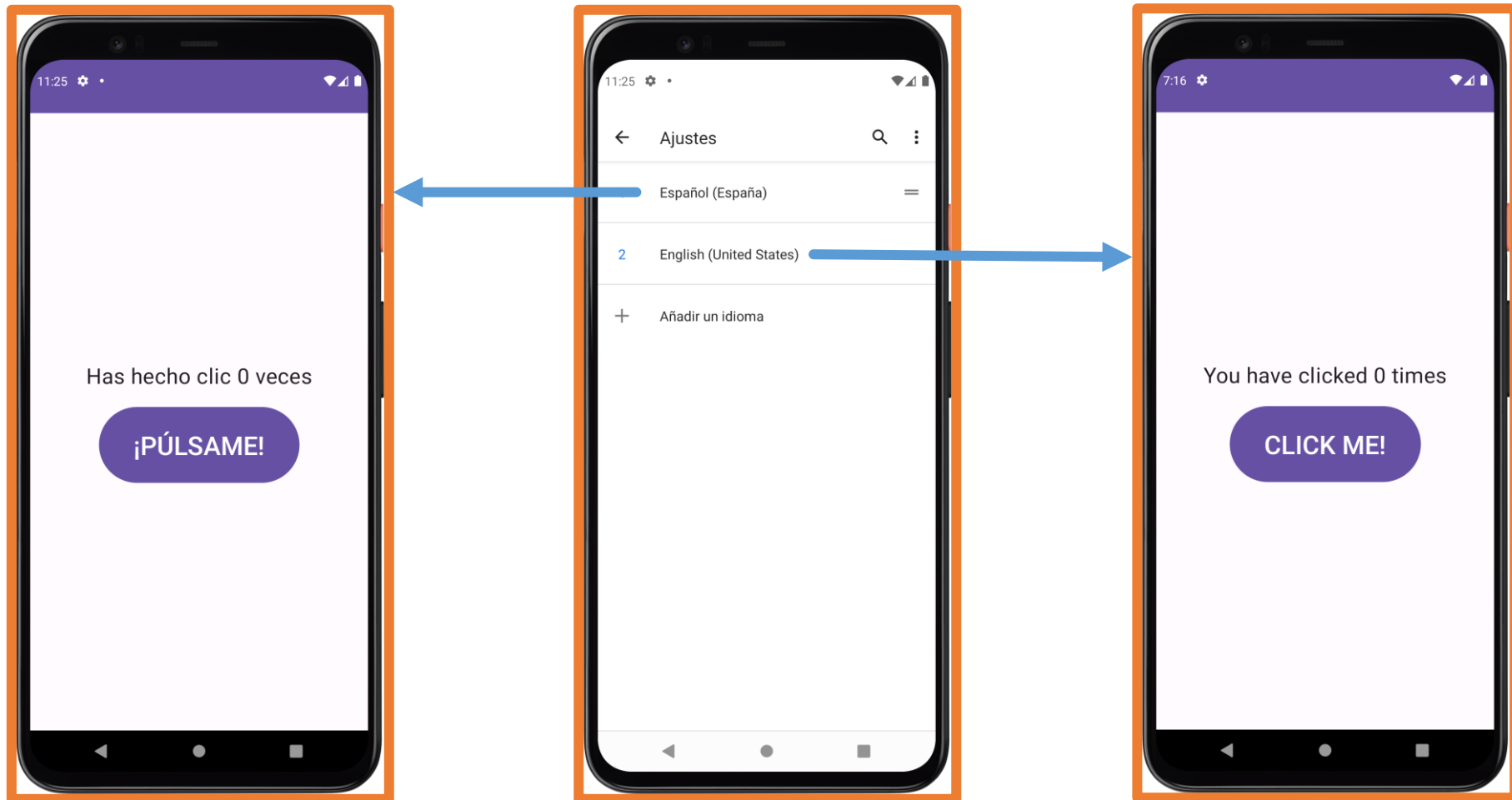
4.- Aplicación multi idioma

El resultado se puede ver con las previsualizaciones @Preview:



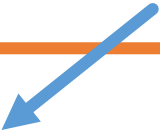
4.- Aplicación multi idioma

Y también se puede comprobar con el emulador:



5.- Valores centralizados que aceptan parámetros

En la aplicación **Contador de clics** se tiene la necesidad de centralizar la siguiente cadena:



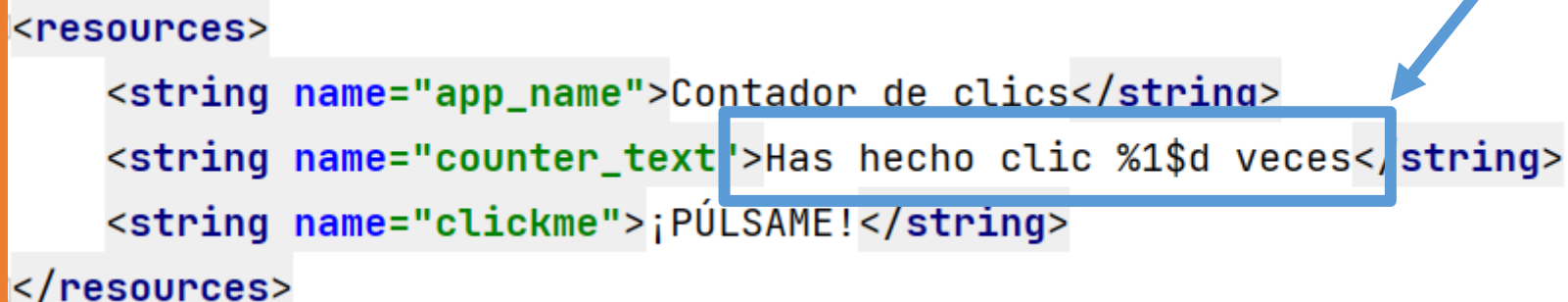
```
Text(  
    text = "Has hecho clic $times veces",  
    fontSize = 25.sp  
)
```

Esta cadena utiliza una variable por lo que a la hora de centralizar dicha cadena se debe pasar ese valor como parámetro.

5.- Valores centralizados que aceptan parámetros

El formato para declarar parámetros es **%NºParámetro\$Tipo**, siendo los tipos de parámetros: s (cadenas), d (números).

```
<resources>
    <string name="app_name">Contador de clics</string>
    <string name="counter_text">Has hecho clic %1$d veces</string>
    <string name="clickme">¡PÚLSAME!</string>
</resources>
```



Por ejemplo, si se quieren recibir varios parámetros:

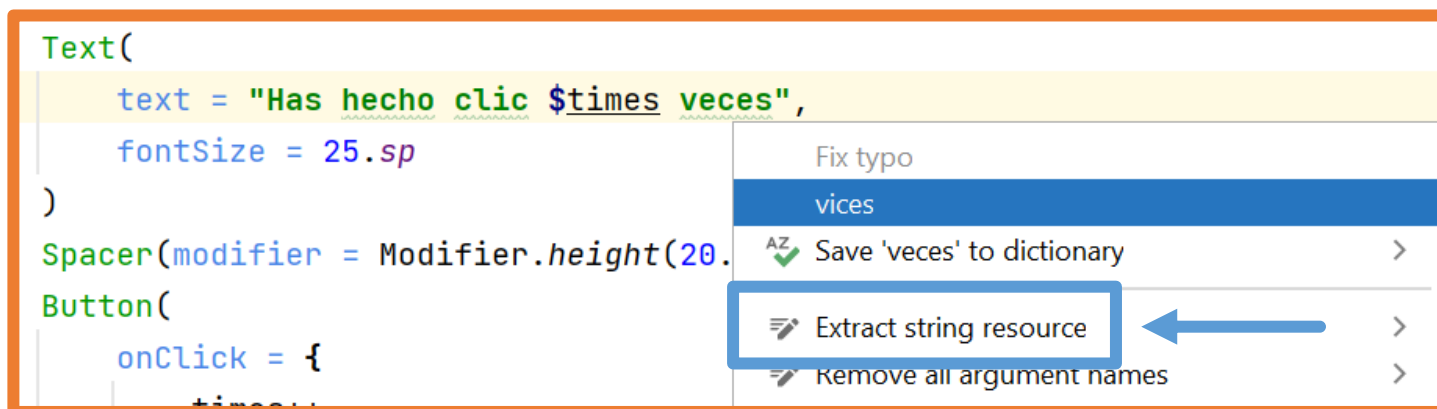
```
<string name="user_points_text">Hola %1$s, has conseguido %2$d</string>
```

En las últimas versiones de Android Studio se puede indicar siempre que el tipo de dato es s (cadena) ya que Android Studio se encarga de convertir automáticamente el dato a String.

5.- Valores centralizados que aceptan parámetros

Android Studio ayuda a crear estos valores centralizados.

Con el cursor encima de la **string hardcoded** se muestra la ayuda contextual con clic derecho o con ALT+INTRO y se selecciona la opción **Extract string resource**:

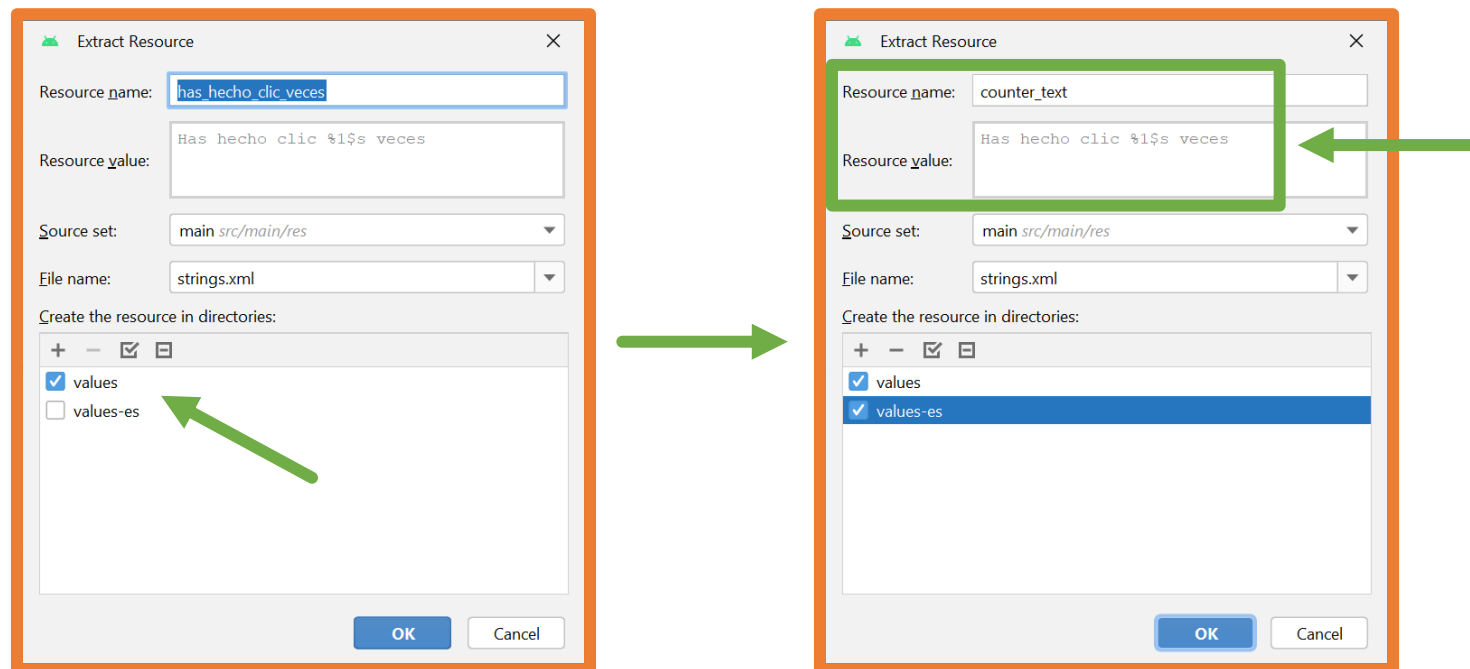


5.- Valores centralizados que aceptan parámetros

Android Studio ayuda a crear estos valores centralizados.

Con el cursor encima de la cadena hardcodeada se muestra la ayuda contextual con clic derecho o con ALT+INTRO y se selecciona la opción **Extract string resource**.

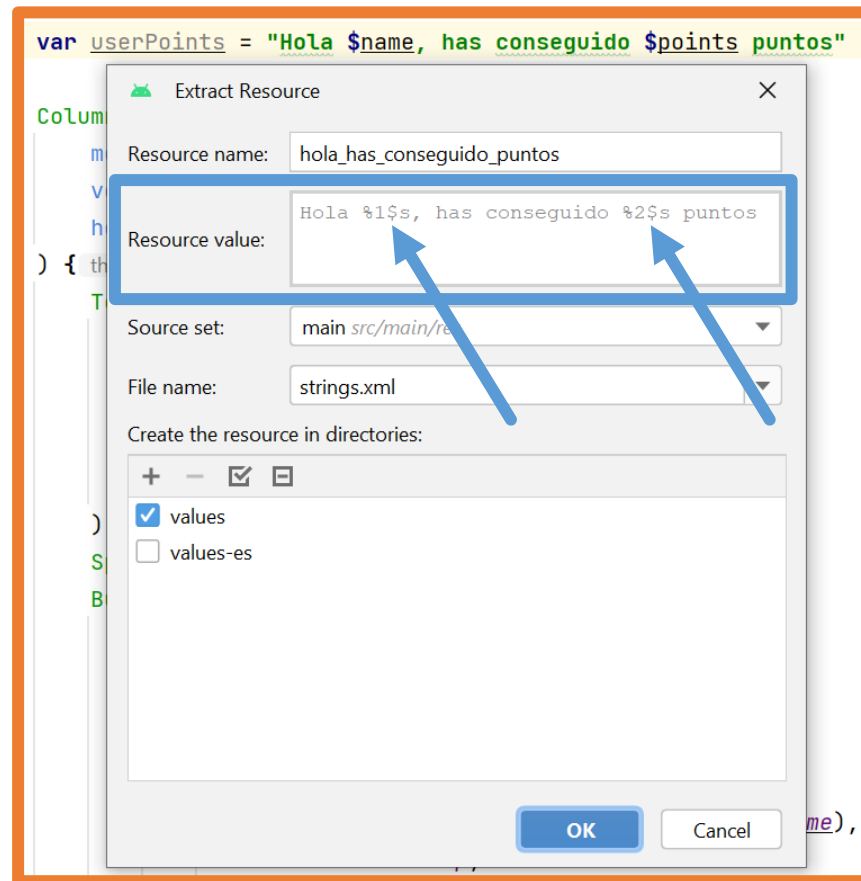
Si se tienen varios archivos strings.xml se puede marcar que se añada el valor a todos los archivos y una vez se añada solo habrá que traducir el valor en cada archivo.



5.- Valores centralizados que aceptan parámetros

Android Studio ayuda a crear estos valores centralizados.

Si la cadena tiene varios parámetros Android Studio los reconocerá:



5.- Valores centralizados que aceptan parámetros

Ahora desde Kotlin en la función **stringResource** se podrán pasar esos parámetros:

```
Text(  
    text = stringResource(  
        R.string.counter_text,  
        times  
    ),  
    fontSize = 25.sp  
)
```

```
var userPoints = stringResource(  
    R.string.user_points,  
    name,  
    points  
)
```

IMPORTANTE

La traducción de aplicaciones tiene un gran inconveniente ya que la misma frase no tiene la misma longitud en todos los idiomas.

Se debe realizar un buen estudio de la aplicación y de los diferentes idiomas para que la aplicación se muestre de manera similar en todos los idiomas en los que se distribuya.

6.- Orientaciones vertical y horizontal

En los dispositivos fijos como televisiones o automóviles no tiene sentido cambiar la orientación, de hecho directamente no disponen del acelerómetro para detectarla.

La mayoría de dispositivos móviles Android (móviles y tablets) permiten cambiar la orientación de la pantalla entre vertical y horizontal.

Existen excepciones como los smartWatch que siendo móviles generalmente solo tienen una orientación.

6.- Orientaciones vertical y horizontal

Si se desarrolla una aplicación para móvil o tablet **es importante tener en cuenta el diseño en las dos orientaciones.**

Gracias al uso de Jetpack Compose los componentes se van a situar ocupando la pantalla tal y como se definan.

Por esta razón, en principio todas las aplicaciones se visualizarán correctamente indistintamente del tamaño y la orientación de la pantalla.

Más adelante se verá cómo mostrar diferentes componentes o los mismos componentes con diferente estilo según el tamaño de pantalla para crear una mejor experiencia de usuario.

6.- Orientaciones vertical y horizontal

Es posible que se quiera diseñar una aplicación que no permita cambiar la orientación.

Para ello en el archivo **AndroidManifest.xml** se debe añadir la siguiente propiedad a la Activity:

`android:screenOrientation="ORIENTACIÓN"`

Esta configuración se debe hacer para todas las Activities de la aplicación en las que se quiera bloquear la orientación.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.ContadorDeClicks"
        tools:targetApi="31">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="portrait"
            android:label="@string/app_name"
            android:theme="@style/Theme.ContadorDeClicks">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```


7.- Values Resource Files

Hasta este punto se han utilizado los **archivos de recursos con valores** para centralizar valores (strings.xml, colors.xml y dims.xml).

Estos archivos también se pueden utilizar para que se carguen dependiendo de la configuración del dispositivo.

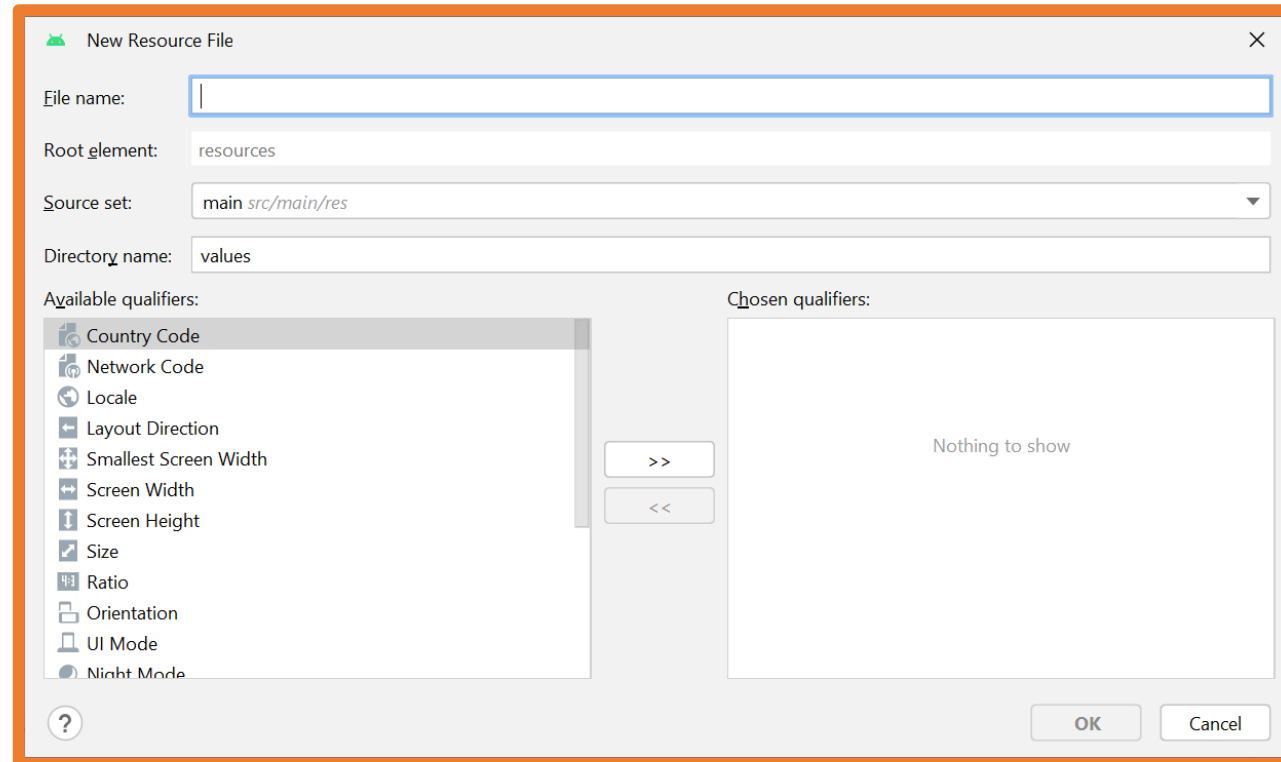
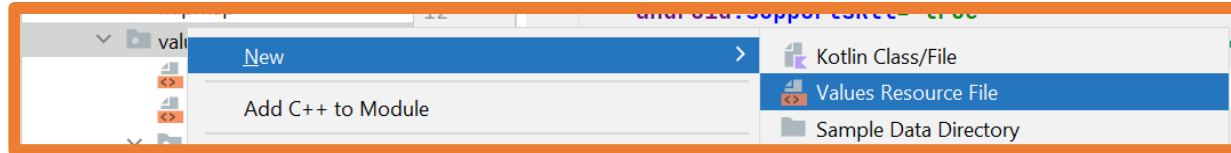
Por ejemplo, se puede cargar un archivo colors.xml cuando la orientación es vertical (portrait) y otro diferente cuando la orientación es horizontal (landscape). Igual que ocurre con los archivos strings.xml dependiendo del idioma.

Con la manera antigua de implementar las interfaces de usuario mediante archivos XML esta configuración tenía mucho más sentido.

A continuación, se va a estudiar cómo crear estos archivos.

7.- Values Resource Files

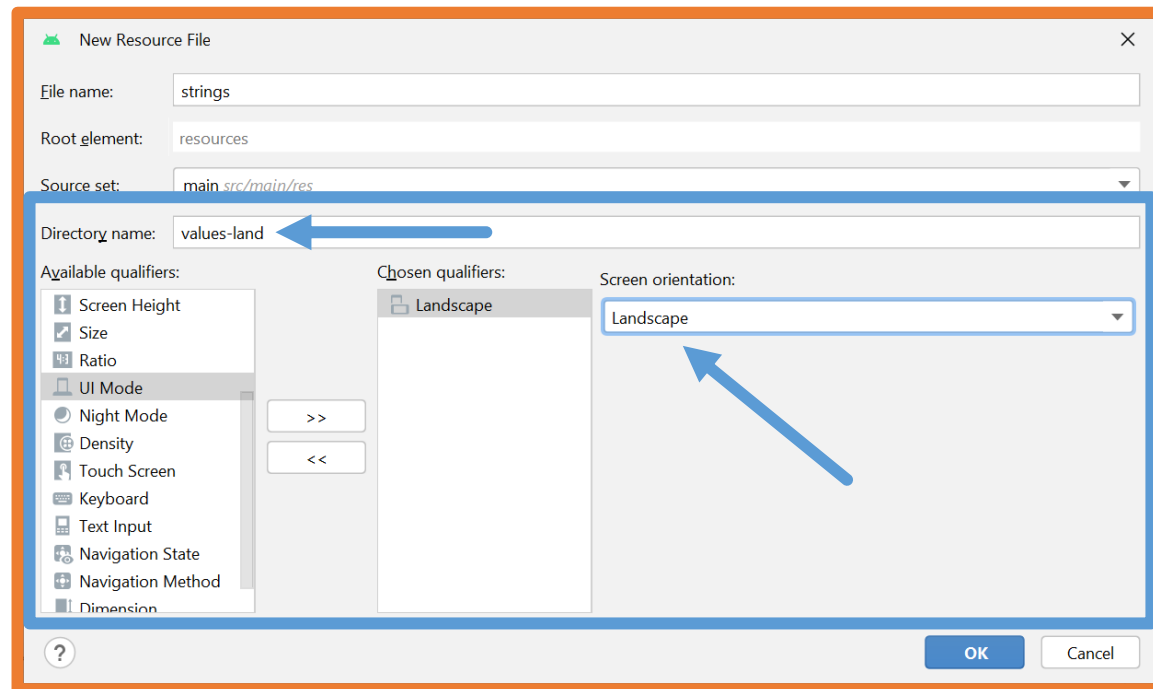
Sobre la carpeta **values** se hace clic con el botón derecho y se selecciona la opción **Values Resource File**.



7.- Values Resource Files

El funcionamiento de estos archivos es el mismo que el comportamiento de los archivos strings.xml.

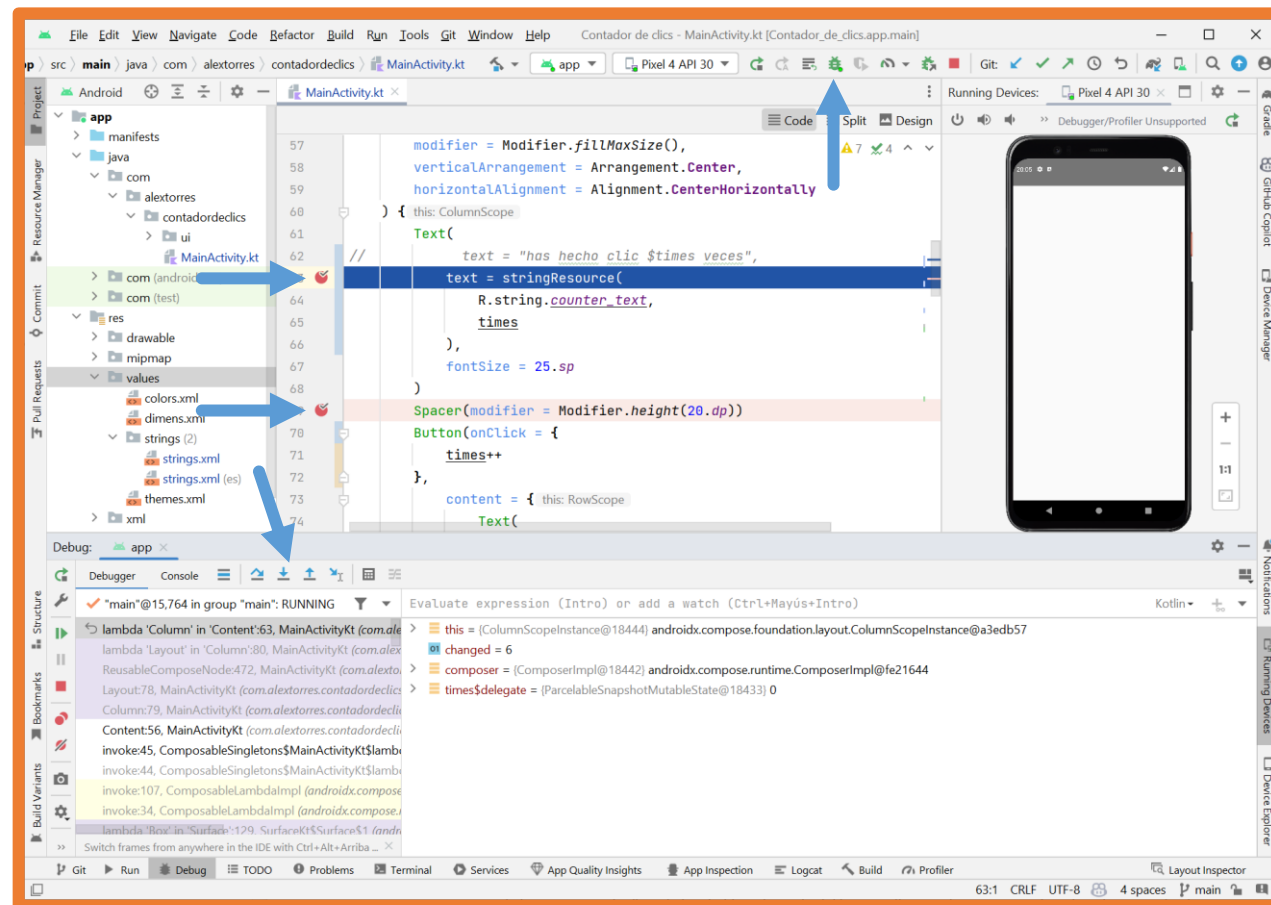
Se creará el archivo en una carpeta que Android cargará según la configuración del dispositivo.



8.- Debug

Android Studio dispone de un **modo depuración** que funciona como en el resto de IDE's.

Se pueden configurar **puntos de ruptura**, ejecutar en modo depuración, avanzar paso a paso...

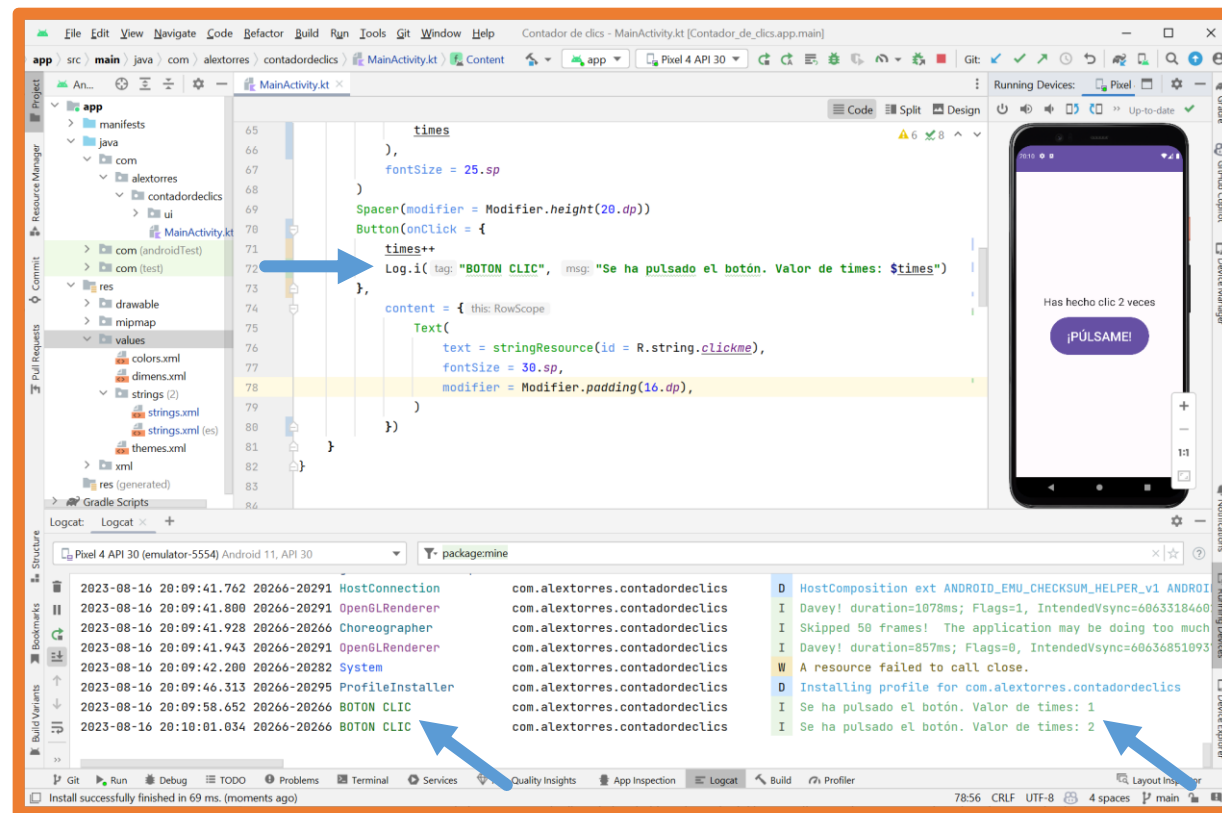


8.- Debug

Para comprobaciones más básicas se puede utilizar la clase **Log** de la siguiente manera:

```
Log.i("ETIQUETA", "MENSAJE")
```

De esta manera, el mensaje aparecerá en la sección **Logcat** de Android Studio.



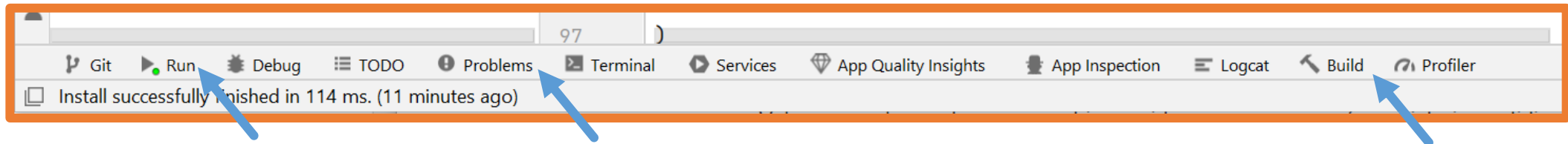
9.- Actualizaciones de Android Studio

Software **Android Studio** recibe actualizaciones que pueden ser de diferentes componentes:

- Android Studio.
- SDK de Android.
- JDK.
- Plugins.
- Gradle.
- Dependencias.

En ocasiones cuando se actualizan componentes hay proyectos que dejan de funcionar hasta que no se configure correctamente.

La manera de mostrar los error dependerá del tipo de actualización que lo ha provocado.



Según el tipo de error la solución se realizará de una manera u otra.

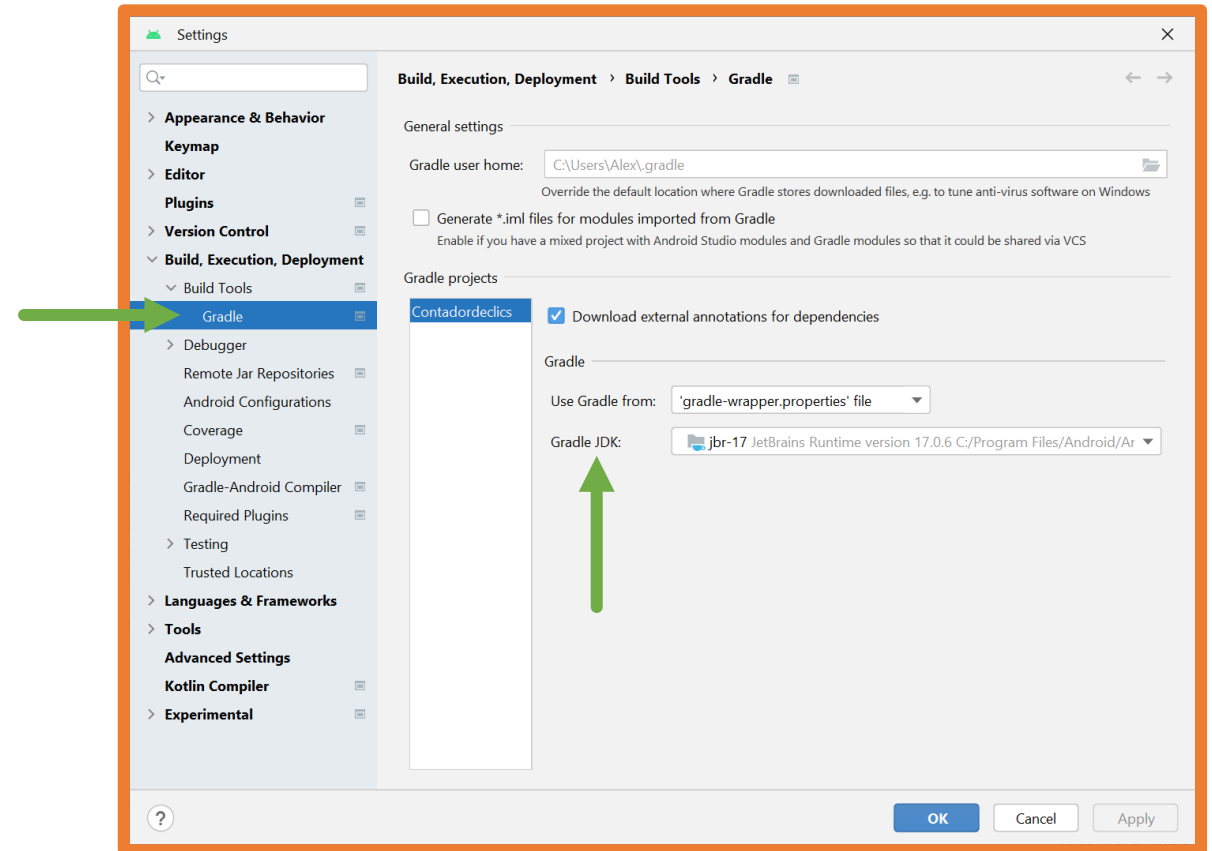
9.- Actualizaciones de Android Studio

Error versión de Java:

Para solucionar el error de versión de Java se debe ir a la configuración:

File → Settings (CTRL+ALT+S)

Y seleccionar el JDK correcto.



9.- Actualizaciones de Android Studio

Error **appcompat**:

En las aplicaciones diseñadas XML se utiliza la librería **appcompat**.

Si se está desarrollando una aplicación que mezcla XML y Jetpack Compose, por ejemplo si se están migrando de XML a Jetpack Compose también se utiliza esa librería.

En ocasiones al actualizar componentes se debe cambiar la versión de la dependencia, esto se configura en el archivo **build.gradle (Module: app)**:

```
dependencies {  
    implementation 'androidx.core:core-ktx:1.7.0'  
    implementation 'androidx.appcompat:appcompat:1.6.0' ←  
    implementation 'com.google.android.material:material:1.8.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
```


Práctica

Actividad 3

Contador de pádel.